



React FSD Продвинутая архитектура для крупных приложений

Модуль 1 - Введение в FSD

Урок 1 - Почему стандартная структура компонентов не масштабируется

Шаг 01:

Миссия 1

Вы работаете в стартапе, где проект начинался с пары компонентов. Сейчас их стало больше 200. Почему простая структура «components/» перестала работать?

Подсказка: Подумайте, что происходит, когда компонентов становится очень много.

Ответ:

- **Трудно поддерживать и находить компоненты в огромной папке**
- Компоненты автоматически становятся медленными
- React перестает работать при большом количестве файлов
- Это запрещено стандартами JavaScript

Шаг 02:

Миссия 2

На проекте «Магазинчик онлайн» каждый модуль хранит и UI, и бизнес-логику вместе. В чем главная проблема?

Подсказка: Подумайте о разделении ответственности.

Ответ:

- **Нарушается принцип разделения ответственности**
- UI перестает обновляться
- JSX нельзя хранить с логикой
- Нельзя писать тесты для компонентов

🔗 **Шаг 03:**

Миссия 3

Разработчик Алиса жалуется: «Я не знаю, в каком файле искать обработку запроса». Почему это часто случается в проектах с плоской структурой?

Подсказка: Подумайте о связи между бизнес-логикой и компонентами.

Ответ:

- **Логика и компоненты перемешаны и раскиданы по разным местам**
- Файлы React нельзя открывать в редакторе
- Проблема в настройках Webpack
- API-запросы должны быть только на сервере

🔗 **Шаг 04:**

Миссия 4

Разработчики спорят: стоит ли класть все хуки в одну папку hooks/? Что здесь может пойти не так?

Подсказка: Вспомните про масштабирование — чем больше кода, тем сложнее навигация.

Ответ:

- **Через время папка превращается в свалку с сотнями хуков**
- Хуки перестают работать вне отдельной папки
- React блокирует повторное использование хуков
- Хуки обязательно должны храниться в компоненте

🔗 **Шаг 05:**

Миссия 5

В проекте «Форум друзей» появились десятки одинаковых кнопок с разными стилями. Почему это признак плохой структуры?

Подсказка: Подумайте о переиспользовании кода.

Ответ:

- **Нет общего слоя для переиспользуемых UI-компонентов**
- CSS нельзя подключать в React
- Компоненты React должны быть только функциональными
- Проблема связана с версией Node.js

🔗 **Шаг 06:**

Миссия 6

На митапе разработчики обсуждают: «Почему баги начинают повторяться в разных местах кода?» Как это связано с плоской структурой?

Подсказка: Подумайте, что будет, если логику копировать в разные компоненты.

Ответ:

- **Повторяющийся код размножается и усложняет поддержку**
- React запрещает исправлять баги в разных местах
- Сборка перестает работать при копировании кода
- Это проблема редактора, а не структуры

Шаг 07:

Миссия 7

Команда решила добавить фичу «Тёмная тема». Но оказалось, что стили разбросаны по разным файлам компонентов. Что это показывает?

Подсказка: Подумайте, как удобно обновлять фичи, если все части хранятся вместе.

Ответ:

- **Нет фиче-ориентированного подхода к организации кода**
- CSS не поддерживает тёмные темы
- React-компоненты нельзя стилизовать
- Фичи всегда добавляются только на сервере

Шаг 08:

Миссия 8

На проекте «Календарь команд» новый разработчик тратит часы, чтобы понять, где лежит логика уведомлений. Почему?

Подсказка: Вспомните о принципе прозрачности структуры.

Ответ:

- **Структура кода не отражает доменные области и фичи**
- React запрещает уведомления
- Файлы нельзя группировать в папки
- В проекте нет ESLint

Шаг 09:

Миссия 9

 Сопоставьте проблемы плоской структуры с их последствиями.

Подсказка: Обратите внимание: проблемы архитектуры почти всегда ведут к росту сложности и багов.

Ответ:

- **Сотни файлов в одной папке** — Сложно найти нужный компонент
- **Дублирование логики** — Исправления нужно делать в десятках мест
- **Нет разделения ответственности** — UI и бизнес-логика переплетены
- **Разбросанные стили** — Невозможно централизованно менять оформление

Шаг 10:

Миссия 10

 Сопоставьте типичные жалобы разработчиков с их корнями в архитектуре.

Подсказка: Вспомните реальные примеры из проектов, где архитектура не масштабировалась.

Ответ:

- **«Не могу найти нужный код»** — Нет структуры по фичам
- **«Опять одни и те же баги»** — Копирование бизнес-логики
- **«Трудно подключить нового разработчика»** — Слишком запутанная структура
- **«Добавление новой функции ломает старые»** — Перемешивание ответственности

Шаг 11:

Миссия 11

 Сопоставьте причины архитектурных проблем с их признаками.

Подсказка: Причина всегда проявляется через симптомы в коде.

Ответ:

- **Нет общей библиотеки UI** — Разные стили у одинаковых элементов
- **Все хуки в одной папке** — Невозможно ориентироваться при большом числе хуков
- **Бизнес-логика в компонентах** — Компоненты сложно тестировать
- **Нет разграничения по доменам** — Непрозрачность структуры

Шаг 12:

Миссия 12

 Сопоставьте элементы кода с проблемами, возникающими при плоской структуре.

Подсказка: Сфокусируйтесь на конкретных артефактах: компоненты, стили, хуки.

Ответ:

- **Компоненты** — Могут содержать дублирующийся код
- **Хуки** — Теряются в одной общей папке
- **API-запросы** — Разбросаны по разным компонентам
- **Стили** — Невозможно централизованно управлять

Шаг 13:

Миссия 13

 Сопоставьте архитектурные ошибки с их долгосрочными рисками.

Подсказка: Подумайте о поддержке и масштабировании.

Ответ:

- **Огромная папка components/** — Рост времени на поиск и поддержку
- **Нет фичевых модулей** — Сложность добавления новых функций
- **Перемешивание логики и UI** — Трудности при тестировании
- **Нет единого стиля оформления** — Разнородный интерфейс

Шаг 14:

Миссия 14

Расположите шаги эволюции проекта от простой структуры к кризису.

Подсказка: Вспомните, как проекты обычно растут.

Ответ:

- Создание первых компонентов в одной папке
- Рост числа файлов и перемешивание логики
- Появление дублирующегося кода

- Сложности поддержки и онбординга новых разработчиков

Шаг 15:

Миссия 15

Расположите действия разработчика при добавлении новой фичи в проект с плохой структурой.

Подсказка: Подумайте, что приходится делать, когда нет модульности.

Ответ:

- Поиск логики в разных файлах
- Копирование кода из других компонентов
- Изменение стилей в нескольких местах
- Ручное исправление багов в старых частях

Шаг 16:

Миссия 16

Расположите проблемы в порядке их проявления при росте проекта.

Подсказка: Начинается с мелочей, а заканчивается серьёзными кризисами.

Ответ:

- Небольшие неудобства при поиске файлов
- Появление дублирования кода
- Рост времени на исправление багов
- Трудности внедрения новых фич

Шаг 17:

Миссия 17

Расположите шаги, через которые проходит команда при попытке масштабировать проект со стандартной структурой.

Подсказка: Вспомните, как хаос заставляет искать архитектурные решения.

Ответ:

- Осознание, что компоненты и логика перемешаны
- Попытка организовать код через новые папки
- Увеличение сложности и путаницы
- Переход к архитектурным подходам, например FSD

Урок 2 - Основные принципы Feature-Sliced Design

Шаг 01:

Миссия 1

Аня работает в команде над крупным приложением и впервые услышала про Feature-Sliced Design. Она спрашивает: какой главный принцип лежит в основе FSD?

Подсказка: Вспомните, что FSD помогает разделять систему так, чтобы изменения в одной части не ломали остальные.

Ответ:

- **Разделение приложения по функциональным признакам (features)**
- Группировка всех компонентов только по слоям (UI, бизнес-логика)
- Разделение исключительно по размерам файлов

- Создание единого огромного модуля без разбиения

🔮 Шаг 02:

Миссия 2

Борис строит архитектуру проекта. Он сомневается: что означает принцип независимости модулей в FSD?

Подсказка: Модули должны быть связаны минимально, чтобы менять один — не приходилось ломать другой.

Ответ:

- **Каждый модуль можно изменять независимо от других**
- Все модули должны использовать одну глобальную переменную
- Модули жёстко связаны и должны меняться одновременно
- Нельзя разделять модули по функционалу

🔮 Шаг 03:

Миссия 3

Катя обсуждает с коллегой структуру проекта: где в FSD рекомендуется хранить бизнес-логику?

Подсказка: Бизнес-правила обычно находятся не в UI и не в API, а там, где живёт сама суть приложения.

Ответ:

- **В слое domain (entities/use-cases)**
- В компонентах интерфейса (UI)
- В конфигурационных файлах сборки
- В тестах

🔮 Шаг 04:

Миссия 4

Игорь столкнулся с проблемой: у него в проекте UI-компоненты напрямую ходят в API. Что нарушается в терминах FSD?

Подсказка: Подумайте, какая ответственность должна лежать на интерфейсе, а какая — на слоях данных.

Ответ:

- **Нарушение принципа разделения обязанностей (separation of concerns)**
- Нарушение принципа DRY
- Нарушение принципа единственного источника истины
- Нарушение принципа комментирования кода

🔮 Шаг 05:

Миссия 5

Оля начала работать с FSD и запуталась: зачем нужен слой shared?

Подсказка: Вспомните, где мы храним общее, чтобы его могли использовать разные части приложения.

Ответ:

- **Хранение переиспользуемых модулей (утилиты, UI-киты)**
- Место для бизнес-логики

- Место для API-запросов
- Только для конфигурации окружения

🔗 Шаг 06:

Миссия 6

Дима решил потренироваться и хочет сопоставить основные слои FSD с их назначением.

Подсказка: Подумайте, что хранит бизнес-правила, а что — только UI или инфраструктуру.

Ответ:

- **Entities** — Содержат бизнес-сущности и правила
- **Features** — Функциональные модули, реализующие сценарии
- **Shared** — Общие переиспользуемые ресурсы
- **Pages** — Композиция слоёв для целых страниц

🔗 Шаг 07:

Миссия 7

Марина хочет сопоставить принципы проектирования в FSD с их кратким описанием.

Подсказка: Сосредоточьтесь на том, зачем каждый принцип нужен при разработке крупных приложений.

Ответ:

- **Разделение ответственности** — Каждый слой выполняет свою роль
- **Изоляция модулей** — Модуль можно менять независимо
- **Явные зависимости** — Все связи между слоями прозрачны
- **Переиспользуемость** — Код можно использовать повторно

🔗 Шаг 08:

Миссия 8

Алексей разрабатывает модуль авторизации. Сопоставьте части FSD с тем, что он туда должен положить.

Подсказка: Вспомните: бизнес-логика и UI должны быть разделены, а общее должно идти в shared.

Ответ:

- **Entities** — Модель пользователя и правила авторизации
- **Features** — Форма логина и обработка входа
- **Shared** — UI-кнопки и поля ввода
- **Processes** — Общий сценарий выхода из системы

🔗 Шаг 09:

Миссия 9

Кирилл проверяет свои знания и хочет сопоставить слои FSD с типичными ошибками при их использовании.

Подсказка: Ошибки часто связаны с нарушением ответственности и неправильными зависимостями.

Ответ:

- **Entities** — Внедрение UI-компонентов внутрь бизнес-логики
- **Features** — Хранение глобальных утилит, не связанных с конкретной фичей

- **Shared** — Перенос бизнес-правил в общий код
- **Pages** — Смешивание кода всех слоёв без структуры

🔗 Шаг 10:

Миссия 10

Света хочет разобраться, чем FSD отличается от классической слоистой архитектуры. Сопоставьте различия.

Подсказка: Вспомните, что FSD ориентирован на функциональные модули, а не только на слои.

Ответ:

- **Классическая архитектура** — Структурирование только по слоям (UI, бизнес, данные)
- **FSD** — Структурирование по функциональным модулям
- **Проблема классики** — Трудности масштабирования при росте приложения
- **Преимущество FSD** — Удобство разделения командной работы

🔗 Шаг 11:

Миссия 11

Максим собирает проект и должен сопоставить инструменты FSD с их ролью.

Подсказка: Вспомните: утилиты — в shared, бизнес — в entities, страницы — в pages.

Ответ:

- **Entities** — Бизнес-логика приложения
- **Features** — Сценарии взаимодействия пользователя
- **Shared** — Общие утилиты и компоненты
- **Pages** — Полные страницы интерфейса

🔗 Шаг 12:

Миссия 12

Настя решает, как построить новый модуль по правилам FSD. Расположите шаги в правильном порядке.

Подсказка: Сначала определите бизнес-сущности, потом добавьте функциональные сценарии, а уже потом UI.

Ответ:

- Определить бизнес-сущности в entities
- Реализовать сценарии во features
- Создать общие утилиты в shared (при необходимости)
- Собрать страницу в pages

🔗 Шаг 13:

Миссия 13

Павел рефакторит старый код под FSD. Как правильно перестроить архитектуру?

Подсказка: Не начинайте с UI, сначала разложите сущности и сценарии.

Ответ:

- Выделить бизнес-сущности в entities
- Разделить функциональные модули во features
- Перенести переиспользуемые части в shared
- Собрать всё в pages

Шаг 14:

Миссия 14

Женя пишет фичу корзины покупок. Расположите действия в порядке реализации по FSD.

Подсказка: Сначала — бизнес-сущности, потом — сценарии и интерфейс.

Ответ:

- Создать сущность товара в entities
- Добавить сценарий добавления в корзину во features
- Использовать кнопки и поля из shared
- Собрать страницу корзины в pages

Шаг 15:

Миссия 15

Лена готовит документацию по FSD. Расположите утверждения в порядке объяснения новичку.

Подсказка: Сначала объясните проблему, потом решение, потом преимущества.

Ответ:

- Описание проблем классической архитектуры
- Представление концепции FSD
- Объяснение принципов разделения
- Демонстрация преимуществ на примере

Шаг 16:

Миссия 16

Виталий обучает команду FSD. Как правильно показать путь от идеи до реализации?

Подсказка: Сначала формулируем сущности, потом сценарии, потом собираем интерфейс.

Ответ:

- Формулировка сущностей в entities
- Разработка сценариев во features
- Добавление переиспользуемых частей в shared
- Формирование страниц в pages

Урок 3 - Обзор слоев App, Pages, Features, Entities, Shared

Шаг 01:

Миссия 1

Вы начали проект с друзьями и обсуждаете, как правильно разделить код. Какой слой FSD отвечает за точку входа и инициализацию приложения?

Подсказка: Этот слой обычно содержит маршрутизацию и провайдеры.

Ответ:

- App
- Pages
- Features
- Shared

Шаг 02:

Миссия 2

Команда обсуждает, где хранить общие UI-компоненты, такие как кнопки и инпуты. В каком слое FSD им место?

Подсказка: Эти элементы не зависят от предметной области и переиспользуются в разных частях проекта.

Ответ:

- **Shared**
- Entities
- Features
- Pages

Шаг 03:

Миссия 3

Разработчик Коля добавляет форму логина. В каком слое FSD должна находиться эта функциональность?

Подсказка: Логика авторизации — это не просто сущность, а действие, доступное пользователю.

Ответ:

- **Features**
- Entities
- Pages
- App

Шаг 04:

Миссия 4

Марина создает страницу профиля пользователя. В каком слое по правилам FSD должна находиться эта страница?

Подсказка: Здесь объединяются несколько фич и сущностей для отображения полной страницы.

Ответ:

- **Pages**
- App
- Features
- Entities

Шаг 05:

Миссия 5

Андрей добавляет модель пользователя и его данные. В каком слое лучше всего разместить такую сущность?

Подсказка: Этот слой описывает бизнес-сущности, которыми оперирует система.

Ответ:

- **Entities**
- Features
- Shared
- App

🔧 Шаг 06:

Миссия 6

Тимур спорит с коллегами: где должен находиться глобальный роутер приложения?

Подсказка: Это часть точки входа, которая связывает страницы и отвечает за маршрутизацию.

Ответ:

- **App**
- Pages
- Shared
- Features

🔧 Шаг 07:

Миссия 7

Сопоставьте слой FSD с его основной ролью.

Подсказка: Вспомните, какой слой ближе к пользователю, а какой — к инфраструктуре.

Ответ:

- **App** — Точка входа, инициализация, роутинг
- **Pages** — Композиция фич и сущностей в готовую страницу
- **Features** — Функциональные сценарии для пользователя
- **Entities** — Бизнес-сущности и их данные
- **Shared** — Переиспользуемые компоненты, утилиты и стили

🔧 Шаг 08:

Миссия 8

Сопоставьте ситуацию с правильным слоем.

Подсказка: Подумайте, что ближе к инфраструктуре, а что — к предметной логике.

Ответ:

- **Кнопка Submit для всех форм** — Shared
- **API-модель заказа** — Entities
- **Форма поиска товаров** — Features
- **Главная страница каталога** — Pages
- **Инициализация приложения с провайдерами** — App

🔧 Шаг 09:

Миссия 9

Сопоставьте тип разработчика с тем, каким слоем он чаще всего работает.

Подсказка: UI-разработчики ближе к кнопкам, а архитекторы — к инициализации.

Ответ:

- **Frontend-разработчик интерфейсов** — Shared
- **Фиче-разработчик** — Features
- **Архитектор приложения** — App
- **Инженер по данным** — Entities
- **Разработчик страниц** — Pages

🔗 Шаг 10:

Миссия 10

Сопоставьте слой с примером из реального проекта.

Подсказка: Вспомните: App запускает всё, а Shared — это кирпичики.

Ответ:

- **App** — index.tsx с ReactDOM.render
- **Pages** — ProfilePage со сборкой фич
- **Features** — LoginForm с обработкой авторизации
- **Entities** — UserModel с типами и API
- **Shared** — Button и Input компоненты

🔗 Шаг 11:

Миссия 11

Сопоставьте слой с его аналогией из повседневной жизни.

Подсказка: Представьте, что проект — это дом, и разные слои — это его части.

Ответ:

- **App** — Фундамент и планировка
- **Pages** — Комнаты с мебелью
- **Features** — Функции техники (например, кондиционер)
- **Entities** — Жители дома
- **Shared** — Инструменты и строительные материалы

🔗 Шаг 12:

Миссия 12

Расположите слои FSD от самого глобального до самого мелкого по уровню детализации.

Подсказка: Вспомните пирамиду: сверху точка входа, снизу — кирпичики.

Ответ:

- App
- Pages
- Features
- Entities
- Shared

🔗 Шаг 13:

Миссия 13

Расположите этапы создания новой страницы по FSD.

Подсказка: Сначала кирпичики, потом фичи, потом сборка.

Ответ:

- Создание сущностей (Entities)
- Создание переиспользуемых компонентов (Shared)
- Реализация фич (Features)
- Сборка страницы (Pages)
- Регистрация страницы в App

Шаг 14:

Миссия 14

Расположите действия команды при масштабировании проекта с использованием FSD.

Подсказка: Начинается всё с базовых слоев, заканчивается интеграцией.

Ответ:

- Создание Shared компонентов
- Добавление Entities
- Реализация Features
- Сборка Pages
- Настройка App

Шаг 15:

Миссия 15

Расположите по порядку слои, которые участвуют при отображении страницы логина.

Подсказка: Вспомните: данные пользователя — сущность, форма — фича, страница — композиция.

Ответ:

- Entities (User)
- Features (LoginForm)
- Pages (LoginPage)
- App (Router)

Шаг 16:

Миссия 16

Расположите шаги по созданию новой бизнес-фичи «Добавить товар в корзину» в архитектуре FSD.

Подсказка: Сначала данные товара, потом кнопка, потом сценарий и страница.

Ответ:

- Создать сущность Product (Entities)
- Добавить кнопку AddToCart (Shared)
- Реализовать фичу AddToCart (Features)
- Собрать страницу CartPage (Pages)
- Зарегистрировать CartPage в App

Модуль 2 - Организация слоев

Урок 1 - Слои проекта как выбрать правильную структуру

Шаг 01:

Миссия 1

Вы работаете над новым приложением для стартапа. Тимлид спрашивает: с чего начинается правильная организация слоёв в проекте?

Подсказка: Подумайте о фундаменте дома: без него всё остальное не имеет смысла.

Ответ:

- С выделения базовых слоёв (UI, бизнес-логика, данные)

- С написания красивых компонентов без структуры
- С внедрения случайных библиотек
- С хаотичного копирования кода из прошлых проектов

🔗 Шаг 02:

Миссия 2

В проекте интернет-магазина команда обсуждает, где хранить API-запросы. Какой слой для этого подходит?

Подсказка: Представьте курьера: он передаёт данные, но не решает, как их использовать.

Ответ:

- **Слой данных (services, api)**
- Слой UI (компоненты)
- Слой роутинга
- Случайный utils-файл

🔗 Шаг 03:

Миссия 3

Разработчик Лена сделала так: в UI-компоненте сразу написала SQL-запрос. Что здесь нарушено?

Подсказка: Вспомните правило: каждый слой должен заниматься своим делом.

Ответ:

- **Нарушение разделения ответственности слоёв**
- Проблема с производительностью
- Отсутствие комментариев
- Слишком короткое имя переменной

🔗 Шаг 04:

Миссия 4

Команда обсуждает: стоит ли смешивать бизнес-логику с UI? Как правильнее?

Подсказка: Думайте о слоях как о разных профессиях: дизайнер и бухгалтер выполняют разные задачи.

Ответ:

- **Нет, бизнес-логика должна быть вынесена в отдельный слой**
- Да, так проще и быстрее
- Можно, если проект маленький
- Лучше всё хранить в App.js

🔗 Шаг 05:

Миссия 5

Андрей предложил складывать все модули в папку utils . Как лучше?

Подсказка: Подумайте: хаотичная свалка или структурированное хранилище?

Ответ:

- **Использовать разделение по слоям (entities, features, shared и т.д.)**
- **Собрать всё в utils** — так проще
- Создать папку trash

- Смешать всё с public

🔗 Шаг 06:

Миссия 6

В проекте нужно подключить роутинг. К какому слою он ближе?

Подсказка: Подумайте о проводнике: он ведёт пользователя, но не решает, какие данные показывать.

Ответ:

- **К слою приложения (app)**
- К слою данных
- К слою бизнес-логики
- К utils

🔗 Шаг 07:

Миссия 7

Сопоставьте слой с его основной задачей.

Подсказка: Вспомните команду: у каждого члена своя роль.

Ответ:

- **UI** — Отображение интерфейса и взаимодействие с пользователем
- **Бизнес-логика** — Реализация правил и сценариев работы
- **Данные** — Работа с API и источниками информации
- **App** — Организация инфраструктуры приложения

🔗 Шаг 08:

Миссия 8

Соотнесите ситуацию с правильным слоем.

Подсказка: Подумайте, где логичнее всего выполнять задачу.

Ответ:

- **Кнопка вызывает модальное окно** — UI
- **Расчёт скидки по купону** — Бизнес-логика
- **Запрос списка товаров с сервера** — Данные
- **Настройка глобального роутера** — App

🔗 Шаг 09:

Миссия 9

Разработчики перепутали слои. Найдите верные пары.

Подсказка: UI — не место для SQL, а данные не должны знать про кнопки.

Ответ:

- **UI-компонент** — Рендерит интерфейс
- **API service** — Отправляет запросы
- **Функция calculateTotal** — Считает сумму заказа
- **App router** — Отвечает за маршрутизацию

🧭 Шаг 10:

Миссия 10

Сопоставьте названия папок с их назначением.

Подсказка: Вспомните FSD-структуру.

Ответ:

- **entities** — Базовые бизнес-сущности
- **features** — Функции, объединяющие сущности
- **widgets** — Группы компонентов для экрана
- **pages** — Отдельные страницы приложения

🧭 Шаг 11:

Миссия 11

Найдите правильные связи.

Подсказка: Слоёный торт — хорошая метафора: каждый слой имеет своё место.

Ответ:

- **UI** — Компоненты React
- **Business** — useCases
- **Data** — API calls
- **App** — Providers и конфигурация

🧭 Шаг 12:

Миссия 12

Свяжите ошибку с её последствием.

Подсказка: Путаница в слоях ведёт к хаосу.

Ответ:

- **UI вызывает напрямую базу данных** — Сложность поддержки
- **Business знает о роутинге** — Нарушение принципа изоляции
- **API вызывает компоненты UI** — Непредсказуемое поведение
- **App хранит бизнес-логику** — Трудности с переносом кода

🧭 Шаг 13:

Миссия 13

Расставьте шаги настройки архитектуры в правильном порядке.

Подсказка: Думайте как архитектор: сначала фундамент, потом этажи.

Ответ:

- Создать базовую структуру папок (entities, features, pages)
- Подключить роутинг и провайдеры
- Разместить бизнес-логику
- Добавить UI-компоненты

🧭 Шаг 14:

Миссия 14

Определите порядок взаимодействия между слоями при загрузке страницы.

Подсказка: Данные приходят снизу, отображение — сверху.

Ответ:

- UI рендерит страницу
- Бизнес-логика обрабатывает данные
- API получает данные
- App инициализирует контекст

💡 **Шаг 15:**

Миссия 15

Расставьте уровни FSD-архитектуры от самого низкого к верхнему.

Подсказка: Вспомните, где находятся данные, а где — страницы.

Ответ:

- entities
- features
- widgets
- pages

💡 **Шаг 16:**

Миссия 16

Расставьте шаги по рефакторингу проекта на FSD.

Подсказка: Сначала вынос, потом организация, затем доработка.

Ответ:

- Вынести модули из utils в слои
- Организовать бизнес-логику
- Настроить глобальные провайдеры
- Разделить UI на компоненты

💡 **Шаг 17:**

Миссия 17

Расставьте действия команды при запуске нового крупного проекта.

Подсказка: Архитектура должна появиться до кода.

Ответ:

- Обсудить и утвердить архитектуру слоёв
- Создать каркас проекта
- Подключить инфраструктуру (роутинг, store)
- Реализовать бизнес-логику и UI

Урок 2 - Entities хранение бизнес-логики и моделей

💡 **Шаг 01:**

Миссия 1

Андрей проектирует новое приложение для стартапа и задумывается, где хранить бизнес-логику. В каком слое FSD должна располагаться основная бизнес-логика?

Подсказка: Этот слой отвечает за правила игры внутри предметной области, а не за интерфейс или инфраструктуру.

Ответ:

- **Entities**
- Features
- Shared
- Pages

 **Шаг 02:**

Миссия 2

Марина пишет модуль расчета комиссии и сомневается, в каком слое хранить формулу. Где ей лучше разместить этот код?

Подсказка: Этот слой — сердце бизнес-логики, он не должен знать о кнопках и API.

Ответ:

- **B Entities**
- B UI
- B Features
- B Widgets

 **Шаг 03:**

Миссия 3

Иван разрабатывает модель User . Где должен храниться объект с правилами валидации и методами вычислений?

Подсказка: В этом слое находятся сущности и их правила.

Ответ:

- **B Entities/User**
- B Features/Login
- B Pages/Home
- B Shared/UI

 **Шаг 04:**

Миссия 4

Алина реализует функционал корзины. Где нужно хранить бизнес-логику по расчету итоговой суммы?

Подсказка: Подумайте: кнопки для добавления в корзину — это Features, но сами расчеты должны жить глубже.

Ответ:

- **B Entities/Cart**
- B Features/AddToCart
- B Shared/API
- B Widgets/Header

 **Шаг 05:**

Миссия 5

Сергей обсуждает архитектуру и утверждает, что бизнес-правила должны быть в Features . Прав ли он?

Подсказка: Features — это надстройка, а бизнес-правила должны находиться ниже.

Ответ:

- Да, это верно
- **Нет, бизнес-правила в Entities**

Шаг 06:

Миссия 6

Сопоставьте слой с его задачей.

Подсказка: Вспомните, что UI показывает, Features связывают действия, а Entities содержат бизнес-логику.

Ответ:

- **Entities** — Хранение бизнес-логики и моделей
- **Features** — Функциональность для пользователя, собранная из сущностей
- **UI** — Компоненты интерфейса
- **Shared** — Общие утилиты и библиотеки

Шаг 07:

Миссия 7

Анна проектирует модуль заказов. Сопоставьте объекты с соответствующим слоем.

Подсказка: Модели и правила — Entities, а кнопки и действия — Features.

Ответ:

- **Модель заказа** — Entities/Order
- **Кнопка оформления** — Features/Order
- **Таблица заказов** — UI
- **Функция логирования** — Shared/Logger

Шаг 08:

Миссия 8

Команда обсуждает слои. Сопоставьте утверждение с правильным слоем.

Подсказка: Слой Entities ничего не знает о внешнем окружении.

Ответ:

- **Содержит бизнес-правила** — Entities
- **Собирает действия для пользователя** — Features
- **Отвечает за внешний вид** — UI
- **Содержит базовые вспомогательные функции** — Shared

Шаг 09:

Миссия 9

Расставьте шаги по проектированию Entities в правильном порядке.

Подсказка: Сначала определяется предметная область, потом создаются модели, затем добавляются правила.

Ответ:

- Определение предметной области
- Создание моделей
- Добавление бизнес-правил

- Интеграция с Features

Шаг 10:

Миссия 10

Расположите действия команды при вынесении бизнес-логики в Entities.

Подсказка: Логику сначала находят, потом выносят, потом тестируют.

Ответ:

- Поиск бизнес-логики в Features
- Создание моделей в Entities
- Перенос логики в модели
- Проверка тестами

Шаг 11:

Миссия 11

Упорядочьте этапы интеграции Entities в проект.

Подсказка: Логично начинать с моделирования и заканчивать использованием в UI.

Ответ:

- Проектирование моделей
- Реализация бизнес-правил
- Подключение в Features
- Использование в UI

Шаг 12:

Миссия 12

Расположите порядок работы программиста с Entities при добавлении новой сущности.

Подсказка: Сначала проектируется структура, потом бизнес-логика, потом интеграция.

Ответ:

- Определение атрибутов сущности
- Написание методов и правил
- Добавление в Features
- Вызов в UI

Шаг 13:

Миссия 13

Расположите шаги рефакторинга проекта для корректного разделения логики в Entities.

Подсказка: Начните с анализа, а завершите тестами.

Ответ:

- Анализ кода Features
- Выделение бизнес-логики
- Перенос в Entities
- Покрытие тестами

Урок 3 - Features организация функциональных блоков

Шаг 01:

Миссия 1

Вы работаете в крупной команде и хотите организовать свои фичи по слоям. Какое из решений подходит для разделения функциональных блоков на слои в React?

Подсказка: Думайте, как разделить код так, чтобы каждый слой выполнял отдельную роль, а компоненты могли быть переиспользованы.

Ответ:

- **Использовать подход с разделением на слои: Presentation, Domain, Data и Application**
- Разделить код по компонентам по типу: классы, функции, хуки
- Использовать один слой для всего кода, чтобы уменьшить количество файлов
- Объединить все фичи в одну общую папку для упрощения навигации

Шаг 02:

Миссия 2

Вы хотите отделить логику бизнес-слоя от слоя представления. Какой подход наиболее подходит для этого?

Подсказка: Думайте о том, как изолировать логику от UI, чтобы улучшить тестируемость и поддержку.

Ответ:

- **Использование паттерна Container/Presentational для разделения логики и UI**
- Включение логики прямо в компоненты представления
- Использование исключительно классовых компонентов для организации логики
- Смешивание логики с UI для упрощения разработки

Шаг 03:

Миссия 3

 Задача: Сопоставьте слои с их назначением.

Слои:

Ответ:

- Presentation
- Domain
- Data
- Application

Назначения:

1. Отвечает за логику бизнес-приложения.
2. Обрабатывает данные и взаимодействует с внешними источниками.
3. Представляет интерфейс и взаимодействует с пользователем.
4. Организует связи между слоями и решает задачи координации.

Подсказка:

Не забывайте, что слои имеют разные задачи: от отображения данных до логики обработки.

- **Presentation** — Представляет интерфейс и взаимодействует с пользователем.

- **Domain** — Отвечает за логику бизнес-приложения.
- **Data** — Обрабатывает данные и взаимодействует с внешними источниками.
- **Application** — Организует связи между слоями и решает задачи координации.

🔮 Шаг 04:

Миссия 4

Как правильно организовать слой данных в архитектуре приложения?

Подсказка: Подумайте о том, как данные должны поступать и передаваться между слоями.

Ответ:

- **Использовать сервисы для получения данных с API и передачи их в нужные слои приложения**
- Хранить все данные в одном центральном объекте в компоненте приложения
- Передавать данные напрямую из компонентов представления в компоненты данных
- Хранить все данные в состоянии компонентов для управления ими внутри приложения

🔮 Шаг 05:

Миссия 5

Расположите слои приложения в правильном порядке взаимодействия:

1. Presentation
2. Application
3. Data
4. Domain

🔮 **Подсказка:**

Подумайте, какой слой работает с пользователем, а какой — с данными. Начните с отображения данных и закончите бизнес-логикой.

Ответ:

- Presentation
- Application
- Data
- Domain

🔮 Шаг 06:

Миссия 6

Какую роль играет слой Application в архитектуре?

Подсказка: Помните, что слой Application координирует взаимодействие других слоев.

Ответ:

- **Слой Application отвечает за координацию действий между слоями и передачу данных**
- Слой Application обрабатывает все данные и отвечает за их хранение
- Слой Application только отображает данные на экране, не взаимодействуя с другими слоями
- Слой Application предоставляет только интерфейс пользователя для взаимодействия с API

🔮 Шаг 07:

Миссия 7

Какую роль играет слой Application в архитектуре?

Подсказка: Помните, что слой Application координирует взаимодействие других слоев.

Ответ:

- **Слой Application отвечает за координацию действий между слоями и передачу данных**
- Слой Application обрабатывает все данные и отвечает за их хранение
- Слой Application только отображает данные на экране, не взаимодействуя с другими слоями
- Слой Application предоставляет только интерфейс пользователя для взаимодействия с API

Шаг 08:

Миссия 8

 Задача: Сопоставьте архитектурные принципы с их определениями.

Принципы:

Ответ:

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Dependency Inversion Principle

Определения:

1. Модуль должен быть открыт для расширений, но закрыт для изменений.
2. Модули высшего уровня не должны зависеть от модулей низшего уровня. Оба типа должны зависеть от абстракций.
3. Объекты должны быть заменяемыми на экземпляры их подтипов без нарушения правильности программы.
4. Каждый модуль должен иметь только одну ответственность.

Подсказка:

Подумайте о принципах объектно-ориентированного программирования и как они помогают создавать расширяемую архитектуру.

- **Single Responsibility Principle** — Каждый модуль должен иметь только одну ответственность.
- **Open/Closed Principle** — Модуль должен быть открыт для расширений, но закрыт для изменений.
- **Liskov Substitution Principle** — Объекты должны быть заменяемыми на экземпляры их подтипов без нарушения правильности программы.
- **Dependency Inversion Principle** — Модули высшего уровня не должны зависеть от модулей низшего уровня. Оба типа должны зависеть от абстракций.

Шаг 09:

Миссия 9

Расположите принципы SOLID в правильном порядке:

1. Single Responsibility Principle
2. Open/Closed Principle
3. Liskov Substitution Principle
4. Interface Segregation Principle

5. Dependency Inversion Principle

Подсказка:

Подумайте о логике, по которой эти принципы были разработаны. Первый принцип должен быть самым базовым и простым для понимания.

Ответ:

- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle
- Interface Segregation Principle
- Dependency Inversion Principle

Шаг 10:

Миссия 10

Какой подход можно использовать для организации слоя Domain в архитектуре?

Подсказка: Слой Domain отвечает за бизнес-логику и должен быть изолирован от других слоев.

Ответ:

- **Использовать паттерн Use Case для инкапсуляции бизнес-логики**
- Включать всю бизнес-логику прямо в компоненты интерфейса
- Использовать глобальное состояние для хранения всей бизнес-логики приложения
- Смешивать бизнес-логику с логикой представления для упрощения разработки

Шаг 11:

Миссия 11

Какой из принципов лучше всего помогает поддерживать разделение слоев в архитектуре?

Подсказка: Подумайте, как правильно изолировать один слой от другого, чтобы изменить один слой, не затронув другие.

Ответ:

- **Dependency Inversion Principle**
- Single Responsibility Principle
- Interface Segregation Principle
- Liskov Substitution Principle

Шаг 12:

Миссия 12

Какой слой отвечает за хранение данных и взаимодействие с базами данных?

Подсказка: Этот слой работает с данными и их сохранением.

Ответ:

- **Слой Infrastructure**
- Слой Presentation
- Слой Application
- Слой Domain

Шаг 13:

Миссия 13

Расположите слои архитектуры в правильном порядке для работы с данными:

1. Слой Infrastructure
2. Слой Application
3. Слой Domain
4. Слой Presentation

 Подсказка:

Данные сначала поступают в слой для их хранения, затем обрабатываются и передаются в слои для отображения.

Ответ:

- Слой Infrastructure
- Слой Application
- Слой Domain
- Слой Presentation

 Шаг 14:

Миссия 14

 **Задача:** Сопоставьте слой архитектуры с его задачей в процессе обработки данных.

Слои:

Ответ:

- Слой Presentation
- Слой Application
- Слой Infrastructure
- Слой Domain

Задачи:

1. Отображает данные пользователю.
2. Обрабатывает бизнес-логику и правила.
3. Хранит данные и взаимодействует с внешними сервисами.
4. Координирует работу системы и управляет процессами.

 Подсказка:

Подумайте, какой слой отвечает за работу с данными, а какой — за их отображение.

- **Слой Presentation** — Отображает данные пользователю.
- **Слой Application** — Координирует работу системы и управляет процессами.
- **Слой Infrastructure** — Хранит данные и взаимодействует с внешними сервисами.
- **Слой Domain** — Обрабатывает бизнес-логику и правила.

 Шаг 15:

Миссия 15

Какой слой обычно содержит код, отвечающий за взаимодействие с базой данных и сторонними API?

Подсказка: Этот слой находится между приложением и внешними системами.

Ответ:

- **Слой Infrastructure**
- Слой Application

- Слой Presentation
- Слой Domain

Шаг 16:

Миссия 16

Расположите слои архитектуры в правильном порядке, начиная с первого этапа обработки данных:

1. Слой Infrastructure
2. Слой Domain
3. Слой Application
4. Слой Presentation

Подсказка:

Данные сначала поступают на слой, который отвечает за хранение и обработку данных, а затем передаются в другие слои для использования и отображения.

Ответ:

- Слой Infrastructure
- Слой Domain
- Слой Application
- Слой Presentation

Урок 4 - Shared создание переиспользуемых компонентов

Шаг 01:

Миссия 1

Что является принципом "чистого кода" при рефакторинге?

Подсказка: Уменьшение дублирования — ключ к чистоте кода.

Ответ:

- **Вынос повторяющегося кода в отдельные функции**
- Добавление длинных комментариев
- Смешивание логики и интерфейса
- Использование непонятных сокращений

Шаг 02:

Миссия 2

Расположите шаги рефакторинга в правильном порядке:

1. Запуск тестов и проверка, что всё работает, как раньше
2. Выбор проблемного участка кода
3. Внесение улучшений (читаемость, удаление дубликатов и т.д.)
4. Поддержание функциональности без изменений

Подсказка:

Подумайте:

Ответ:

- Сначала нужно определить, где именно код требует улучшений.
- Потом выполняются изменения.

- Важно помнить, что поведение программы не должно измениться.
- И только в конце нужно убедиться с помощью тестов, что всё осталось как раньше.
- Выбор проблемного участка кода
- Внесение улучшений (читаемость, удаление дубликатов и т.д.)
- Поддержание функциональности без изменений
- Запуск тестов и проверка, что всё работает, как раньше

🔮 Шаг 03:

Миссия 3

📖 Задача: Сопоставление понятий рефакторинга

Формулировка: Сопоставьте понятие с его правильным объяснением.

Понятия:

Ответ:

- Рефакторинг
- Цель рефакторинга
- Проблема "дублирование кода"
- Проблема "плохие названия"

Определения:

1. Изменение внутренней структуры кода без изменения внешнего поведения.
2. Сделать код более понятным, удобным для поддержки и расширения.
3. Когда один и тот же фрагмент программы повторяется много раз.
4. Когда имена переменных или функций не отражают их смысл.

🔮 Подсказка:

Подумайте, зачем нужен рефакторинг: он не добавляет новые функции, а улучшает сам код. Обратите внимание на типичные проблемы — они часто связаны либо с повторением, либо с непонятными именами.

- **Рефакторинг** — Изменение внутренней структуры кода без изменения внешнего поведения.
- **Цель рефакторинга** — Сделать код более понятным, удобным для поддержки и расширения.
- **Проблема "дублирование кода"** — Когда один и тот же фрагмент программы повторяется много раз.
- **Проблема "плохие названия"** — Когда имена переменных или функций не отражают их смысл.

🔮 Шаг 04:

Миссия 4

Команда разработчиков внедряет ThemeProvider для единого оформления. Где его правильнее разместить в архитектуре FSD?

Подсказка: Подумайте, какой слой используется для глобальных универсальных инструментов, доступных всем слоям приложения.

Ответ:

- **В слое Shared**
- В слое Features
- В слое Entities

- В слое Widgets

🔮 Шаг 05:

Миссия 5

Разработчик Саша хочет вынести иконки, которые используются по всему приложению. В какой слой он должен их поместить?

Подсказка: Если компонент используется многократно и не зависит от бизнес-логики, он должен быть в универсальном месте.

Ответ:

- **Shared**
- Features
- Pages
- Entities

🔮 Шаг 06:

Миссия 6

Оля создала универсальный компонент Button , который используется на разных страницах. Где он должен находиться?

Подсказка: Кнопка не содержит бизнес-логики и нужна всем слоям приложения.

Ответ:

- **Shared**
- Features
- Widgets
- Entities

🔮 Шаг 07:

Миссия 7

Василий добавляет функцию логирования ошибок, которая должна быть доступна всем слоям. Где её разместить?

Подсказка: Глобальные утилиты должны находиться в одном месте, чтобы ими могли пользоваться все.

Ответ:

- **Shared**
- Entities
- Features
- Pages

🔮 Шаг 08:

Миссия 8

Команда внедряет кастомный хук useMediaQuery . Он не зависит от бизнес-логики. Куда его поместить?

Подсказка: Такие хуки являются переиспользуемыми инструментами, как и утилиты.

Ответ:

- **Shared**
- Widgets

- Features
- Entities

🔗 Шаг 09:

Миссия 9

Сопоставьте компонент с правильным слоем архитектуры FSD.

Подсказка: Вспомните: Shared — всё общее, Features — функционал, Entities — бизнес-сущности, Widgets — крупные блоки интерфейса.

Ответ:

- **Button** — Shared
- **LoginForm** — Features
- **User** — Entities
- **Header** — Widgets

🔗 Шаг 10:

Миссия 10

Сопоставьте инструмент или хук с правильным слоем.

Подсказка: Думайте о назначении: глобальный, функциональный, сущностный или интерфейсный.

Ответ:

- **ThemeProvider** — Shared
- **useAuth** — Features
- **ProductEntity** — Entities
- **Sidebar** — Widgets

🔗 Шаг 11:

Миссия 11

Соотнесите задачу с правильным слоем FSD.

Подсказка: Определите, общий ли это инструмент или часть бизнес-логики.

Ответ:

- **Создание функции debounce** — Shared
- **Реализация корзины покупок** — Features
- **Модель Order** — Entities
- **Компонент Footer** — Widgets

🔗 Шаг 12:

Миссия 12

Сопоставьте универсальный компонент с его назначением.

Подсказка: Shared отвечает за многократное переиспользование.

Ответ:

- **Modal** — Диалоговое окно, доступное всем частям приложения
- **Spinner** — Индикатор загрузки, используемый в разных страницах
- **Typography** — Унифицированные стили текста
- **Icon** — Графический символ для повторного использования

Шаг 13:

Миссия 13

Расставьте шаги по созданию нового компонента в Shared.

Подсказка: Начните с анализа потребности, а закончите подключением.

Ответ:

- Определить, нужен ли общий компонент
- Создать файл и структуру компонента
- Реализовать функционал и стили
- Подключить компонент в нужных местах

Шаг 14:

Миссия 14

Определите порядок действий при вынесении существующего компонента в Shared.

Подсказка: Сначала найдите повторяющийся код, затем вынесите его, убедитесь в корректности и подключите в проекте.

Ответ:

- Найти повторяющийся компонент
- Вынести компонент в Shared
- Обновить импорты в проекте
- Проверить работоспособность

Модуль 3 - Компоненты и переиспользование

Урок 1 - Atomic Design vs FSD где границы компонентов

Шаг 01:

Миссия 1

Аня — фронтенд-разработчик, и она начала новый проект. Коллега спросил её: «Ты будешь использовать Atomic Design или сразу FSD?» Что в первую очередь отличает Atomic Design от FSD?

Подсказка: Подумайте о том, что Atomic Design больше про UI-компоненты, а FSD — про организацию слоёв приложения.

Ответ:

- **Atomic Design описывает уровни UI-компонентов, а FSD** — архитектурные слои приложения
- **Atomic Design** — это паттерн для Redux, а FSD — для GraphQL
- **Atomic Design описывает DevOps-процессы, а FSD** — UI-стили
- **Atomic Design связан только с тестированием, а FSD** — с продакшн-сборкой

Шаг 02:

Миссия 2

Петя делает дизайн-систему. Вопрос: компонент Button — это какой уровень Atomic Design?

Подсказка: Вспомните: атомы — это самые простые элементы.

Ответ:

- **Атом**

- Молекула
- Организм
- Шаблон

Шаг 03:

Миссия 3

Саша обсуждает с командой FSD. Где правильнее разместить компонент «Форма входа»?

Подсказка: Подумайте, где в FSD обычно находятся законченные бизнес-фичи.

Ответ:

- **В слое features**
- В слое shared
- В слое entities
- В слое app

Шаг 04:

Миссия 4

Катя увидела, что команда путает: где хранить Avatar-компонент? Он используется в разных местах. Где его правильнее положить?

Подсказка: Подумайте, где должны жить переиспользуемые UI-компоненты.

Ответ:

- **В shared/ui**
- В features
- В entities
- В pages

Шаг 05:

Миссия 5

Игорь работает с FSD. Вопрос: компонент «UserCard» (содержит имя, фото, статус) скорее относится к какому слою?

Подсказка: Это не просто UI-деталь, а бизнес-сущность.

Ответ:

- **entities**
- features
- shared
- pages

Шаг 06:

Миссия 6

Маша спорит: Atomic Design предлагает «Организмы». Что им ближе всего соответствует в FSD?

Подсказка: Организмы — это уже сложные комбинации элементов, похожие на фичи.

Ответ:

- **features**
- entities
- shared
- app

🔮 Шаг 07:

Миссия 7

В команде обсуждают Atomic Design. «Молекула» — это что за сущность?

Подсказка: Вспомните: молекула соединяет несколько атомов, но сама ещё не сложная фича.

Ответ:

- **Комбинация атомов в небольшой компонент**
- Самая мелкая неделимая единица UI
- Готовая страница
- Группа приложений

🔮 Шаг 08:

Миссия 8

Дима пишет код и запутался: «Страница профиля» в FSD — это какой слой?

Подсказка: Подумайте, в каком месте собираются фичи и сущности в готовый экран.

Ответ:

- **pages**
- features
- entities
- shared

🔮 Шаг 09:

Миссия 9

📄 Сопоставьте уровни Atomic Design с их описанием.

Подсказка: Вспомните: от маленького к большому.

Ответ:

- **Атом** — Самый маленький, неделимый элемент интерфейса (например, кнопка)
- **Молекула** — Комбинация атомов в небольшой самостоятельный компонент
- **Организм** — Сложный компонент, объединяющий молекулы и атомы
- **Шаблон** — Структура страницы без конкретного контента

🔮 Шаг 10:

Миссия 10

Свяжите слои FSD с их назначением.

Подсказка: Вспомните, shared — для общих вещей, entities — для бизнес-сущностей, features — для завершённых действий.

Ответ:

- **shared** — Слой с переиспользуемыми утилитами и UI-компонентами
- **entities** — Слой бизнес-сущностей (например, User)
- **features** — Слой законченных пользовательских действий
- **pages** — Сборка фич и сущностей в конкретные страницы

🔮 Шаг 11:

Миссия 11

Сопоставьте элементы Atomic Design и FSD.

Подсказка: Попробуйте найти параллели: атом $\approx$ ui, организм $\approx$ feature.

Ответ:

- **Атом** — shared/ui
- **Молекула** — entities
- **Организм** — features
- **Шаблон** — pages

🔮 Шаг 12:

Миссия 12

Команда спорит: где хранить Notification-компонент? Сопоставьте компонент с правильным слоем.

Подсказка: Notification — это бизнес-сущность, а не просто кнопка.

Ответ:

- **Avatar** — shared/ui
- **Notification** — entities
- **LoginForm** — features
- **ProfilePage** — pages

🔮 Шаг 13:

Миссия 13

Сопоставьте реальные ситуации с правильным архитектурным выбором.

Подсказка: Вспомните, что FSD решает организацию кода, а Atomic Design — структуру UI.

Ответ:

- **Разработчик ищет, где хранить логику авторизации** — features
- **Дизайнер объясняет, как комбинируются кнопки и поля** — Atomic Design
- **Нужно переиспользовать компонент Input** — shared
- **Нужно показать профиль с данными пользователя** — entities

🔮 Шаг 14:

Миссия 14

Расположите уровни Atomic Design от меньшего к большему.

Подсказка: Начните с самого простого, заканчивая готовой страницей.

Ответ:

- Атом
- Молекула
- Организм
- Шаблон

🔮 Шаг 15:

Миссия 15

Упорядочите шаги работы над фичей в FSD.

Подсказка: Сначала создаём сущности, потом действия, а уже потом собираем страницу.

Ответ:

- Создать entities

- Создать features
- Собрать страницу в pages
- Добавить UI в shared

🔮 Шаг 16:

Миссия 16

Расположите в порядке усложнения: от простого компонента до полноценного экрана.

Подсказка: UI-элемент → сущность → фича → страница.

Ответ:

- shared/ui компонент
- entities компонент
- features компонент
- страница в pages

🔮 Шаг 17:

Миссия 17

Расположите действия команды по внедрению Atomic Design и FSD в проект.

Подсказка: Сначала определяем UI-базу, потом архитектурные слои.

Ответ:

- Создать атомы и молекулы
- Определить сущности (entities)
- Собрать фичи (features)
- Собрать страницы (pages)

📖 Урок 2 - Контейнерные и презентационные компоненты

🔮 Шаг 01:

Миссия 1

Алиса работает над новым проектом и заметила, что часть компонентов перегружена логикой. Она вспоминает разделение на контейнерные и презентационные компоненты. Что характерно для презентационного компонента?

Подсказка: Подумайте, какой тип компонентов больше заботится о верстке и отображении, чем о данных.

Ответ:

- Он отвечает только за UI и получает данные через props
- Он управляет состоянием и API-запросами
- Он содержит бизнес-логику и роутинг
- Он хранит глобальное состояние приложения

🔮 Шаг 02:

Миссия 2

Боб пишет чат-приложение. Он создал компонент ChatList, который тянет данные из API и сам рендерит список сообщений. Как лучше разделить этот компонент?

Подсказка: Вспомните правило: логика отдельно, отображение отдельно.

Ответ:

- **Создать контейнер `ChatListContainer` для данных и презентационный `ChatList` для отображения**
- Оставить всё как есть, ведь работает
- Сделать один общий компонент без props
- Перенести всё в `App.js`

 **Шаг 03:**

Миссия 3

Катя спорит с коллегами: нужно ли презентационный компонент подключать к Redux напрямую?

Подсказка: Подумайте, кто должен знать о глобальном состоянии.

Ответ:

- Да, презентационные компоненты напрямую подписываются на Redux
- **Нет, подключение к Redux лучше вынести в контейнерный компонент**
- Да, но только если мало данных
- Можно подключать как угодно

 **Шаг 04:**

Миссия 4

Дима рефакторит компонент `Profile`. В нём есть логика загрузки данных пользователя и верстка карточки профиля. Что правильнее сделать?

Подсказка: Вспомните, какой компонент лучше держит fetch-запросы.

Ответ:

- **Вынести загрузку данных в контейнер, а карточку оставить презентационной**
- Оставить всё вместе для удобства
- Перенести верстку в контейнер
- Дублировать компонент для каждого пользователя

 **Шаг 05:**

Миссия 5

Ирина пишет интернет-магазин. Она заметила, что её презентационный компонент `ProductCard` начал содержать много условий фильтрации товаров. Что это значит?

Подсказка: Подумайте, какой компонент должен принимать решения о данных.

Ответ:

- **Презентационный компонент перегружен логикой** — её стоит вынести в контейнер
- Это нормально, пусть UI-компоненты решают всё
- Нужно переписать компонент на классовый
- Следует использовать inline-стили вместо условий

 **Шаг 06:**

Миссия 6

 **Задача:** Сопоставьте тип компонента с его характеристикой.

Подсказка: Вспомните, какой компонент больше про UI, а какой — про данные.

Ответ:

- **Контейнерный компонент** — Отвечает за состояние и взаимодействие с API
- **Презентационный компонент** — Отвечает за внешний вид и получает данные через props
- **Smart Component** — Синоним контейнерного компонента
- **Dumb Component** — Синоним презентационного компонента

Шаг 07:

Миссия 7

 Петя спорит: можно ли презентационный компонент тестировать отдельно? Сопоставьте виды тестов с тем, где они чаще применяются.

Подсказка: UI-компоненты легко проверить на рендеринг, а контейнеры — на работу с логикой.

Ответ:

- **Презентационный компонент** — Чаще тестируется через snapshot-тесты
- **Контейнерный компонент** — Чаще тестируется через unit-тесты на бизнес-логику
- **Компоненты с API-запросами** — Проверяются с помощью mock-запросов
- **Компоненты с версткой** — Проверяются на корректный рендеринг элементов

Шаг 08:

Миссия 8

 Лена делает блог-платформу. Сопоставьте части приложения с их реализацией.

Подсказка: UI лучше отделять от загрузки данных.

Ответ:

- **Компонент PostContainer** — Загружает посты с сервера и передает их в UI
- **Компонент PostList** — Рендерит список постов
- **Компонент UserContainer** — Достаёт данные пользователя из API
- **Компонент UserCard** — Отображает фото и имя пользователя

Шаг 09:

Миссия 9

 Дима хочет понять, зачем нужны контейнерные компоненты. Сопоставьте цель с правильным объяснением.

Подсказка: Контейнеры снимают нагрузку с UI и управляют состоянием.

Ответ:

- **Разделение обязанностей** — UI-компоненты не перегружены логикой
- **Переиспользуемость** — Один UI можно подключать к разным контейнерам
- **Упрощение тестов** — UI и данные можно проверять отдельно
- **Масштабируемость** — Приложение легче расширять

Шаг 10:

Миссия 10

Расположите шаги при создании связки контейнер + презентационный компонент:

Подсказка: Сначала получаем данные, потом передаём их в UI, потом рендерим.

Ответ:

- Контейнер получает данные
- Контейнер передаёт данные через props
- Презентационный компонент рендерит UI
- Пользователь видит результат

 Шаг 11:

Миссия 11

Расположите шаги рефакторинга компонента с бизнес-логикой в контейнер и UI:

Подсказка: Логика уходит в контейнер, интерфейс остаётся в UI.

Ответ:

- Выделить бизнес-логику
- Создать контейнерный компонент
- Передать данные через props
- Оставить презентационный компонент для UI

 Шаг 12:

Миссия 12

Расположите действия разработчика, когда он тестирует контейнерный и презентационный компонент:

Подсказка: Контейнер проверяется на логику, UI — на рендеринг.

Ответ:

- Протестировать бизнес-логику контейнера
- Протестировать передачу props
- Протестировать корректный рендеринг UI
- Собрать все части вместе

 Шаг 13:

Миссия 13

Расположите шаги создания интернет-магазина с разделением контейнеров и UI:

Подсказка: Данные приходят из API, а карточка товара их показывает.

Ответ:

- Контейнер загружает список товаров
- Контейнер передаёт данные карточкам
- Презентационные карточки рендерят товары
- Пользователь видит витрину

 Шаг 14:

Миссия 14

Расположите шаги при подключении Redux к компонентам:

Подсказка: Контейнер работает с Redux, UI только получает данные.

Ответ:

- Подключить контейнер к Redux
- Получить данные из store

- Передать данные в UI
- Рендерить UI-компонент

Шаг 15:

Миссия 15

Расположите шаги при оптимизации крупного приложения:

Подсказка: Сначала разделите ответственность, потом настройте переиспользование.

Ответ:

- Определить перегруженные компоненты
- Вынести бизнес-логику в контейнеры
- Оставить презентационные компоненты для UI
- Повысить переиспользуемость

Урок 3 - Разделение UI и логики

Шаг 01:

Миссия 1

При разделении UI и логики, что важнее всего при проектировании компонентов?

Подсказка: Компоненты должны быть независимыми и легко переиспользуемыми.

Ответ:

- **Выделение логики в отдельные хуки или функции**
- Микс логики и представления в одном компоненте
- Использование большого количества глобальных переменных
- Создание монолитных компонентов с большим количеством props

Шаг 02:

Миссия 2

Какой из следующих принципов поможет разделить UI и логику в большом приложении?

Подсказка: Это связано с принципами инкапсуляции и абстракции.

Ответ:

- **Использование контейнерных и презентационных компонентов**
- Объединение UI и бизнес-логики в одном компоненте
- Прямой доступ к данным из компонента UI
- Создание статического интерфейса без обработки состояния

Шаг 03:

Миссия 3

 Задача: Сопоставьте типы компонентов с их основными обязанностями.

Типы компонентов:

Ответ:

- Контейнерные компоненты
- Презентационные компоненты
- Компоненты с логикой
- UI-компоненты

Обязанности:

1. Обработывают данные и передают их в дочерние компоненты.
2. Отвечают за внешний вид и отображение информации.
3. Содержат бизнес-логику и взаимодействуют с API.
4. Преобразуют данные в UI и обеспечивают независимость от состояния приложения.

Подсказка:

Контейнерные компоненты и компоненты с логикой работают с данными, а UI-компоненты фокусируются на представлении.

- **Контейнерные компоненты** — Обработывают данные и передают их в дочерние компоненты.
- **Презентационные компоненты** — Отвечают за внешний вид и отображение информации.
- **Компоненты с логикой** — Содержат бизнес-логику и взаимодействуют с API.
- **UI-компоненты** — Преобразуют данные в UI и обеспечивают независимость от состояния приложения.

Шаг 04:

Миссия 4

Как правильно организовать логику в React-приложении, чтобы разделить ответственность компонентов?

Подсказка: Стремитесь к декомпозиции и разделению задач.

Ответ:

- **Разделение логики на отдельные функции и хуки**
- Все данные держать в одном компоненте
- Смешивание логики и представления в одном компоненте
- Использование прямого импорта состояния во все компоненты

Шаг 05:

Миссия 5

Расположите шаги правильной архитектуры приложения в правильном порядке:

1. Создание контейнерных компонентов для работы с состоянием
2. Разделение логики и UI-компонентов
3. Использование хуков для выделения бизнес-логики
4. Создание UI-компонентов для отображения данных

Подсказка:

Начинайте с организационной структуры компонентов, а затем выделяйте логику.

Ответ:

- Создание контейнерных компонентов для работы с состоянием
- Разделение логики и UI-компонентов
- Использование хуков для выделения бизнес-логики
- Создание UI-компонентов для отображения данных

Шаг 06:

Миссия 6

Какую стратегию лучше всего использовать для управления состоянием в крупных приложениях React?

Подсказка: Управление состоянием важно для масштабируемости приложения.

Ответ:

- **Использование глобальных менеджеров состояния (Redux, Context API)**
- Передача состояния через props на всех уровнях компонента
- Хранение состояния только внутри UI-компонентов
- Использование большого количества useState в каждом компоненте

Шаг 07:

Миссия 7

 Задача: Сопоставьте стратегию управления состоянием с её описанием.

Стратегии:

Ответ:

- Redux
- Context API
- useState
- Redux Toolkit

Описание:

1. Управление состоянием на глобальном уровне, для всего приложения.
2. Простая локальная разработка состояния для небольших компонентов.
3. Упрощает и ускоряет создание actions и reducers в Redux.
4. Хранение состояния в контексте для нескольких компонентов на уровне дерева компонентов.

Подсказка:

Подумайте, какие инструменты подходят для больших приложений, а какие для маленьких и средних.

- **Redux** — Управление состоянием на глобальном уровне, для всего приложения.
- **Context API** — Хранение состояния в контексте для нескольких компонентов на уровне дерева компонентов.
- **useState** — Простая локальная разработка состояния для небольших компонентов.
- **Redux Toolkit** — Упрощает и ускоряет создание actions и reducers в Redux.

Шаг 08:

Миссия 8

Когда имеет смысл использовать контейнерные компоненты для управления состоянием в приложениях?

Подсказка: Контейнерные компоненты должны решать задачи, связанные с управлением данными и состоянием, а не только с отображением UI.

Ответ:

- **Когда компоненту необходимо управлять состоянием и передавать данные в другие компоненты**
- Когда компонент только отображает данные и не имеет состояния
- Когда нужно сделать компонент независимым от других частей приложения
- Когда компонент не имеет необходимости в реиспользовании и абстракции

🔗 Шаг 09:

Миссия 9

Расположите шаги создания компонента с разделением UI и логики в правильном порядке:

1. Определение роли компонента (UI или контейнер)
2. Разделение логики и отображения
3. Использование хуков для логики состояния
4. Проверка и тестирование компонент

🔗 Подсказка:

Для начала всегда определяйте роль компонента, а затем разделяйте UI и логику.

Ответ:

- Определение роли компонента (UI или контейнер)
- Разделение логики и отображения
- Использование хуков для логики состояния
- Проверка и тестирование компонент

🔗 Шаг 10:

Миссия 10

📝 Задача: Сопоставьте каждый тип компонента с его примером использования.

Типы компонентов:

Ответ:

- Контейнерный компонент
- UI-компонент

Примеры:

1. Форма для регистрации пользователя с состоянием и логикой отправки данных.
2. Кнопка, которая отображает текст и вызывает действие при клике.

🔗 Подсказка:

Контейнерные компоненты управляют состоянием, а UI-компоненты фокусируются только на визуализации данных.

- **Контейнерный компонент** — Форма для регистрации пользователя с состоянием и логикой отправки данных.
- **UI-компонент** — Кнопка, которая отображает текст и вызывает действие при клике.

🔗 Шаг 11:

Миссия 11

Когда можно использовать Context API для управления состоянием в приложении?

Подсказка: Context API — это хороший выбор для приложений с небольшим количеством данных, но он не подходит для больших проектов с множеством состояний.

Ответ:

- **Когда приложение имеет небольшое количество состояний, и данные нужно передавать между компонентами на разных уровнях**
- Когда приложение большое, и состояние используется во многих компонентах одновременно
- Когда все состояние централизовано в одном месте, например, в Redux

- Когда данные должны быть доступны только в одном компоненте, а не в дереве компонентов

🔗 Шаг 12:

Миссия 12

Расположите шаги для разделения UI и логики в правильном порядке:

1. Создание компонентов с четким разделением ответственности
2. Выделение логики в отдельные хуки
3. Определение структуры приложения и роутинга
4. Разработка UI-компонентов для визуализации данных

🔗 Подсказка:

Для начала создайте структуру, а затем делите логику и визуализацию.

Ответ:

- Создание компонентов с четким разделением ответственности
- Выделение логики в отдельные хуки
- Определение структуры приложения и роутинга
- Разработка UI-компонентов для визуализации данных

🔗 Шаг 13:

Миссия 13

Расположите шаги для эффективного использования Context API в приложении с правильным порядком:

1. Определение нужных данных и компонентов, которым они будут передаваться
2. Создание провайдера, который будет передавать данные через дерево компонентов
3. Подключение компонентов, которые будут получать данные из контекста
4. Оптимизация производительности, если это необходимо (например, через мемоизацию)

🔗 Подсказка:

Сначала определите, что нужно передавать через контекст, а затем уже настройте провайдеры и получателей.

Ответ:

- Определение нужных данных и компонентов, которым они будут передаваться
- Создание провайдера, который будет передавать данные через дерево компонентов
- Подключение компонентов, которые будут получать данные из контекста
- Оптимизация производительности, если это необходимо (например, через мемоизацию)

🔗 Шаг 14:

Миссия 14

Что из следующего помогает разделить логику и отображение в React-компонентах?

Подсказка: Принцип разделения UI и логики важен для удобства тестирования и реиспользования кода.

Ответ:

- **Использование контейнерных компонентов и UI-компонентов**
- Написание всего кода в одном компоненте
- Использование только одного компонента для UI и логики

- Отсутствие разделения кода по компонентам

🔗 Шаг 15:

Миссия 15

📖 Задача: Сопоставьте типы компонентов с их основными задачами.

Типы компонентов:

Ответ:

- Контейнерный компонент
- UI-компонент

Задачи:

1. Ответственность за логику и данные, передача этих данных в другие компоненты.
2. Отображение UI без изменения данных.

🔗 Подсказка:

Контейнерный компонент управляет состоянием, а UI-компонент фокусируется только на визуализации.

- **Контейнерный компонент** — Ответственность за логику и данные, передача этих данных в другие компоненты.
- **UI-компонент** — Отображение UI без изменения данных.

🔗 Шаг 16:

Миссия 16

Расположите шаги для создания контейнерного компонента в правильном порядке:

1. Определение источников данных для компонента
2. Создание состояния внутри компонента с использованием `useState` или `useReducer`
3. Передача состояния и данных в дочерние компоненты
4. Добавление логики для обновления состояния и обработки событий

🔗 Подсказка:

Контейнерный компонент должен управлять состоянием и передавать его в другие компоненты.

Ответ:

- Определение источников данных для компонента
- Создание состояния внутри компонента с использованием `useState` или `useReducer`
- Передача состояния и данных в дочерние компоненты
- Добавление логики для обновления состояния и обработки событий

🔗 Шаг 17:

Миссия 17

Что является основным преимуществом разделения UI и логики в React-компонентах?

Подсказка: Правильное разделение UI и логики улучшает тестируемость и реиспользуемость компонентов.

Ответ:

- **Улучшение тестируемости и реиспользуемости компонентов**
- Снижение количества кода и упрощение структуры приложения
- Повышение производительности за счет упрощения UI компонентов

- Уменьшение размера приложения и ускорение загрузки страниц

Урок 4 - Хуки и кастомные хуки как отдельная фича

Шаг 01:

Миссия 1

Что такое хук useState в React?

Подсказка: Он используется для управления состоянием в функциональных компонентах.

Ответ:

- **Хук для создания состояния в функциональных компонентах**
- Хук для работы с побочными эффектами
- Хук для работы с формами
- Хук для управления событиями

Шаг 02:

Миссия 2

Что делает хук useEffect в React?

Подсказка: Он используется для работы с побочными эффектами в компонентах.

Ответ:

- **Позволяет выполнять побочные эффекты в компонентах**
- Используется для работы с состоянием компонента
- Мемоизирует значения между рендерами
- Обрабатывает события пользователя

Шаг 03:

Миссия 3

 Задача: Сопоставьте хуки с их назначением.

Формулировка: Сопоставьте хук с его задачей.

Хуки:

Ответ:

- useState
- useEffect
- useContext

Задачи:

1. Обрабатывает побочные эффекты компонента.
2. Предоставляет доступ к состоянию в функциональных компонентах.
3. Предоставляет доступ к контексту внутри компонентов.

Подсказка:

Каждый хук имеет свою область применения: для состояния, эффектов или контекста.

- **useState** — Предоставляет доступ к состоянию в функциональных компонентах.
- **useEffect** — Обрабатывает побочные эффекты компонента.
- **useContext** — Предоставляет доступ к контексту внутри компонентов.

🔮 Шаг 04:

Миссия 4

Когда лучше использовать хук `useMemo`?

Подсказка: Этот хук помогает оптимизировать рендеринг компонентов, мемоизируя вычисления.

Ответ:

- **Для мемоизации значений и предотвращения лишних вычислений**
- Для работы с состоянием компонента
- Для обработки побочных эффектов
- Для работы с контекстом

🔮 Шаг 05:

Миссия 5

Что такое кастомный хук в React?

Подсказка: Кастомные хуки позволяют переиспользовать логику между компонентами.

Ответ:

- **Функция, которая использует хуки React и возвращает логику для переиспользования**
- Функция, которая возвращает JSX-код для компонентов
- Функция для работы с состоянием внутри компонента
- Функция для обработки событий внутри компонента

🔮 Шаг 06:

Миссия 6

📄 Задача: Сопоставьте кастомный хук с его назначением.

Формулировка: Сопоставьте кастомный хук с его задачей.

Кастомные хуки:

Ответ:

- `useFetch`
- `useLocalStorage`
- `usePrevious`

Задачи:

1. Получение данных с сервера через `fetch`.
2. Сохранение значений в `localStorage` и их получение.
3. Получение предыдущего значения пропса или состояния.

🔮 Подсказка:

Каждый кастомный хук решает конкретную задачу и может использовать стандартные хуки внутри.

- **`useFetch`** — Получение данных с сервера через `fetch`.
- **`useLocalStorage`** — Сохранение значений в `localStorage` и их получение.
- **`usePrevious`** — Получение предыдущего значения пропса или состояния.

🔧 Шаг 07:

Миссия 7

Расположите шаги по созданию кастомного хука для работы с состоянием в правильном порядке:

1. Использование `useState` для создания состояния
2. Реализация логики обработки состояния
3. Возврат состояния и функции для его обновления

🔧 Подсказка:

Начните с создания состояния, затем обработайте логику и в конце обязательно предоставьте функции для обновления состояния.

Ответ:

- Использование `useState` для создания состояния
- Реализация логики обработки состояния
- Возврат состояния и функции для его обновления

🔧 Шаг 08:

Миссия 8

Когда необходимо использовать хук `useCallback`?

Подсказка: Этот хук помогает избежать лишних пересозданий функций при рендерах.

Ответ:

- **Для мемоизации функции, чтобы избежать её пересоздания при каждом рендере**
- Для создания состояния в компоненте
- Для обработки побочных эффектов
- Для работы с контекстом в приложении

🔧 Шаг 09:

Миссия 9

📄 Задача: Сопоставьте хук с его применением.

Формулировка: Сопоставьте хук с правильным случаем его использования.

Хуки:

Ответ:

- `useCallback`
- `useMemo`
- `useRef`

Применение:

1. Для мемоизации значений между рендерами.
2. Для сохранения значения между рендерами, которое не вызывает перерендер компонента.
3. Для предотвращения лишних пересозданий функции при каждом рендере.

🔧 Подсказка:

Подумайте, какие хуки используются для мемоизации, а какие для сохранения значения, не влияя на рендер компонента.

- **`useCallback`** — Для предотвращения лишних пересозданий функции при каждом рендере.

- **useMemo** — Для мемоизации значений между рендерами.
- **useRef** — Для сохранения значения между рендерами, которое не вызывает перерендер компонента.

Шаг 10:

Миссия 10

Расположите шаги для оптимизации производительности приложения с использованием хуков в правильном порядке:

1. Мемоизация значений с помощью `useMemo`
2. Мемоизация функции с помощью `useCallback`
3. Использование `useRef` для сохранения значений без перерендеров

Подсказка:

Обратите внимание, что различные хуки решают разные задачи: мемоизация значений, мемоизация функций и сохранение значений без перерендеров.

Ответ:

- Мемоизация значений с помощью `useMemo`
- Мемоизация функции с помощью `useCallback`
- Использование `useRef` для сохранения значений без перерендеров

Шаг 11:

Миссия 11

Что такое `useReducer` и когда его стоит использовать?

Подсказка: Он помогает управлять состоянием в более сложных компонентах, где нужно учитывать более сложную логику.

Ответ:

- Для управления более сложным состоянием, когда `useState` становится неудобным
- Для мемоизации функций в компонентах
- Для сохранения значений между рендерами
- Для обработки побочных эффектов в компонентах

Шаг 12:

Миссия 12

 Задача: Сопоставьте хук `useReducer` с его возможным применением.

Формулировка: Сопоставьте `useReducer` с ситуацией, где он наиболее полезен.

Ситуации:

Ответ:

- Управление сложным состоянием
- Управление состоянием с большим количеством переходов
- Реализация логики обновления состояния через диспатч

Назначение:

1. Когда управление состоянием становится слишком сложным для `useState`.
2. Когда требуется контроль за множественными действиями и переходами состояний.
3. Когда обновление состояния происходит через отправку `action` (диспатч).

Подсказка:

Подумайте, какие задачи решаются с помощью `useReducer`: сложное состояние и управление переходами между состояниями.

- **Управление сложным состоянием** — Когда управление состоянием становится слишком сложным для `useState`.
- **Управление состоянием с большим количеством переходов** — Когда требуется контроль за множественными действиями и переходами состояний.
- **Реализация логики обновления состояния через диспатч** — Когда обновление состояния происходит через отправку `action` (диспатч).

Шаг 13:

Миссия 13

Расположите шаги для создания кастомного хука `useFetch` для работы с данными из API в правильном порядке:

1. Создание состояния для хранения данных и ошибки
2. Использование `useEffect` для выполнения запроса при монтировании компонента
3. Возврат данных и ошибок из хука

Подсказка:

Подумайте, какие шаги важны для работы с асинхронными запросами в React: сначала нужно подготовить состояние, потом запросить данные и вернуть результат.

Ответ:

- Создание состояния для хранения данных и ошибки
- Использование `useEffect` для выполнения запроса при монтировании компонента
- Возврат данных и ошибок из хука

Шаг 14:

Миссия 14

Когда использовать хук `useLayoutEffect`, а не `useEffect`?

Подсказка: Этот хук выполняется синхронно после всех изменений DOM.

Ответ:

- **Когда необходимо синхронно работать с DOM до того, как пользователь увидит изменения**
- Когда требуется запустить асинхронную операцию после рендера
- Когда нужно вернуть данные с сервера после рендера компонента
- Когда нужно обновить состояние без повторного рендера компонента

Шаг 15:

Миссия 15

 Задача: Сопоставьте хук `useEffect` с его правильным применением.

Формулировка: Сопоставьте `useEffect` с его назначением в различных ситуациях.

Ситуации:

Ответ:

- Запуск побочных эффектов при изменении состояния
- Подготовка к асинхронной операции после рендера
- Очистка подписок и таймеров при размонтировании компонента

Назначение:

1. Для работы с данными после рендера компонента.
2. Для очистки ресурсов при удалении компонента.
3. Для выполнения операций, зависящих от состояния.

Подсказка:

Подумайте, какие задачи решаются с помощью `useEffect`: побочные эффекты, асинхронные операции, а также очистка ресурсов.

- **Запуск побочных эффектов при изменении состояния** — Для выполнения операций, зависящих от состояния.
- **Подготовка к асинхронной операции после рендера** — Для работы с данными после рендера компонента.
- **Очистка подписок и таймеров при размонтировании компонента** — Для очистки ресурсов при удалении компонента.

Шаг 16:

Миссия 16

Расположите шаги для создания хука `useDebounce` в правильном порядке:

1. Использование `useState` для хранения значения
2. Использование `useEffect` для отслеживания изменений значения
3. Создание таймера с помощью `setTimeout` для задержки обновления значения
4. Возврат значения, которое будет обновляться с задержкой

Подсказка:

Думайте о том, как реализовать задержку при изменении значения. Для этого понадобится и состояние, и таймер.

Ответ:

- Использование `useState` для хранения значения
- Использование `useEffect` для отслеживания изменений значения
- Создание таймера с помощью `setTimeout` для задержки обновления значения
- Возврат значения, которое будет обновляться с задержкой

Модуль 4 - State Management

Урок 1 - Redux Toolkit и организация фичей

Шаг 01:

Миссия 1

Какой из этих методов является частью Redux Toolkit для работы с состоянием?

Подсказка: Подумайте, что входит в состав современного инструментария для упрощения работы с Redux.

Ответ:

- `createSlice`
- `useState`
- `useEffect`
- `useContext`

Шаг 02:

Миссия 2

Что происходит при создании slice с помощью createSlice в Redux Toolkit?

Подсказка: slice — это часть состояния, и она описывает как обновлять это состояние.

Ответ:

- **Определяется редуктор и действия для изменения части состояния**
- Создается глобальный объект состояния для всего приложения
- Устанавливается начальное значение для всех состояний в приложении
- Определяется только бизнес-логика без состояния

Шаг 03:

Миссия 3

Расположите шаги для создания Redux slice в правильном порядке:

1. Определить начальное состояние
2. Создать редуктор с действиями
3. Экспортировать действия и редуктор
4. Подключить slice в хранилище с помощью configureStore

 **Подсказка:**

Все начинается с начального состояния, и потом вы постепенно добавляете редукторы и действия, чтобы сделать состояние управляемым.

Ответ:

- Определить начальное состояние
- Создать редуктор с действиями
- Экспортировать действия и редуктор
- Подключить slice в хранилище с помощью configureStore

Шаг 04:

Миссия 4

 **Задача:** Сопоставьте компоненты с их задачами при использовании Redux Toolkit.

Формулировка: Сопоставьте компонент Redux Toolkit с его назначением.

Компоненты:

Ответ:

- createSlice
- configureStore
- createAsyncThunk
- createEntityAdapter

Назначения:

1. Создает slice с редукторами и действиями для управления состоянием
2. Позволяет асинхронно выполнять действия и управлять их состоянием
3. Подключает slice к глобальному хранилищу
4. Упрощает управление коллекциями объектов

 **Подсказка:**

Каждый компонент выполняет свою уникальную задачу: slice, хранилище, асинхронные действия и управление коллекциями.

- **createSlice** — Создает slice с редукторами и действиями для управления состоянием
- **configureStore** — Подключает slice к глобальному хранилищу
- **createAsyncThunk** — Позволяет асинхронно выполнять действия и управлять их состоянием
- **createEntityAdapter** — Упрощает управление коллекциями объектов

Шаг 05:

Миссия 5

Какой компонент Redux Toolkit используется для управления асинхронными операциями?

Подсказка: Это компонент, который помогает обрабатывать асинхронные действия и их состояния.

Ответ:

- **createAsyncThunk**
- createSlice
- createEntityAdapter
- configureStore

Шаг 06:

Миссия 6

 Задача: Сопоставьте этапы работы с createAsyncThunk.

Формулировка: Сопоставьте действия с их этапами при работе с асинхронным действием в Redux Toolkit.

Этапы:

Ответ:

- Запуск асинхронной операции
- Обработка успешного завершения операции
- Обработка ошибки

Действия:

1. Выполнение запроса к API или другому источнику
2. Диспатч успешного действия с полученными данными
3. Диспатч действия с ошибкой

Подсказка:

Создавая асинхронные действия, важно учитывать последовательность: запрос, успешное выполнение или ошибка.

- **Запуск асинхронной операции** — Выполнение запроса к API или другому источнику
- **Обработка успешного завершения операции** — Диспатч успешного действия с полученными данными
- **Обработка ошибки** — Диспатч действия с ошибкой

Шаг 07:

Миссия 7

Расположите шаги для создания асинхронного действия с помощью `createAsyncThunk` в правильном порядке:

1. Создание `thunk` с использованием `createAsyncThunk`
2. Диспатч действия
3. Обработка успешного ответа
4. Обработка ошибки

 Подсказка:

Не забудьте, что `thunk` это асинхронная операция, которая должна сначала быть создана, а затем диспатчиться в `store`.

Ответ:

- Создание `thunk` с использованием `createAsyncThunk`
- Диспатч действия
- Обработка успешного ответа
- Обработка ошибки

 Шаг 08:

Миссия 8

Что такое `createEntityAdapter` в `Redux Toolkit`?

Подсказка: Это компонент, который помогает работать с коллекциями объектов.

Ответ:

- **Упрощает работу с коллекциями объектов в `Redux Store`**
- Предоставляет функцию для работы с асинхронными действиями
- Создает и управляет состоянием для всего приложения
- Предоставляет API для работы с UI компонентами

 Шаг 09:

Миссия 9

 **Задача:** Сопоставьте методы `createEntityAdapter` с их назначением.

Формулировка: Сопоставьте метод с его описанием для работы с коллекциями объектов.

Методы:

Ответ:

- `addOne`
- `removeOne`
- `updateOne`

Назначения:

1. Добавляет один объект в коллекцию
2. Удаляет один объект из коллекции
3. Обновляет один объект в коллекции

 Подсказка:

Эти методы помогают вам манипулировать объектами в коллекции без изменения состояния всего приложения.

- **`addOne`** — Добавляет один объект в коллекцию
- **`removeOne`** — Удаляет один объект из коллекции

- **updateOne** — Обновляет один объект в коллекции

🔗 Шаг 10:

Миссия 10

Что такое thunk в контексте Redux Toolkit?

Подсказка: Это способ обработки асинхронных операций в Redux.

Ответ:

- **Функция для выполнения асинхронных операций и управления их состоянием**
- Компонент для управления состоянием в приложении
- Механизм для работы с UI компонентами
- Система для связи с API-сервисами

🔗 Шаг 11:

Миссия 11

Расположите шаги для добавления нового объекта в коллекцию через createEntityAdapter:

1. Создать коллекцию с помощью createEntityAdapter
2. Добавить новый объект с помощью метода addOne
3. Подключить коллекцию к Redux Store
4. Определить начальное состояние для коллекции

🔗 **Подсказка:**

Начинаем с создания коллекции и определения состояния. После этого мы добавляем данные и подключаем коллекцию в store.

Ответ:

- Создать коллекцию с помощью createEntityAdapter
- Добавить новый объект с помощью метода addOne
- Подключить коллекцию к Redux Store
- Определить начальное состояние для коллекции

🔗 Шаг 12:

Миссия 12

Какой из этих методов позволяет обновить объект в коллекции через createEntityAdapter?

Подсказка: Подумайте, что нужно делать, если объект в коллекции изменился.

Ответ:

- **updateOne**
- removeOne
- addOne
- deleteOne

🔗 Шаг 13:

Миссия 13

Расположите шаги для асинхронного процесса работы с API в правильном порядке:

1. Создание и настройка createAsyncThunk
2. Вызов API в асинхронной функции
3. Обработка успешного ответа в редьюсере

4. Обработка ошибки в редьюсере

Подсказка:

Помните, что асинхронная операция требует правильной обработки успешного ответа и ошибок.

Ответ:

- Создание и настройка `createAsyncThunk`
- Вызов API в асинхронной функции
- Обработка успешного ответа в редьюсере
- Обработка ошибки в редьюсере

Шаг 14:

Миссия 14

 **Задача:** Сопоставьте действия с их типами в `createAsyncThunk`.

Формулировка: Сопоставьте типы действий с их назначением в асинхронной операции.

Действия:

Ответ:

- `pending`
- `fulfilled`
- `rejected`

Назначения:

1. Когда запрос выполнен успешно
2. Когда запрос ожидает выполнения
3. Когда запрос завершён с ошибкой

Подсказка:

Эти типы действий соответствуют трем состояниям асинхронного запроса.

- **`pending`** — Когда запрос ожидает выполнения
- **`fulfilled`** — Когда запрос выполнен успешно
- **`rejected`** — Когда запрос завершён с ошибкой

Шаг 15:

Миссия 15

Что такое `selector` в `Redux Toolkit`?

Подсказка: Это функция для извлечения данных из состояния.

Ответ:

- **Функция для извлечения данных из состояния**
- Функция для обновления состояния
- Компонент для отображения данных
- Механизм для обработки ошибок

Шаг 16:

Миссия 16

Расположите шаги для использования `selector` для извлечения данных из состояния:

1. Создать `selector` с помощью `createSelector`

2. Использовать selector для получения данных
3. Подключить селектор к компоненту
4. Добавить селектор в Redux Slice

Подсказка:

Не забывайте подключить селектор в slice, чтобы использовать его в компонентах.

Ответ:

- Создать selector с помощью createSelector
- Использовать selector для получения данных
- Подключить селектор к компоненту
- Добавить селектор в Redux Slice

Урок 2 - Локальный state в компонентах и фичах

Шаг 01:

Миссия 1

Ваша задача — понять, какой подход нужно использовать для управления состоянием внутри компонента.

Подсказка: Важно помнить, что локальное состояние компонента используется, чтобы он мог реагировать на изменения, происходящие внутри.

Ответ:

- **Использование useState для управления состоянием компонента**
- Использование Redux для небольших компонентов
- Передача состояния через props в дочерние компоненты
- Использование классовых компонентов для локального состояния

Шаг 02:

Миссия 2

Какой хук позволяет вам отслеживать изменения состояния и обновлять компонент, когда это необходимо?

Подсказка: Этот хук используется для работы с состоянием внутри функциональных компонентов, он возвращает пару: текущее состояние и функцию для его обновления.

Ответ:

- **useState**
- useEffect
- useReducer
- useMemo

Шаг 03:

Миссия 3

 Сопоставьте типы состояний с их правильными примерами.

Типы состояний:

Ответ:

- Простой state
- Массивы и объекты
- Функции состояния

Примеры:

1. Отслеживание текущего значения введенного текста в поле
 2. Хранение списка товаров в корзине
 3. Обновление состояния на основе предыдущего состояния
- **Простой state** — Отслеживание текущего значения введенного текста в поле
 - **Массивы и объекты** — Хранение списка товаров в корзине
 - **Функции состояния** — Обновление состояния на основе предыдущего состояния

🔗 Шаг 04:

Миссия 4

Как можно обновить состояние компонента, если оно зависит от предыдущего значения?

Подсказка: Помните, что для правильного обновления состояния, когда оно зависит от предыдущего, используется специальная функция в хуке `useState`.

Ответ:

- **Использовать функцию обновления состояния с предыдущим состоянием**
- Использовать `setState()` в классовых компонентах
- Использовать `useEffect` для обновления состояния
- Использовать Context API для обновления состояния

🔗 Шаг 05:

Миссия 5

📄 Сопоставьте хук с его описанием.

Хуки:

Ответ:

- `useState`
- `useEffect`
- `useCallback`

Описание:

1. Хук, который запоминает функцию, чтобы не пересоздавать её при каждом рендере компонента.
 2. Хук для выполнения побочных эффектов, таких как запросы к серверу.
 3. Хук для работы с локальным состоянием компонента.
- **`useState`** — Хук для работы с локальным состоянием компонента.
 - **`useEffect`** — Хук для выполнения побочных эффектов, таких как запросы к серверу.
 - **`useCallback`** — Хук, который запоминает функцию, чтобы не пересоздавать её при каждом рендере компонента.

🔗 Шаг 06:

Миссия 6

Расположите этапы правильного использования хука `useEffect`, чтобы выполнялся запрос данных при монтировании компонента:

1. Определить функцию, которая будет выполнять запрос к API
2. Вызвать `useEffect` с пустым массивом зависимостей
3. Обработать полученные данные и обновить состояние с помощью `useState`

4. Рендерить компонент с полученными данными

Подсказка:

Хук `useEffect` позволяет выполнять побочные эффекты, такие как запросы к серверу, после рендера компонента. Помните, что пустой массив зависимостей означает выполнение эффекта только один раз.

Ответ:

- Определить функцию, которая будет выполнять запрос к API
- Вызвать `useEffect` с пустым массивом зависимостей
- Обработать полученные данные и обновить состояние с помощью `useState`
- Рендерить компонент с полученными данными

Шаг 07:

Миссия 7

Какой из этих хуков позволяет выполнить действие только при изменении определённого состояния?

Подсказка: Этот хук позволяет отслеживать изменения не всех состояний, а только тех, которые указаны в массиве зависимостей.

Ответ:

- **`useEffect`**
- `useState`
- `useReducer`
- `useMemo`

Шаг 08:

Миссия 8

 Сопоставьте правильные хуки с их описаниями.

Хуки:

Ответ:

- `useEffect`
- `useState`
- `useMemo`

Описание:

1. Возвращает мемоизированное значение, которое пересчитывается только если изменились зависимости.
 2. Позволяет отслеживать изменения состояния компонента.
 3. Используется для выполнения побочных эффектов, таких как запросы к серверу.
- **`useEffect`** — Используется для выполнения побочных эффектов, таких как запросы к серверу.
 - **`useState`** — Позволяет отслеживать изменения состояния компонента.
 - **`useMemo`** — Возвращает мемоизированное значение, которое пересчитывается только если изменились зависимости.

Шаг 09:

Миссия 9

Расположите шаги для правильного использования хука `useMemo`, чтобы избежать лишних перерасчётов:

1. Создать функцию, которая вычисляет дорогостоящие данные
2. Передать эту функцию в `useMemo`, чтобы результат кешировался
3. Использовать мемоизированное значение в компоненте
4. Пересчитать значение только при изменении зависимостей

 Подсказка:

Не забывайте, что хук `useMemo` помогает избежать ненужных перерасчётов, кешируя результат вычислений, если зависимости не изменились.

Ответ:

- Создать функцию, которая вычисляет дорогостоящие данные
- Передать эту функцию в `useMemo`, чтобы результат кешировался
- Использовать мемоизированное значение в компоненте
- Пересчитать значение только при изменении зависимостей

 Шаг 10:

Миссия 10

Какой из этих хуков позволяет кэшировать вычисления и избежать лишних перерасчётов?

Подсказка: Этот хук используется для оптимизации производительности, предотвращая повторные вычисления одного и того же значения, если зависимости не изменяются.

Ответ:

- **`useMemo`**
- `useState`
- `useEffect`
- `useReducer`

 Шаг 11:

Миссия 11

Расположите шаги для правильной работы с хуком `useReducer` для управления сложным состоянием:

1. Определить редуктор с логикой изменения состояния
2. Использовать `useReducer` для получения состояния и `dispatch`-функции
3. Вызвать `dispatch` для изменения состояния
4. Передать редуктор и начальное состояние в `useReducer`

 Подсказка:

Хук `useReducer` идеально подходит для управления состоянием, которое зависит от нескольких значений и требует сложной логики.

Ответ:

- Определить редуктор с логикой изменения состояния
- Использовать `useReducer` для получения состояния и `dispatch`-функции
- Вызвать `dispatch` для изменения состояния
- Передать редуктор и начальное состояние в `useReducer`

Шаг 12:

Миссия 12

Какой хук можно использовать для управления сложным состоянием с несколькими значениями, когда `useState` становится неудобным?

Подсказка: Хук предназначен для работы с логикой изменения состояния, когда нужно больше контроля над обновлениями.

Ответ:

- **`useReducer`**
- `useState`
- `useEffect`
- `useCallback`

Шаг 13:

Миссия 13

 Сопоставьте хуки с их основными особенностями.

Хуки:

Ответ:

- `useState`
- `useReducer`
- `useEffect`

Особенности:

1. Позволяет выполнять побочные эффекты, например, запросы к серверу.
2. Используется для работы с состоянием, которое зависит от нескольких действий.
3. Используется для управления состоянием компонента, которое не требует сложной логики.

 **Подсказка:**

Обратите внимание на то, как каждый хук решает разные задачи в React. `useReducer` и `useState` — это методы для работы с состоянием, но они решают разные проблемы.

- **`useState`** — Используется для управления состоянием компонента, которое не требует сложной логики.
- **`useReducer`** — Используется для работы с состоянием, которое зависит от нескольких действий.
- **`useEffect`** — Позволяет выполнять побочные эффекты, например, запросы к серверу.

Шаг 14:

Миссия 14

Расположите шаги для правильного использования хука `useState` в функциональных компонентах React:

1. Объявить состояние с помощью `useState`
2. Использовать состояние в компоненте
3. Обновить состояние с помощью функции, возвращаемой `useState`
4. Рендерить компонент с обновленным состоянием

 **Подсказка:**

Не забывайте, что `useState` всегда возвращает два значения: текущее состояние и функцию для его обновления.

Ответ:

- Объявить состояние с помощью `useState`
- Использовать состояние в компоненте
- Обновить состояние с помощью функции, возвращаемой `useState`
- Рендерить компонент с обновленным состоянием

Шаг 15:

Миссия 15

Какой хук нужно использовать для оптимизации рендеринга компонента, чтобы избежать лишних перерисовок при изменении состояния?

Подсказка: Этот хук помогает запомнить результат вычислений и повторно использовать его при одинаковых входных данных.

Ответ:

- **`useMemo`**
- `useState`
- `useEffect`
- `useReducer`

Шаг 16:

Миссия 16

Расположите этапы правильного использования хука `useContext` для работы с глобальным состоянием в приложении:

1. Создать и экспортировать контекст с помощью `createContext`
2. Создать провайдер и обернуть компоненты, которые будут использовать контекст
3. Использовать `useContext` для получения данных из контекста
4. Обновить данные контекста в компонентах, где это необходимо

 **Подсказка:**

Хук `useContext` позволяет передавать данные через дерево компонентов, минуя пропсы. Но не забывайте оборачивать компоненты в провайдер, чтобы они имели доступ к контексту.

Ответ:

- Создать и экспортировать контекст с помощью `createContext`
- Создать провайдер и обернуть компоненты, которые будут использовать контекст
- Использовать `useContext` для получения данных из контекста
- Обновить данные контекста в компонентах, где это необходимо

Шаг 17:

Миссия 17

Какой хук позволяет оптимизировать производительность, предотвращая лишние рендеры компонента, если значения не изменились?

Подсказка: Этот хук возвращает мемоизированную версию функции, которая изменяется только при изменении зависимостей.

Ответ:

- **`useCallback`**

- useMemo
- useEffect
- useState

Урок 3 - Shared state между фичами

Шаг 01:

Миссия 1

Представьте, что вы работаете в большой команде и у вас есть несколько компонентов, которые используют общий state. Как правильно организовать управление состоянием между этими компонентами?

Подсказка: Использование централизованного хранилища, такого как Redux, помогает избежать множества ошибок с состоянием.

Ответ:

- **Использовать централизованное хранилище состояния**
- Передавать состояние через пропсы на каждом уровне компонента
- Дублировать состояние в каждом компоненте, который его использует
- Использовать глобальные переменные для хранения состояния

Шаг 02:

Миссия 2

Предположим, у вас есть компонент для отображения информации о пользователе и компонент для отображения уведомлений. Оба компонента используют общий state для информации о пользователе. Какой метод управления состоянием вы выберете для этого случая?

Подсказка: Управление состоянием через контекст или централизованное хранилище может быть отличным выбором для таких ситуаций.

Ответ:

- **Использовать Redux или React Context API для управления общим состоянием**
- Использовать только пропсы и передавать состояние от родителя к потомку
- Создать отдельный state в каждом компоненте, даже если они используют одни и те же данные
- Использовать глобальные переменные для хранения состояния

Шаг 03:

Миссия 3

 Задача: Сопоставьте компоненты с методами для обмена состоянием.

Формулировка: Сопоставьте компонент с правильным методом управления состоянием для обмена данными с другими компонентами.

Компоненты:

Ответ:

- Компонент А
- Компонент В
- Компонент С

Методы:

1. React Context API
2. Redux

3. Пропсы и колбэки

Подсказка:

Для более сложных приложений с большим количеством состояния лучше использовать Redux или Context API. Для небольших компонентов, передающих данные вверх или вниз по дереву, лучше использовать пропсы и колбэки.

- **Компонент А** — Redux
- **Компонент В** — React Context API
- **Компонент С** — Пропсы и колбэки

Шаг 04:

Миссия 4

Как лучше всего организовать управление состоянием для большого приложения с множеством различных фич, которые должны обмениваться данными?

Подсказка: В случае большого приложения наилучшим решением может быть использование централизованного хранилища или отдельные сторы для каждой фичи.

Ответ:

- **Использовать централизованное хранилище с разделением на фичи или модули**
- Использовать глобальные переменные для хранения всего состояния
- Дублировать состояние в каждом компоненте, который использует данные
- Передавать состояние через пропсы на каждом уровне компонента

Шаг 05:

Миссия 5

 **Задача:** Сопоставьте методы управления состоянием с их преимуществами.

Формулировка: Сопоставьте метод управления состоянием с его основными преимуществами для крупного приложения.

Методы:

Ответ:

- Redux
- React Context API
- Местное состояние компонентов

Преимущества:

1. Простота использования и идеален для небольших приложений с минимальной связностью
2. Полный контроль над состоянием и возможностью быстрого масштабирования
3. Подходит для приложений с множеством глобальных состояний и высокой взаимозависимостью между компонентами

Подсказка:

Подумайте, как разные методы помогают вам масштабировать приложение и управлять состоянием. Redux подходит для больших приложений, а местное состояние — для небольших.

- **Redux** — Подходит для приложений с множеством глобальных состояний и высокой взаимозависимостью между компонентами
- **React Context API** — Простота использования и идеален для небольших приложений с минимальной связностью

- **Местное состояние компонентов** — Полный контроль над состоянием и возможностью быстрого масштабирования

Шаг 06:

Миссия 6

Какой из следующих методов управления состоянием подходит для небольших приложений, где не требуется масштабирование?

Подсказка: Для небольших проектов лучше использовать более легковесные решения, такие как местное состояние компонентов.

Ответ:

- **Местное состояние компонентов**
- Redux
- React Context API
- Golang Channels

Шаг 07:

Миссия 7

Какой инструмент или библиотеку лучше всего использовать для управления состоянием в приложении с несколькими вкладками, которые должны синхронизировать данные между собой?

Подсказка: Redux или другие библиотеки, которые поддерживают глобальное состояние, могут помочь в решении таких задач.

Ответ:

- **Redux**
- React Context API
- Локальное состояние каждого компонента
- Глобальные переменные

Шаг 08:

Миссия 8

 Задача: Сопоставьте управление состоянием с типом приложения, где оно наиболее подходит.

Формулировка: Сопоставьте метод управления состоянием с типом приложения.

Типы приложений:

Ответ:

- Маленькое приложение
- Среднее приложение
- Крупное приложение

Методы управления состоянием:

1. Местное состояние компонентов
2. React Context API
3. Redux

Подсказка:

Подумайте, какие методы лучше всего подходят для разных размеров приложений и какие возможности они предоставляют.

- **Маленькое приложение** — Местное состояние компонентов
- **Среднее приложение** — React Context API
- **Крупное приложение** — Redux

Шаг 09:

Миссия 9

Представьте, что вы строите приложение для управления задачами, и несколько компонентов должны следить за статусом каждой задачи. Какое решение лучше всего подойдет для таких требований?

Подсказка: Для синхронизации данных между несколькими компонентами стоит использовать глобальное хранилище состояния.

Ответ:

- **Использовать Redux или React Context API**
- Использовать локальное состояние для каждого компонента
- Использовать только глобальные переменные
- Передавать состояние через пропсы на каждом уровне компонента

Шаг 10:

Миссия 10

Расположите шаги настройки Redux в правильном порядке:

1. Создать store с помощью createStore
2. Использовать Provider для передачи store в приложение
3. Создать редьюсеры для управления состоянием
4. Подключить компоненты к store с помощью connect

Подсказка:

Redux начинается с создания хранилища, а затем используются редьюсеры и компоненты для работы с состоянием.

Ответ:

- Создать store с помощью createStore
- Использовать Provider для передачи store в приложение
- Создать редьюсеры для управления состоянием
- Подключить компоненты к store с помощью connect

Шаг 11:

Миссия 11

Что из следующего является основным недостатком использования централизованного хранилища для управления состоянием в больших приложениях?

Подсказка: При работе с централизованным хранилищем нужно быть осторожным с количеством состояний и их взаимозависимостями.

Ответ:

- **Сложность управления состоянием при росте приложения**
- Быстродействие хранилища
- Трудности с интеграцией с API
- Необходимость частой синхронизации с сервером

Шаг 12:

Миссия 12

Какой метод управления состоянием рекомендуется использовать, если ваше приложение имеет большое количество компонентов, которые должны взаимодействовать друг с другом?

Подсказка: Для таких приложений лучше использовать решения, которые обеспечивают централизованное хранилище состояния.

Ответ:

- **Redux или React Context API**
- Локальное состояние в компонентах
- Глобальные переменные
- Передача состояния через пропсы

Шаг 13:

Миссия 13

Когда необходимо использовать React Context API вместо Redux?

Подсказка: Контекст API идеально подходит для передачи состояния между компонентами, но он не такой масштабируемый, как Redux для сложных приложений.

Ответ:

- **Когда приложение маленькое и не требует сложной логики управления состоянием**
- Когда требуется хранить и обновлять состояние в больших и сложных приложениях
- Когда нужно синхронизировать состояния между различными сервисами приложения
- Когда приложение использует только один компонент для управления состоянием

Шаг 14:

Миссия 14

 Задача: Сопоставьте методы управления состоянием с их преимуществами.

Формулировка: Сопоставьте метод управления состоянием с его основным преимуществом.

Методы управления состоянием:

Ответ:

- Redux
- React Context API
- Местное состояние компонентов

Преимущества:

1. Подходит для небольших приложений, где не нужно централизованное хранилище.
2. Масштабируемое решение для сложных приложений с множеством состояний и взаимодействий.
3. Простой и быстрый способ передачи данных между компонентами.

 **Подсказка:**

Сделайте выбор, исходя из размера приложения и количества состояний, которые нужно управлять.

- **Redux** — Масштабируемое решение для сложных приложений с множеством состояний и взаимодействий.

- **React Context API** — Подходит для небольших приложений, где не нужно централизованное хранилище.
- **Местное состояние компонентов** — Простой и быстрый способ передачи данных между компонентами.

Шаг 15:

Миссия 15

Что такое глобальное состояние в контексте React приложений?

Подсказка: Глобальное состояние доступно для всех компонентов приложения и должно быть централизованным для обеспечения синхронности данных.

Ответ:

- **Состояние, которое доступно для всех компонентов приложения и хранится в одном месте**
- Состояние, которое локально хранится в каждом компоненте и передается через пропсы
- Состояние, которое должно быть синхронизировано с сервером в реальном времени
- Состояние, которое хранится в базе данных и передается через API

Шаг 16:

Миссия 16

Расположите шаги для создания централизованного хранилища состояния в Redux в правильном порядке:

1. Создать редьюсеры для обработки действий
2. Подключить store с помощью Provider в корневой компонент
3. Создать store с помощью createStore
4. Подключить компоненты с помощью connect

Подсказка:

Процесс создания Redux хранилища начинается с самого хранилища, а затем подключаются редьюсеры и компоненты для работы с ним.

Ответ:

- Создать редьюсеры для обработки действий
- Подключить store с помощью Provider в корневой компонент
- Создать store с помощью createStore
- Подключить компоненты с помощью connect

Шаг 17:

Миссия 17

Какой метод является лучшим для управления состоянием в приложениях с множеством компонентов, которые должны синхронизировать своё состояние?

Подсказка: Выберите метод, который лучше всего подходит для масштабируемых приложений с большими объемами данных.

Ответ:

- **Использование Redux или аналогичного глобального хранилища состояния**
- Использование местного состояния каждого компонента
- Использование глобальных переменных для хранения данных
- Передача состояния через пропсы от родительского компонента

Урок 4 - Использование RTK Query для асинхронных данных

Шаг 01:

Миссия 1

В вашем проекте появился новый API, и нужно реализовать асинхронное получение данных с помощью RTK Query. Какой метод из библиотеки RTK Query вы используете для получения данных?

Подсказка: RTK Query имеет специальные методы для работы с асинхронными запросами.

Ответ:

- ``useQuery``
- ``useMutation``
- ``createAsyncThunk``
- ``useEffect``

Шаг 02:

Миссия 2

Вы хотите обновить данные на сервере через RTK Query. Какой хук вам нужно использовать для выполнения мутации?

Подсказка: Для отправки данных на сервер используйте метод для мутаций, а не для запросов.

Ответ:

- ``useMutation``
- ``useQuery``
- ``createSlice``
- ``useEffect``

Шаг 03:

Миссия 3

 Задача: Сопоставьте методы RTK Query с их назначением

Методы:

Ответ:

- ``useQuery``
- ``useMutation``
- ``createApi``

Назначения:

1. Хук для выполнения мутаций (создание, обновление, удаление данных)
2. Хук для получения данных с сервера (например, GET запросы)
3. Создает объект API для работы с запросами и мутациями

Подсказка:

Подумайте, какой метод для чего применяется. Например, для отправки запросов на сервер используется ``useQuery``, а для отправки данных — ``useMutation``.

- ``useQuery`` — Хук для получения данных с сервера (например, GET запросы)
- ``useMutation`` — Хук для выполнения мутаций (создание, обновление, удаление данных)
- ``createApi`` — Создает объект API для работы с запросами и мутациями

Шаг 04:

Миссия 4

Ваше приложение использует RTK Query для загрузки списка пользователей. В какой момент лучше всего делать запрос, чтобы загрузить данные при монтировании компонента?

Подсказка: Подумайте о жизненном цикле компонента. Когда нужно выполнить асинхронный запрос?

Ответ:

- В хук `useEffect``
- В методе `componentDidMount``
- В конструкторе компонента
- В хендлере кнопки

Шаг 05:

Миссия 5

Какой способ кэширования в RTK Query используется по умолчанию?

Подсказка: RTK Query имеет механизм автоматического кэширования для запросов.

Ответ:

- Кэширование на основе идентификатора запроса
- Кэширование на основе времени ответа
- Кэширование на основе времени запроса
- Кэширование на основе хеширования данных

Шаг 06:

Миссия 6

Расположите шаги для выполнения запроса с RTK Query в правильном порядке:

1. Определите методы для GET или POST запросов внутри объекта API.
2. Создайте slice для данных с помощью `createSlice``.
3. Используйте хук `useQuery`` или `useMutation`` для получения/отправки данных.
4. Настройте параметры кэширования и обработки ошибок в объекте API.

 **Подсказка:**

Сначала создайте объект API, потом настройте его методы и только потом используйте хуки в компонентах.

Ответ:

- Определите методы для GET или POST запросов внутри объекта API.
- Создайте slice для данных с помощью `createSlice``.
- Используйте хук `useQuery`` или `useMutation`` для получения/отправки данных.
- Настройте параметры кэширования и обработки ошибок в объекте API.

Шаг 07:

Миссия 7

Что происходит, если запрос, выполненный через RTK Query, завершается с ошибкой?

Подсказка: RTK Query автоматически обрабатывает ошибки, предоставляя соответствующие флаги состояния.

Ответ:

- Ошибка будет отображена в состоянии запроса (например, ``isError`` или ``error`` в ответе)
- Запрос будет повторен автоматически
- Ошибка будет игнорироваться и данные не обновятся
- Ошибка будет приведена к пустому состоянию и запрос завершится с успехом

Шаг 08:

Миссия 8

 Задача: Сопоставьте типы состояний RTK Query с их значениями

Типы состояний:

Ответ:

- ``isLoading``
- ``isSuccess``
- ``isError``
- ``isUninitialized``

Значения:

1. Запрос еще не был выполнен
2. Запрос завершился успешно
3. Запрос в процессе выполнения
4. Во время выполнения запроса произошла ошибка

Подсказка:

Каждое из этих состояний помогает понять, что происходит с запросом в определенный момент времени.

- **``isLoading``** — Запрос в процессе выполнения
- **``isSuccess``** — Запрос завершился успешно
- **``isError``** — Во время выполнения запроса произошла ошибка
- **``isUninitialized``** — Запрос еще не был выполнен

Шаг 09:

Миссия 9

Какая функция RTK Query позволяет настроить поведение запроса при ошибке, например, повторить запрос несколько раз?

Подсказка: Для настройки повторных попыток запросов используйте специальную настройку в RTK Query.

Ответ:

- **``retry``**
- ``onError``
- ``onRetry``
- ``useMutation``

Шаг 10:

Миссия 10

Расположите этапы работы с RTK Query в правильном порядке:

1. Создайте API с помощью ``createApi``
2. Настройте методы для запросов и мутаций
3. Используйте хуки ``useQuery`` или ``useMutation`` в компонентах
4. Настройте поведение кэширования, обработки ошибок и retries

Подсказка:

Сначала создайте API, затем настройте методы и только потом используйте хуки в компонентах.

Ответ:

- Создайте API с помощью ``createApi``
- Настройте методы для запросов и мутаций
- Используйте хуки ``useQuery`` или ``useMutation`` в компонентах
- Настройте поведение кэширования, обработки ошибок и retries

Шаг 11:

Миссия 11

В чем заключается принцип работы кэширования в RTK Query?

Подсказка: Кэширование позволяет повторно использовать данные, что улучшает производительность.

Ответ:

- **RTK Query использует кэш на основе идентификатора запроса и повторно использует данные, если они еще актуальны**
- RTK Query кэширует все данные в одном месте, независимо от запроса
- Кэширование работает только в пределах одного компонента
- Каждый запрос требует кэширования вручную

Шаг 12:

Миссия 12

Что такое RTK Query и как он помогает в работе с асинхронными запросами?

Подсказка: RTK Query предоставляет удобные абстракции для работы с API в Redux.

Ответ:

- **RTK Query упрощает работу с API, автоматически управляя состоянием запроса.**
- RTK Query работает только с синхронными данными.
- RTK Query используется только для запросов типа GET.
- RTK Query заменяет весь Redux, включая экшены и редьюсеры.

Шаг 13:

Миссия 13

 Задача: Сопоставьте флаг состояния RTK Query с его значением.

Флаги:

Ответ:

- ``isFetching``
- ``isSuccess``
- ``isError``
- ``isUninitialized``

Значения:

1. Запрос находится в процессе выполнения.
2. Запрос завершен успешно.
3. Запрос завершился с ошибкой.
4. Запрос еще не был выполнен.

Подсказка:

Обратите внимание на флаги состояния запроса, они помогают отслеживать процесс выполнения запроса в реальном времени.

- ``isFetching`` — Запрос находится в процессе выполнения.
- ``isSuccess`` — Запрос завершен успешно.
- ``isError`` — Запрос завершился с ошибкой.
- ``isUninitialized`` — Запрос еще не был выполнен.

Шаг 14:

Миссия 14

Расположите шаги для интеграции RTK Query в ваш проект:

1. Установите необходимые зависимости и подключите Redux Toolkit.
2. Создайте объект API с помощью ``createApi``.
3. Настройте slice для данных с помощью RTK Query.
4. Используйте хуки ``useQuery`` или ``useMutation`` в компонентах для получения данных.

Подсказка:

Начните с установки зависимостей, потом настройте API, и только после этого используйте хуки в компонентах.

Ответ:

- Установите необходимые зависимости и подключите Redux Toolkit.
- Создайте объект API с помощью ``createApi``.
- Настройте slice для данных с помощью RTK Query.
- Используйте хуки ``useQuery`` или ``useMutation`` в компонентах для получения данных.

Шаг 15:

Миссия 15

Какой хук RTK Query используется для выполнения запросов с мутациями?

Подсказка: Для мутаций в RTK Query есть специальный хук, который позволяет отправлять данные на сервер.

Ответ:

- ``useMutation``
- ``useQuery``
- ``usePost``
- ``useFetch``

Шаг 16:

Миссия 16

 **Задача:** Сопоставьте API-методы RTK Query с их действиями.

Методы:

Ответ:

- GET
- POST
- PUT
- DELETE

Действия:

1. Обновление существующих данных на сервере.
2. Удаление данных с сервера.
3. Получение данных с сервера.
4. Добавление новых данных на сервер.

 Подсказка:

Каждый метод HTTP соответствует определенному действию с данными на сервере. Задумайтесь, какой метод отвечает за получение, обновление, удаление и добавление данных.

- **GET** — Получение данных с сервера.
- **POST** — Добавление новых данных на сервер.
- **PUT** — Обновление существующих данных на сервере.
- **DELETE** — Удаление данных с сервера.

Модуль 5 - Routing и асинхронность

Урок 1 - Lazy loading и code splitting по фичам

 Шаг 01:

Миссия 1

Что такое lazy loading в контексте React?

Подсказка: Lazy loading позволяет загружать компоненты только по мере их необходимости, улучшая производительность приложения.

Ответ:

- **Загрузка компонентов по мере их запроса пользователем.**
- Загрузка всех компонентов сразу при старте приложения.
- Загрузка компонентов только с сервера.
- Отображение всех компонентов, независимо от того, нужны ли они или нет.

 Шаг 02:

Миссия 2

Расположите шаги для реализации lazy loading в React в правильном порядке:

1. Создайте компонент, который будет загружаться лениво.
2. Используйте `React.lazy()` для асинхронной загрузки компонента.
3. Обработайте ошибки загрузки с помощью компонента `‘Suspense’`.
4. Настройте fallback для отображения во время загрузки компонента.

 Подсказка:

Не забудьте о `‘Suspense’` для обработки состояния загрузки!

Ответ:

- Создайте компонент, который будет загружаться лениво.

- Используйте `React.lazy()` для асинхронной загрузки компонента.
- Обработайте ошибки загрузки с помощью компонента `Suspense`.
- Настройте `fallback` для отображения во время загрузки компонента.

🔗 Шаг 03:

Миссия 3

📖 Задача: Сопоставьте понятие с его описанием для lazy loading в React.

Понятия:

Ответ:

- `React.lazy()`
- `Suspense`
- `fallback`
- Частичная загрузка

Описание:

1. Функция, которая позволяет загружать компонент лениво, по мере необходимости.
2. Компонент, который показывает запасной UI (например, `spinner`), пока не будет загружен основной компонент.
3. Элемент, который отображается во время загрузки компонента.
4. Метод оптимизации, при котором загружаются только те части приложения, которые нужны в данный момент.

🔗 Подсказка:

Помните, что ленивая загрузка помогает избежать загрузки ненужных частей интерфейса сразу.

- **`React.lazy()`** — Функция, которая позволяет загружать компонент лениво, по мере необходимости.
- **`Suspense`** — Компонент, который показывает запасной UI (например, `spinner`), пока не будет загружен основной компонент.
- **`fallback`** — Элемент, который отображается во время загрузки компонента.
- **Частичная загрузка** — Метод оптимизации, при котором загружаются только те части приложения, которые нужны в данный момент.

🔗 Шаг 04:

Миссия 4

Какую функцию вы вызываете для создания компонента с ленивой загрузкой в React?

Подсказка: Для этого есть специальная функция в React, которая возвращает компонент, загружаемый асинхронно.

Ответ:

- **`React.lazy()`**
- `lazy()`
- `React.load()`
- `useLazy()`

🔗 Шаг 05:

Миссия 5

Расположите шаги для оптимизации загрузки кода с использованием code splitting в React в правильном порядке:

1. Определите, какие части приложения можно загружать асинхронно.
2. Используйте `React.lazy()` для ленивой загрузки компонентов.
3. Используйте `Suspense` для отображения UI, пока компоненты не загрузятся.
4. Разделите приложение на модули с помощью динамического импорта.

Подсказка:

Code splitting помогает улучшить производительность, загружая только нужные части кода.

Ответ:

- Определите, какие части приложения можно загружать асинхронно.
- Используйте `React.lazy()` для ленивой загрузки компонентов.
- Используйте `Suspense` для отображения UI, пока компоненты не загрузятся.
- Разделите приложение на модули с помощью динамического импорта.

Шаг 06:

Миссия 6

 Задача: Сопоставьте методы с их функциональностью в процессе загрузки асинхронных компонентов.

Методы:

Ответ:

- `React.lazy()`
- `Suspense`
- Динамический импорт

Функциональность:

1. Функция для асинхронной загрузки компонентов.
2. Компонент, отображающий индикатор загрузки во время загрузки компонента.
3. Метод JavaScript для динамической загрузки модуля.

Подсказка:

Используйте динамический импорт для разбиения кода, а `Suspense` для управления состоянием загрузки.

- **`React.lazy()`** — Функция для асинхронной загрузки компонентов.
- **`Suspense`** — Компонент, отображающий индикатор загрузки во время загрузки компонента.
- **Динамический импорт** — Метод JavaScript для динамической загрузки модуля.

Шаг 07:

Миссия 7

Какая функция используется для динамического импорта компонентов в React?

Подсказка: Это стандартный JavaScript метод для импорта модулей по требованию.

Ответ:

- `import()`
- `require()`
- `load()`
- `dynamicImport()`

🔗 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте функции с их назначением при реализации ленивой загрузки.

Функции:

Ответ:

- `React.lazy()`
- `Suspense`
- `fallback`

Назначение:

1. Оборачивает компонент и обрабатывает его асинхронную загрузку.
2. Отображает запасной UI (например, `spinner`) во время загрузки компонента.
3. Элемент, который показывается, пока компонент не загрузится.

🔗 Подсказка:

Помните, что для ленивой загрузки нужно использовать два компонента: `Suspense` и `lazy()`.

- **`React.lazy()`** — Оборачивает компонент и обрабатывает его асинхронную загрузку.
- **`Suspense`** — Отображает запасной UI (например, `spinner`) во время загрузки компонента.
- **`fallback`** — Элемент, который показывается, пока компонент не загрузится.

🔗 Шаг 09:

Миссия 9

Какая из следующих библиотек помогает с lazy loading в React?

Подсказка: Библиотека, которая упрощает асинхронную загрузку компонентов с нужными `fallback`.

Ответ:

- **`React Loadable`**
- `React Router`
- `React Async`
- `Redux`

🔗 Шаг 10:

Миссия 10

Расположите шаги для асинхронной загрузки компонентов с помощью `React.lazy()` в правильном порядке:

1. Определите компоненты, которые нужно загружать асинхронно.
2. Оборачивайте компоненты в `React.lazy()`.
3. Настройте `fallback` с помощью `Suspense`.
4. Используйте асинхронно загружаемые компоненты в вашем приложении.

🔗 Подсказка:

Не забудьте о настройке `fallback` для обработки состояния загрузки компонента.

Ответ:

- Определите компоненты, которые нужно загружать асинхронно.
- Оборачивайте компоненты в `React.lazy()`.
- Настройте `fallback` с помощью `Suspense`.

- Используйте асинхронно загружаемые компоненты в вашем приложении.

🔗 Шаг 11:

Миссия 11

Какой метод позволяет загружать части приложения по мере необходимости?

Подсказка: Это один из подходов для оптимизации начальной загрузки.

Ответ:

- **Code splitting**
- Global state management
- Memoization
- State lifting

🔗 Шаг 12:

Миссия 12

Расположите шаги для настройки ленивой загрузки в правильном порядке:

1. Используйте `React.lazy()` для оборачивания компонентов.
2. Добавьте компонент `Suspense` для отображения индикатора загрузки.
3. Настройте fallback для отображения UI до загрузки компонента.

🔗 Подсказка:

Подумайте, как правильно организовать асинхронную загрузку и показ компонентов с индикатором загрузки.

Ответ:

- Используйте `React.lazy()` для оборачивания компонентов.
- Добавьте компонент `Suspense` для отображения индикатора загрузки.
- Настройте fallback для отображения UI до загрузки компонента.

🔗 Шаг 13:

Миссия 13

📖 Задача: Сопоставьте методы с их функциями в асинхронной загрузке компонентов.

Методы:

Ответ:

- `React.lazy()`
- `Suspense`
- fallback

Функции:

1. Метод для асинхронной загрузки компонентов.
2. Компонент, показывающий UI до загрузки компонента.
3. Отображает запасной UI (например, индикатор загрузки) во время загрузки компонента.

🔗 Подсказка:

Для асинхронной загрузки компонентов используйте комбинацию `React.lazy()` и `Suspense`.

- **`React.lazy()`** — Метод для асинхронной загрузки компонентов.
- **`Suspense`** — Отображает запасной UI (например, индикатор загрузки) во время загрузки компонента.

- **fallback** — Компонент, показывающий UI до загрузки компонента.

🔗 Шаг 14:

Миссия 14

Какая библиотека помогает с оптимизацией асинхронной загрузки компонентов в React?

Подсказка: Эта библиотека улучшает ленивую загрузку и удобна для работы с большими приложениями.

Ответ:

- **React Loadable**
- React Router
- Axios
- Redux-Saga

🔗 Шаг 15:

Миссия 15

Расположите компоненты для ленивой загрузки в правильном порядке:

1. Импортируйте компонент с помощью `React.lazy()`.
2. Оборачивайте компонент в `Suspense`.
3. Используйте `fallback` для отображения UI во время загрузки.

🔗 Подсказка:

Подумайте, какие шаги нужны для асинхронной загрузки компонента и отображения индикатора загрузки.

Ответ:

- Импортируйте компонент с помощью `React.lazy()`.
- Оборачивайте компонент в `Suspense`.
- Используйте `fallback` для отображения UI во время загрузки.

🔗 Шаг 16:

Миссия 16

Какой подход используется для уменьшения начальной загрузки приложения с помощью разбивки кода?

Подсказка: Это связано с разделением больших файлов на части, которые загружаются по мере необходимости.

Ответ:

- **Lazy loading**
- Context API
- Memoization
- HOC (Higher Order Component)

🔗 Шаг 17:

Миссия 17

Расположите шаги для внедрения ленивой загрузки с использованием `React.lazy()` в правильном порядке:

1. Импортировать компонент с помощью `React.lazy()`.

2. Оборачивать компонент в ``Suspense``.
3. Настроить fallback, чтобы показывать UI, пока компонент загружается.

Подсказка:

Подумайте, как правильно организовать загрузку компонента, чтобы не блокировать основной поток.

Ответ:

- Импортировать компонент с помощью ``React.lazy()``.
- Оборачивать компонент в ``Suspense``.
- Настроить fallback, чтобы показывать UI, пока компонент загружается.

Шаг 18:

Миссия 18

 Задача: Сопоставьте термин с его описанием в контексте ленивой загрузки.

Термины:

Ответ:

- ``React.lazy()``
- ``Suspense``
- Code splitting

Описания:

1. Позволяет загружать компоненты по мере необходимости.
2. Оборачивает асинхронно загружаемый компонент и показывает индикатор загрузки.
3. Процесс разделения кода приложения на отдельные чанки для более быстрой загрузки.

Подсказка:

При использовании ``React.lazy()`` и ``Suspense`` важно учитывать, что каждый компонент загружается по мере его запроса.

- **``React.lazy()``** — Позволяет загружать компоненты по мере необходимости.
- **``Suspense``** — Оборачивает асинхронно загружаемый компонент и показывает индикатор загрузки.
- **Code splitting** — Процесс разделения кода приложения на отдельные чанки для более быстрой загрузки.

Шаг 19:

Миссия 19

Какой из методов позволяет эффективно управлять загрузкой больших приложений, разбивая их на части?

Подсказка: Это один из самых популярных способов повышения производительности в современных приложениях.

Ответ:

- **Code splitting**
- Memoization
- Event delegation
- Debouncing

Урок 2 - Routing на уровне Pages и Features

Шаг 01:

Миссия 1

Представьте, что вы работаете над большим проектом с множеством страниц. Как лучше организовать роутинг для разных разделов приложения?

Подсказка: Использование разделения маршрутов по функциональным блокам может значительно улучшить архитектуру приложения.

Ответ:

- **Использовать роутинг на уровне Pages и Features**
- Создавать один большой компонент для всех страниц
- Использовать только один роут для всех компонентов
- Создавать роуты только на уровне компонентов

Шаг 02:

Миссия 2

Ваши коллеги предложили добавить ленивую загрузку для страниц, чтобы ускорить начальную загрузку. Какой подход для этого вам кажется самым подходящим?

Подсказка: Для оптимизации скорости важно загружать страницы только тогда, когда они реально нужны.

Ответ:

- **Использовать React.lazy и Suspense для динамической загрузки компонентов**
- Загружать все страницы при старте приложения
- Использовать обычный импорт для всех компонентов
- Загружать страницы с сервера при каждом запросе

Шаг 03:

Миссия 3

 Задача: Сопоставьте подходы к организации роутинга с их описаниями.

Подходы:

Ответ:

- Роутинг на уровне страниц
- Роутинг на уровне фич
- Ленивая загрузка компонентов
- Глобальный роутинг

Описания:

1. Система маршрутов разделена на части, которые соответствуют функциональным блокам приложения.
2. Компоненты загружаются только по мере необходимости, что ускоряет начальную загрузку.
3. Все маршруты определяются на уровне основного компонента, без разделения на функциональные блоки.
4. Маршруты определяются и привязываются на уровне каждого компонента.

 **Подсказка:**

Подумайте, как каждый из этих подходов влияет на поддержку и масштабируемость вашего приложения.

- **Роутинг на уровне страниц** — Все маршруты определяются на уровне основного компонента, без разделения на функциональные блоки.
- **Роутинг на уровне фич** — Система маршрутов разделена на части, которые соответствуют функциональным блокам приложения.
- **Ленивая загрузка компонентов** — Компоненты загружаются только по мере необходимости, что ускоряет начальную загрузку.
- **Глобальный роутинг** — Маршруты определяются и привязываются на уровне каждого компонента.

Шаг 04:

Миссия 4

Расположите шаги настройки роутинга на уровне страниц и фич в правильном порядке.

1. Определите маршруты для каждой страницы или фичи.
2. Настройте компонент для загрузки данных (если необходимо) для каждой страницы.
3. Подключите маршруты к компонентам с помощью React Router.
4. Реализуйте обработку ошибок и загрузки (если необходимо).

Подсказка:

Подумайте, в каком порядке шаги должны быть выполнены, чтобы обеспечить правильный рендеринг страниц.

Ответ:

- Определите маршруты для каждой страницы или фичи.
- Настройте компонент для загрузки данных (если необходимо) для каждой страницы.
- Подключите маршруты к компонентам с помощью React Router.
- Реализуйте обработку ошибок и загрузки (если необходимо).

Шаг 05:

Миссия 5

Ваш коллега забыл добавить страницу с 404 ошибкой. Какую страницу вы создадите для обработки ошибок?

Подсказка: Роутинг должен позволять обрабатывать ситуации, когда пользователь заходит на несуществующую страницу.

Ответ:

- **Страницу 404 с сообщением об ошибке**
- Страницу с главной страницей приложения
- Страницу с ошибкой сервера 500
- Страницу с пустым контентом

Шаг 06:

Миссия 6

В процессе разработки коллега предложил организовать роутинг таким образом, чтобы страницы не перезагружались при изменении состояния. Какой подход поможет вам в этом?

Подсказка: Для сохранения состояния страниц, вам нужно избежать полной перезагрузки.

Ответ:

- **Использовать React Router с механизмом client-side routing**
- Перезагружать страницы при каждом изменении состояния
- Использовать серверный роутинг для каждой страницы
- Загружать страницы через iframe

 **Шаг 07:**

Миссия 7

Вы столкнулись с ситуацией, когда пользователи часто переходят между несколькими страницами, но при этом скорость работы приложения оставляет желать лучшего. Что вы предложите сделать?

Подсказка: Для улучшения производительности можно использовать подходы, которые минимизируют повторные загрузки.

Ответ:

- **Использовать React.lazy и React Suspense для ленивой загрузки компонентов**
- Загружать все страницы при старте приложения
- Использовать серверные маршруты для всех страниц
- Использовать iframe для каждой страницы

 **Шаг 08:**

Миссия 8

 **Задача:** Сопоставьте типы роутинга с их характеристиками.

Типы роутинга:

Ответ:

- Client-side роутинг
- Server-side роутинг
- Progressive веб-аппликация (PWA) роутинг

Характеристики:

1. Маршруты обрабатываются на сервере при каждом запросе.
2. Роутинг выполняется на клиентской стороне, с минимальной перезагрузкой страниц.
3. Приложение работает без постоянного подключения к интернету и использует роутинг для создания «офлайн» версии.

 **Подсказка:**

Обратите внимание на то, где обрабатываются маршруты и какие возможности это даёт для производительности и доступности.

- **Client-side роутинг** — Роутинг выполняется на клиентской стороне, с минимальной перезагрузкой страниц.
- **Server-side роутинг** — Маршруты обрабатываются на сервере при каждом запросе.
- **Progressive веб-аппликация (PWA) роутинг** — Приложение работает без постоянного подключения к интернету и использует роутинг для создания «офлайн» версии.

 **Шаг 09:**

Миссия 9

Расположите шаги для настройки маршрутов на уровне фич в правильном порядке.

1. Разделите функциональные блоки приложения на страницы или компоненты.
2. Настройте маршруты для каждой функциональной области.
3. Реализуйте обработку загрузки для каждой страницы.
4. Подключите маршруты через React Router.

Подсказка:

Подумайте, с чего следует начать при организации роутинга на уровне фич, чтобы разделение было логичным и удобным.

Ответ:

- Разделите функциональные блоки приложения на страницы или компоненты.
- Настройте маршруты для каждой функциональной области.
- Реализуйте обработку загрузки для каждой страницы.
- Подключите маршруты через React Router.

Шаг 10:

Миссия 10

Ваш коллега задаёт вопрос: «Какие методы оптимизации роутинга могут помочь при работе с большим количеством страниц?» Какой ответ будет правильным?

Подсказка: Для работы с большим количеством страниц важно организовать так, чтобы только необходимые ресурсы загружались.

Ответ:

- **Использовать динамическую загрузку с React.lazy и Suspense**
- Загружать все страницы при старте приложения
- Использовать серверный роутинг для каждой страницы
- Подключать все страницы в глобальном роуте сразу

Шаг 11:

Миссия 11

 **Задача:** Сопоставьте методы работы с асинхронными данными с их описанием.

Методы работы с асинхронностью:

Ответ:

- React Query
- Redux Thunk
- Async/await

Описание:

1. Используется для обработки асинхронных действий в Redux, предоставляя доступ к функциям через middleware.
2. Обеспечивает выполнение асинхронных операций внутри функции с использованием синтаксиса, основанного на промисах.
3. Библиотека, которая помогает эффективно кешировать, синхронизировать и управлять состоянием асинхронных запросов в React-приложении.

Подсказка:

Обратите внимание, как методы связаны с различными подходами работы с асинхронными запросами и состоянием данных.

- **React Query** — Библиотека, которая помогает эффективно кешировать, синхронизировать и управлять состоянием асинхронных запросов в React-приложении.
- **Redux Thunk** — Используется для обработки асинхронных действий в Redux, предоставляя доступ к функциям через middleware.
- **Async/await** — Обеспечивает выполнение асинхронных операций внутри функции с использованием синтаксиса, основанного на промисах.

🔧 Шаг 12:

Миссия 12

Вы работаете над проектом, в котором необходимо асинхронно загружать данные с сервера, чтобы отобразить список пользователей. Какой метод в React лучше всего подходит для этого?

Подсказка: Для асинхронных запросов стоит использовать более удобный способ работы с промисами и состоянием.

Ответ:

- **Использовать React Query для загрузки и кеширования данных с сервера**
- Использовать Redux для хранения всех данных в одном глобальном состоянии
- Загружать данные напрямую через useEffect и useState без кеширования
- Отправлять все запросы через библиотеку Axios без дополнительной логики

🔧 Шаг 13:

Миссия 13

📖 Задача: Сопоставьте React хуки с их назначением.

Хуки:

Ответ:

- useState
- useEffect
- useContext

Назначение:

1. Позволяет компоненту «слушать» изменения в контексте и получить доступ к данным, переданным через Context API.
2. Позволяет изменять состояние внутри компонента.
3. Позволяет выполнять побочные эффекты в компонентах, такие как запросы к серверу или подписки на события.

🔧 Подсказка:

Обратите внимание, как хуки используются для работы с состоянием, побочными эффектами и контекстом в компонентах.

- **useState** — Позволяет изменять состояние внутри компонента.
- **useEffect** — Позволяет выполнять побочные эффекты в компонентах, такие как запросы к серверу или подписки на события.
- **useContext** — Позволяет компоненту «слушать» изменения в контексте и получить доступ к данным, переданным через Context API.

🔧 Шаг 14:

Миссия 14

Расположите шаги для работы с асинхронными запросами в правильном порядке:

1. Инициализация запроса к серверу.
2. Обработка ответа от сервера.
3. Отображение полученных данных в компоненте.
4. Обработка ошибок и состояние загрузки.

Подсказка:

Думайте логически: какие шаги происходят в процессе запроса и какие данные или состояния нужно обработать сначала.

Ответ:

- Инициализация запроса к серверу.
- Обработка ответа от сервера.
- Отображение полученных данных в компоненте.
- Обработка ошибок и состояние загрузки.

Шаг 15:

Миссия 15

Когда вы работаете с большим приложением и внедряете асинхронность, какой подход будет наиболее подходящим для масштабируемости?

Подсказка: Разделение на независимые блоки с возможностью автономной работы поможет улучшить масштабируемость.

Ответ:

- **Использовать микросервисную архитектуру для изоляции фичей и независимой работы**
- Использовать одно глобальное состояние для всех данных и приложений
- Применить подход с большим количеством общих компонентов для всех фичей
- Встроить все фичи в один монолитный проект без разделения

Урок 3 - Асинхронные данные и управление эффектами

Шаг 01:

Миссия 1

Вы решили загрузить данные о пользователях с API и отобразить их на странице. Какой React-хук вам поможет получить асинхронные данные?

Подсказка: Подумайте о хуке, который помогает работать с побочными эффектами, такими как запросы к серверу.

Ответ:

- **useEffect**
- useState
- useContext
- useReducer

Шаг 02:

Миссия 2

Расположите шаги работы с асинхронными данными в правильном порядке:

1. Запрос данных с сервера.

2. Обработка ошибок запроса.
3. Отображение данных на странице.
4. Обновление состояния компонента с результатами запроса.

Подсказка:

Порядок выполнения операций важен. Что должно происходить первым и последним?

Ответ:

- Запрос данных с сервера.
- Обработка ошибок запроса.
- Отображение данных на странице.
- Обновление состояния компонента с результатами запроса.

Шаг 03:

Миссия 3

 Задача: Сопоставьте асинхронные методы с их описанием.

Методы:

Ответ:

- `async/await`
- `fetch`
- `useEffect`

Описание:

1. Метод для работы с асинхронными операциями, позволяющий писать код, как синхронный.
2. Функция для выполнения HTTP-запросов с возможностью обработки ответа и ошибок.
3. React-хук, который используется для выполнения побочных эффектов, таких как запросы к серверу.

Подсказка:

Подумайте о том, какие методы применяются для работы с асинхронностью в JavaScript и React.

- **`async/await`** — Метод для работы с асинхронными операциями, позволяющий писать код, как синхронный.
- **`fetch`** — Функция для выполнения HTTP-запросов с возможностью обработки ответа и ошибок.
- **`useEffect`** — React-хук, который используется для выполнения побочных эффектов, таких как запросы к серверу.

Шаг 04:

Миссия 4

Какая из следующих опций будет правильной для обработки ошибок при асинхронных запросах в React?

Подсказка: Программируйте так, чтобы пользователи знали о возможных ошибках. Это поможет избежать неожиданных ситуаций.

Ответ:

- **Использовать `try/catch` внутри функции, где выполняется запрос**
- Обрабатывать ошибки с помощью `setState`
- Не обрабатывать ошибки и полагаться на стандартное поведение браузера
- Игнорировать ошибки и продолжать выполнение кода

🔗 Шаг 05:

Миссия 5

Расположите действия при загрузке данных с сервера в правильном порядке:

1. Инициализация состояния компонента для хранения данных.
2. Отправка запроса к серверу.
3. Обработка ответа от сервера и обновление состояния компонента.
4. Отображение данных на UI.

🔗 Подсказка:

Что происходит первым, когда вы начинаете загружать данные с сервера? А что после получения данных?

Ответ:

- Инициализация состояния компонента для хранения данных.
- Отправка запроса к серверу.
- Обработка ответа от сервера и обновление состояния компонента.
- Отображение данных на UI.

🔗 Шаг 06:

Миссия 6

📖 Задача: Сопоставьте хуки с их правильным использованием в контексте работы с асинхронными данными.

Хуки:

Ответ:

- `useEffect`
- `useState`
- `useReducer`

Применение:

1. Для обработки побочных эффектов, таких как запросы к серверу.
2. Для хранения и обновления состояния компонента.
3. Для управления состоянием, когда требуется более сложное обновление состояния.

🔗 Подсказка:

Каждый хук используется для своей цели. Подумайте, где вам нужно следить за изменениями и где вы хотите просто обновить состояние.

- **`useEffect`** — Для обработки побочных эффектов, таких как запросы к серверу.
- **`useState`** — Для хранения и обновления состояния компонента.
- **`useReducer`** — Для управления состоянием, когда требуется более сложное обновление состояния.

🔗 Шаг 07:

Миссия 7

В каком случае лучше использовать `useReducer`, а не `useState` для управления состоянием?

Подсказка: `useReducer` помогает, когда вам нужно более сложное управление состоянием с несколькими действиями.

Ответ:

- Когда состояние зависит от множества действий, которые изменяют его по-разному.
- Когда нужно просто обновить одно значение в состоянии.
- Когда не нужно управлять состоянием вообще.
- Когда не требуется масштабируемость компонента.

 Шаг 08:

Миссия 8

Расположите шаги для правильной обработки асинхронного запроса с использованием `fetch` в React:

1. Использование `then` или `await` для получения ответа.
2. Обработка ошибки с помощью `catch` или блока `try/catch`.
3. Отправка HTTP-запроса с помощью `fetch`.
4. Обработка успешного ответа и обновление состояния компонента.

 Подсказка:

Порядок выполнения запросов важен для корректной работы вашего приложения. Как должен выглядеть стандартный процесс?

Ответ:

- Использование `then` или `await` для получения ответа.
- Обработка ошибки с помощью `catch` или блока `try/catch`.
- Отправка HTTP-запроса с помощью `fetch`.
- Обработка успешного ответа и обновление состояния компонента.

 Шаг 09:

Миссия 9

 **Задача:** Сопоставьте типы данных, возвращаемые из асинхронных операций в JavaScript, с их описаниями.

Типы данных:

Ответ:

- Promise
- Response
- JSON

Описание:

1. Представляет собой объект, который будет в будущем содержать результат асинхронной операции.
2. Ответ от сервера на запрос, содержащий данные о статусе и содержимом ответа.
3. Формат данных, который может быть использован для передачи информации между клиентом и сервером, обычно в виде строки.

 Подсказка:

Подумайте, какие данные вам возвращает `fetch` после успешного запроса.

- **Promise** — Представляет собой объект, который будет в будущем содержать результат асинхронной операции.

- **Response** — Ответ от сервера на запрос, содержащий данные о статусе и содержимом ответа.
- **JSON** — Формат данных, который может быть использован для передачи информации между клиентом и сервером, обычно в виде строки.

Шаг 10:

Миссия 10

Что из перечисленного является правильным способом вызова асинхронной функции внутри ``useEffect``?

Подсказка: Используйте асинхронные функции внутри ``useEffect``, но помните, что сам ``useEffect`` не может быть асинхронным.

Ответ:

- **Создайте внутри ``useEffect`` асинхронную функцию и вызовите её сразу.**
- Просто сделайте ``useEffect`` асинхронным.
- Вызовите асинхронную функцию прямо внутри ``useEffect`` без обертки.
- Используйте callback-функцию внутри ``useEffect``.

Шаг 11:

Миссия 11

 Задача: Сопоставьте методы обработки асинхронных запросов с их описаниями.

Методы:

Ответ:

- `then`
- `catch`
- `finally`

Описание:

1. Используется для обработки успешного завершения асинхронной операции.
2. Используется для обработки ошибок в случае неудачного завершения асинхронной операции.
3. Используется для выполнения кода, который должен быть выполнен в любом случае (независимо от результата).

Подсказка:

Обратите внимание, в каком порядке вы обрабатываете успешный результат, ошибку и завершающие действия.

- **`then`** — Используется для обработки успешного завершения асинхронной операции.
- **`catch`** — Используется для обработки ошибок в случае неудачного завершения асинхронной операции.
- **`finally`** — Используется для выполнения кода, который должен быть выполнен в любом случае (независимо от результата).

Шаг 12:

Миссия 12

Что из перечисленного позволяет работать с асинхронными данными в React с использованием хуков?

Подсказка: Для обработки асинхронных операций используйте хуки React.

Ответ:

- **useEffect с асинхронными функциями**
- useRef
- useContext
- useCallback

Шаг 13:

Миссия 13

Расположите шаги правильного использования `useEffect` с асинхронным запросом:

1. Создайте асинхронную функцию внутри `useEffect`.
2. Обработайте результат запроса.
3. Вызовите асинхронную функцию.
4. Убедитесь, что данные обновляются в состоянии компонента.

Подсказка:

Подумайте, как работает асинхронная функция внутри `useEffect` и какие шаги логичны для обработки данных.

Ответ:

- Создайте асинхронную функцию внутри `useEffect`.
- Обработайте результат запроса.
- Вызовите асинхронную функцию.
- Убедитесь, что данные обновляются в состоянии компонента.

Шаг 14:

Миссия 14

 Задача: Сопоставьте типы состояния и их правильные применения в контексте работы с асинхронными данными в React.

Типы состояния:

Ответ:

- loading
- data
- error

Применения:

1. Хранение данных, полученных с сервера.
2. Отображение индикатора загрузки, пока данные не будут получены.
3. Хранение сообщения об ошибке в случае сбоя запроса.

Подсказка:

Представьте, какие состояния вам понадобятся, чтобы обеспечить хороший пользовательский опыт.

- **loading** — Отображение индикатора загрузки, пока данные не будут получены.
- **data** — Хранение данных, полученных с сервера.
- **error** — Хранение сообщения об ошибке в случае сбоя запроса.

Шаг 15:

Миссия 15

Что из следующего является правильным способом обработки ошибок при асинхронном запросе в React?

Подсказка: Ошибки можно обрабатывать с помощью `try/catch` или `.catch()` при асинхронных операциях.

Ответ:

- Использовать `try/catch` или `.catch()` для обработки ошибок.
- Игнорировать ошибки и продолжать выполнение.
- Обрабатывать ошибки только на сервере.
- Использовать только `finally` для ошибок.

Урок 4 - Сценарии fallback и error boundaries

Шаг 01:

Миссия 1

Что такое error boundary в React?

Подсказка: Это механизм для перехвата и обработки ошибок в компонентах.

Ответ:

- Это компонент, который перехватывает ошибки JavaScript в его дочерних компонентах.
- Это компонент для асинхронной загрузки данных.
- Это метод для контроля стилей компонента.
- Это способ управления состоянием в React приложении.

Шаг 02:

Миссия 2

Расположите шаги для создания компонента error boundary в правильном порядке:

1. Создать класс-компонент, который реализует метод `componentDidCatch`.
2. Добавить состояние для хранения информации об ошибке.
3. Вставить дочерние компоненты в этот компонент.
4. Возвращать fallback UI при перехвате ошибки.

Подсказка:

Этот компонент должен обрабатывать ошибки, происходящие в его дочерних компонентах и показывать пользователю понятное сообщение.

Ответ:

- Создать класс-компонент, который реализует метод `componentDidCatch`.
- Добавить состояние для хранения информации об ошибке.
- Вставить дочерние компоненты в этот компонент.
- Возвращать fallback UI при перехвате ошибки.

Шаг 03:

Миссия 3

 Задача: Сопоставьте события с их правильными описаниями при использовании error boundaries.

События:

Ответ:

- `componentDidCatch`
- `static getDerivedStateFromError`

Описание:

1. Этот метод используется для обновления состояния компонента при возникновении ошибки.
2. Этот метод позволяет изменить UI, прежде чем будет показан fallback UI.

 **Подсказка:**

Подумайте, как можно обработать ошибки и как будет изменяться UI для пользователя.

- **`componentDidCatch`** — Этот метод используется для обновления состояния компонента при возникновении ошибки.
- **`static getDerivedStateFromError`** — Этот метод позволяет изменить UI, прежде чем будет показан fallback UI.

 **Шаг 04:**

Миссия 4

Какой тип компонента обычно используется для реализации error boundary в React?

Подсказка: Error boundaries могут быть реализованы только в компонентах класса.

Ответ:

- **Классовый компонент**
- Функциональный компонент с хуками
- Функциональный компонент с ``useEffect``
- Компонент, использующий ``React.memo``

 **Шаг 05:**

Миссия 5

Расположите шаги создания fallback UI в правильном порядке:

1. Создать компонент, который будет отображать заглушку.
2. Использовать условную отрисовку для отображения заглушки при ошибке.
3. Убедиться, что fallback UI отображается в случае ошибки в дочерних компонентах.

 **Подсказка:**

Представьте, что происходит с UI, когда что-то пошло не так — нужно отобразить что-то понятное пользователю.

Ответ:

- Создать компонент, который будет отображать заглушку.
- Использовать условную отрисовку для отображения заглушки при ошибке.
- Убедиться, что fallback UI отображается в случае ошибки в дочерних компонентах.

 **Шаг 06:**

Миссия 6

 **Задача:** Сопоставьте ошибочные сценарии с методами для их обработки через error boundary.

Ошибки:

Ответ:

- Ошибка загрузки данных
- Ошибка рендеринга

Методы:

1. `componentDidCatch`
2. `static getDerivedStateFromError`

 Подсказка:

Каждый метод используется для разных целей при обработке ошибок в компонентах.

- **Ошибка загрузки данных** — `componentDidCatch`
- **Ошибка рендеринга** — `static getDerivedStateFromError`

 Шаг 07:

Миссия 7

Какой элемент позволяет React компонента в случае ошибки отобразить fallback UI?

Подсказка: Это нужно для замены UI при ошибке.

Ответ:

- ``ErrorBoundary``
- ``useEffect``
- ``Suspense``
- ``useState``

 Шаг 08:

Миссия 8

Что будет происходить, если ошибка возникнет в дочернем компоненте, но нет error boundary?

Подсказка: В таком случае приложение может «упасть», и пользователь не увидит никаких данных.

Ответ:

- **Приложение «упадет», и пользователь увидит ошибку.**
- Приложение продолжит работать, но данные не будут обновляться.
- Приложение заменит ошибку на заглушку.
- Приложение проигнорирует ошибку и продолжит выполнение.

 Шаг 09:

Миссия 9

Расположите шаги для добавления fallback UI в дочерний компонент:

1. Использовать error boundary для перехвата ошибок.
2. Добавить fallback UI, который будет показываться при ошибке.
3. Протестировать компонент, вызвав ошибку в нем.
4. Отобразить сообщение об ошибке, если не удастся загрузить данные.

 Подсказка:

Не забудьте, что fallback UI должен быть информативным для пользователя, чтобы он знал, что происходит.

Ответ:

- Использовать error boundary для перехвата ошибок.
- Добавить fallback UI, который будет показываться при ошибке.
- Протестировать компонент, вызвав ошибку в нем.
- Отобразить сообщение об ошибке, если не удастся загрузить данные.

🔗 Шаг 10:

Миссия 10

📖 Задача: Сопоставьте ошибки с методами обработки в error boundaries.

Ошибки:

Ответ:

- Ошибка в рендере
- Ошибка в асинхронном запросе

Методы:

1. componentDidCatch
2. static getDerivedStateFromError

🔗 Подсказка:

Ошибки могут возникать как при рендере компонента, так и при выполнении асинхронных операций.

- **Ошибка в рендере** — static getDerivedStateFromError
- **Ошибка в асинхронном запросе** — componentDidCatch

🔗 Шаг 11:

Миссия 11

Что такое Suspense в React и как это связано с fallback UI?

Подсказка: Suspense позволяет отображать заглушку до того, как данные будут загружены.

Ответ:

- **Suspense позволяет отобразить fallback UI, пока ожидаются данные.**
- Suspense необходим для работы с Redux в React.
- Suspense управляет состоянием компонентов в React.
- Suspense отвечает за маршрутизацию в React.

🔗 Шаг 12:

Миссия 12

Расположите шаги для внедрения Suspense в приложение React:

1. Создать компонент с асинхронной загрузкой данных.
2. Использовать `Suspense` для обертки этого компонента.
3. Добавить fallback UI в компонент для отображения в случае ожидания.
4. Проверить отображение заглушки до того, как данные будут загружены.

🔗 Подсказка:

Внедрение Suspense помогает улучшить UX, показывая пользователю что-то до завершения загрузки данных.

Ответ:

- Создать компонент с асинхронной загрузкой данных.
- Использовать `Suspense` для обертки этого компонента.
- Добавить fallback UI в компонент для отображения в случае ожидания.
- Проверить отображение заглушки до того, как данные будут загружены.

🔗 Шаг 13:

Миссия 13

📖 Задача: Сопоставьте методы с их ролью в процессе обработки асинхронных ошибок в React.

Методы:

Ответ:

- `Suspense`
- `componentDidCatch`

Роль:

1. Используется для отображения fallback UI до завершения загрузки.
2. Используется для перехвата ошибок при асинхронных операциях.

🔗 Подсказка:

Каждый метод играет свою роль в обработке ошибок и ожидании данных.

- **`Suspense`** — Используется для отображения fallback UI до завершения загрузки.
- **`componentDidCatch`** — Используется для перехвата ошибок при асинхронных операциях.

🔗 Шаг 14:

Миссия 14

Что произойдет, если не добавить `componentDidCatch` в компонент с error boundary?

Подсказка: Без этого метода ошибки будут оставаться необработанными.

Ответ:

- **Ошибки не будут перехвачены и приведут к сбою приложения.**
- Приложение автоматически отобразит fallback UI.
- Приложение продолжит работать без изменений.
- Ошибки будут игнорироваться без какого-либо уведомления.

🔗 Шаг 15:

Миссия 15

Расположите шаги для обработки ошибки в `componentDidCatch`:

1. Перехватить ошибку с помощью метода `componentDidCatch`.
2. Обновить состояние компонента для отображения fallback UI.
3. Показать сообщение об ошибке пользователю.
4. Логировать ошибку для отладки.

🔗 Подсказка:

Не забудьте, что перехват ошибки — это только первый шаг в обработке ошибок.

Ответ:

- Перехватить ошибку с помощью метода `componentDidCatch`.

- Обновить состояние компонента для отображения fallback UI.
- Показать сообщение об ошибке пользователю.
- Логировать ошибку для отладки.

🔗 Шаг 16:

Миссия 16

📖 Задача: Сопоставьте компоненты с их задачами в процессе обработки ошибок.

Компоненты:

Ответ:

- Error Boundary
- Suspense

Задачи:

1. Отображает fallback UI при ожидании данных.
2. Перехватывает ошибки и предотвращает сбой приложения.

🔗 Подсказка:

Обратите внимание, что оба компонента выполняют разные функции, но направлены на улучшение пользовательского опыта при ошибках.

- **Error Boundary** — Перехватывает ошибки и предотвращает сбой приложения.
- **Suspense** — Отображает fallback UI при ожидании данных.

🔗 Шаг 17:

Миссия 17

Что будет, если не использовать fallback UI с Suspense?

Подсказка: В случае ошибки или долгой загрузки данных пользователь не увидит ничего.

Ответ:

- **Пользователь не увидит ничего, пока данные не загрузятся.**
- Пользователь увидит стандартное сообщение об ошибке.
- Приложение продолжит работу без изменений.
- Пользователь будет перенаправлен на страницу с ошибкой.

🔗 Шаг 18:

Миссия 18

Расположите шаги для применения Error Boundary в приложении:

1. Создать компонент с методом `componentDidCatch``.
2. Обновить состояние с ошибкой в компоненте.
3. Использовать Error Boundary для оборачивания дочерних компонентов.
4. Показать fallback UI при возникновении ошибки.

🔗 Подсказка:

Не забудьте, что Error Boundary обрабатывает только ошибки в компоненте и его дочерних компонентах.

Ответ:

- Создать компонент с методом `componentDidCatch``.
- Обновить состояние с ошибкой в компоненте.

- Использовать Error Boundary для оборачивания дочерних компонентов.
- Показать fallback UI при возникновении ошибки.

Модуль 6 - Тестирование и документация

Урок 1 - Unit-тесты компонентов и фич

Шаг 01:

Миссия 1

Аня делает SPA на React и решила добавить загрузку профиля пользователя через Suspense. Что будет отображаться, пока данные ещё не подгрузились?

Подсказка: Вспомни: Suspense нужен, чтобы показать что-то вместо контента, пока идёт загрузка.

Ответ:

- **Fallback-компонент (например, спиннер или скелетон)**
- Ошибка 404
- Пустая страница без элементов
- Компонент с заглушкой ErrorBoundary

Шаг 02:

Миссия 2

Сергей открыл страницу чата, но API временно недоступно. Как правильно обработать эту ситуацию с помощью ErrorBoundary?

Подсказка: ErrorBoundary нужен для отлавливания ошибок рендеринга и JS, а не для сетевых запросов напрямую.

Ответ:

- Показать всегда пустой div
- **Отобразить fallback-интерфейс с сообщением об ошибке**
- Перезапустить весь React-приложение
- Ничего не делать, ошибка сама исчезнет

Шаг 03:

Миссия 3

Олег добавил lazy loading для админ-панели. Что произойдет, если импортируемый модуль долго грузится?

Подсказка: React.lazy работает только вместе с Suspense.

Ответ:

- Появится ошибка компиляции
- **Будет показан fallback-компонент из Suspense**
- Компонент загрузится моментально
- React автоматически перезапустит приложение

Шаг 04:

Миссия 4

Марина тестирует ErrorBoundary. Она намеренно бросила ошибку в компоненте. Что именно перехватит ErrorBoundary?

Подсказка: `ErrorBoundary` работает только в определённых случаях.

Ответ:

- **Ошибки рендеринга и жизненного цикла дочерних компонентов**
- Ошибки в обработчиках событий
- Ошибки в асинхронных коллбэках
- Ошибки в глобальных скриптах вне React

🔮 Шаг 05:

Миссия 5

Дима хочет, чтобы при падении страницы заказа пользователь видел сообщение «Что-то пошло не так». Как этого достичь?

Подсказка: Подумай про `ErrorBoundary` и его `fallback UI`.

Ответ:

- Добавить `console.error` в компонент
- **Использовать `ErrorBoundary` с кастомным `fallback`**
- Выключить `strict mode`
- Обновлять страницу каждые 5 секунд

🔮 Шаг 06:

Миссия 6

Елена внедрила `Suspense` для загрузки списка фильмов. Но иногда API возвращает ошибку. Что ей нужно дополнительно использовать?

Подсказка: `Suspense` решает только проблему ожидания, а ошибки — другая история.

Ответ:

- **`ErrorBoundary`**
- `React.StrictMode`
- `setTimeout` для повторной загрузки
- `LocalStorage`

🔮 Шаг 07:

Миссия 7

Фронтенд-команда внедряет маршрутизацию. Как правильно показать спиннер при загрузке нового `route` с `lazy` компонентом?

Подсказка: Вспомни связку `React.lazy` + `Suspense`.

Ответ:

- **Оборачивать `route`-компонент в `Suspense` с `fallback`**
- Использовать `useEffect` с `console.log`
- Заранее импортировать все страницы
- Выводить `alert` при загрузке

🔮 Шаг 08:

Миссия 8

Катя настроила `ErrorBoundary`, но замечает, что при ошибке в обработчике `onClick` он не срабатывает. Почему?

Подсказка: `ErrorBoundary` не ловит всё подряд, только определённые ошибки.

Ответ:

- Ошибка в React-контексте
- **ErrorBoundary не перехватывает ошибки в обработчиках событий**
- Она неправильно импортировала компонент
- Нужно использовать React.StrictMode

🔮 Шаг 09:

Миссия 9

Сопоставьте концепты Suspense и ErrorBoundary с их ролями в приложении.

Подсказка: Suspense отвечает за ожидание, а ErrorBoundary за ошибки.

Ответ:

- **Suspense** — Показывает fallback, пока идёт загрузка
- **ErrorBoundary** — Перехватывает ошибки рендера
- **Lazy loading** — Отложенная загрузка компонентов
- **Fallback UI** — Альтернативный интерфейс при ожидании или ошибке

🔮 Шаг 10:

Миссия 10

Артём строит сложный UI. Сопоставьте ситуации и то, что отобразится пользователю.

Подсказка: Внимательно подумай, где нужен Suspense, а где ErrorBoundary.

Ответ:

- **Модуль ещё загружается** — Спиннер через Suspense
- **API вернул ошибку 500** — Сообщение из ErrorBoundary
- **Компонент рендерится успешно** — Основной UI
- **Асинхронный импорт страницы** — Fallback из Suspense

🔮 Шаг 11:

Миссия 11

Сопоставьте ключевые элементы с их назначением.

Подсказка: Каждый инструмент имеет чёткую цель: ожидание, ошибки, резервный UI.

Ответ:

- **ErrorBoundary** — Позволяет продолжить работу приложения при падении части UI
- **Suspense fallback** — UI, отображающийся во время ожидания
- **Lazy component** — Компонент, подгружаемый асинхронно
- **Ошибка в render()** — Перехватывается ErrorBoundary

🔮 Шаг 12:

Миссия 12

Расставьте шаги обработки ошибки в React в правильном порядке.

Подсказка: Сначала ошибка, потом ErrorBoundary, потом fallback.

Ответ:

- Происходит ошибка в дочернем компоненте
- ErrorBoundary перехватывает ошибку
- Рендерится fallback UI

- Приложение продолжает работу без краша

🔗 Шаг 13:

Миссия 13

Упорядочите шаги загрузки компонента через React.lazy и Suspense.

Подсказка: Подумай о последовательности: импорт, ожидание, отображение.

Ответ:

- Компонент импортируется асинхронно
- Suspense показывает fallback
- Загрузка завершается
- Рендерится реальный компонент

🔗 Шаг 14:

Миссия 14

Расположите шаги при переходе на новый route с lazy компонентом.

Подсказка: Сперва навигация, потом ожидание.

Ответ:

- Пользователь кликает на ссылку
- Suspense активирует fallback
- Загружается lazy компонент
- Отображается страница

📖 Урок 2 - Integration-тесты для слоев Features и Entities

🔗 Шаг 01:

Миссия 1

Аня пишет integration-тест для feature «Добавление комментария». Что она должна протестировать в первую очередь?

Подсказка: В integration-тестах проверяется связка нескольких слоёв, а не отдельные функции.

Ответ:

- **Корректное взаимодействие UI и API при добавлении комментария**
- Форматирование строки внутри одной функции
- Стили кнопки отправки
- Отсутствие консольных ошибок

🔗 Шаг 02:

Миссия 2

Сергей тестирует entity «UserProfile». Какой сценарий лучше подходит для integration-теста?

Подсказка: В entity важно проверить их использование в связке с features.

Ответ:

- **Загрузка профиля и отображение имени пользователя в UI**
- Наличие defaultProps в компоненте
- Корректность lint-правил
- Работу отдельных util-функций

🔗 Шаг 03:

Миссия 3

Марина пишет integration-тест на feature «Лайк поста». Что будет ключевым критерием успешности?

Подсказка: Важна проверка реального сценария от UI до изменения данных.

Ответ:

- Количество тестов в проекте
- **После клика на кнопку «Лайк» число лайков увеличивается**
- Отсутствие console.log
- Загрузка всех зависимостей заранее

🔗 Шаг 04:

Миссия 4

Олег проверяет интеграцию feature «Авторизация» с entity «User». Что должно быть предметом теста?

Подсказка: В integration-тестах имитируется поведение пользователя.

Ответ:

- **Пользователь вводит логин и пароль и попадает на главную страницу**
- Только правильность regex для пароля
- Корректность CSS для формы
- Загрузка шрифтов без ошибок

🔗 Шаг 05:

Миссия 5

Дима делает integration-тест для feature «Добавить товар в корзину». Что должно быть в проверке?

Подсказка: Тест должен охватывать полное взаимодействие между слоями.

Ответ:

- **После клика товар появляется в списке корзины**
- Функция reducer возвращает новый state
- Все propTypes корректно описаны
- Нет ошибок ESLint

🔗 Шаг 06:

Миссия 6

Елена пишет тест на feature «Редактирование профиля». Что проверяется в integration-тесте?

Подсказка: Здесь важно увидеть полный путь данных от UI до entity.

Ответ:

- **После редактирования данные обновляются и отображаются в UI**
- Файл компонента весит меньше 50 Кб
- Используются хуки вместо классов
- В тесте нет console.log

🔮 Шаг 07:

Миссия 7

Катя проверяет feature «Поиск товаров». Что лучше всего подходит для integration-теста?

Подсказка: Подумай, что должен реально видеть пользователь.

Ответ:

- **После ввода текста появляются подходящие результаты**
- Правильность написания типов в TypeScript
- Размер кнопки поиска соответствует макету
- Функция debounce вызывается один раз

🔮 Шаг 08:

Миссия 8

Артём пишет integration-тест для feature «Отправка сообщения в чат». Что должно быть в результате?

Подсказка: Думай о том, что ожидает пользователь после действия.

Ответ:

- **Сообщение отображается в списке чата**
- Событие onClick вызвано один раз
- Все импорты без ошибок
- Функция reducer возвращает корректный state

🔮 Шаг 09:

Миссия 9

Сопоставьте feature и то, что проверяется в integration-тесте.

Подсказка: Обратите внимание: тесты имитируют реальные сценарии использования.

Ответ:

- **Feature: Добавление комментария** — Комментарий появляется в списке
- **Feature: Лайк поста** — Счётчик лайков увеличивается
- **Feature: Поиск товаров** — Результаты отображаются по запросу
- **Feature: Авторизация** — Пользователь попадает на главную страницу

🔮 Шаг 10:

Миссия 10

Сопоставьте entity и ожидаемое поведение в интеграции.

Подсказка: Entity — это слой данных, а integration-тесты проверяют их в связке с UI.

Ответ:

- **Entity: User** — Имя отображается после авторизации
- **Entity: Product** — Товар добавляется в корзину
- **Entity: Message** — Сообщение появляется в чате
- **Entity: Profile** — Изменённые данные сохраняются

🔮 Шаг 11:

Миссия 11

Сопоставьте разработчиков и сценарии их integration-тестов.

Подсказка: Каждый разработчик тестирует разную связку features и entities.

Ответ:

- **Аня** — Добавление комментария и его отображение
- **Сергей** — Отображение профиля пользователя
- **Марина** — Лайк увеличивает счётчик
- **Олег** — Успешная авторизация перенаправляет на главную

Шаг 12:

Миссия 12

Сопоставьте элементы теста и их описание.

Подсказка: Подумайте о том, что именно проверяется на каждом уровне.

Ответ:

- **UI действие** — Клик или ввод текста
- **API вызов** — Запрос к серверу или моку
- **Entity** — Обновление данных
- **Feature** — Полный пользовательский сценарий

Шаг 13:

Миссия 13

Сопоставьте тип теста и его цель.

Подсказка: Юнит-тесты и интеграционные тесты имеют разные задачи.

Ответ:

- **Unit-тест** — Проверяет отдельную функцию
- **Integration-тест** — Проверяет взаимодействие нескольких модулей
- **End-to-end тест** — Проверяет полный сценарий пользователя
- **Snapshot-тест** — Сравнивает структуру UI

Шаг 14:

Миссия 14

Расставьте шаги integration-теста для feature «Авторизация» в правильном порядке.

Подсказка: Сначала ввод данных, потом отправка, затем проверка результата.

Ответ:

- Пользователь вводит логин и пароль
- Кликает по кнопке «Войти»
- Происходит запрос к API
- Пользователь перенаправляется на главную

Шаг 15:

Миссия 15

Расположите шаги integration-теста для feature «Добавить комментарий».

Подсказка: Подумайте о последовательности действий пользователя.

Ответ:

- Пользователь вводит текст комментария
- Нажимает кнопку «Отправить»

- Запрос уходит к API
- Комментарий отображается в списке

🔮 Шаг 16:

Миссия 16

Упорядочите шаги интеграционного теста для feature «Лайк поста».

Подсказка: Сначала действие, затем реакция.

Ответ:

- Пользователь кликает по иконке лайка
- Событие вызывает обновление entity Post
- API фиксирует новый лайк
- Счётчик лайков увеличивается на UI

🔮 Шаг 17:

Миссия 17

Определите правильный порядок действий при написании integration-теста для feature «Добавление товара в корзину».

Подсказка: Сначала действие пользователя, потом обновление состояния и UI.

Ответ:

- Клик по кнопке «Добавить в корзину»
- API сохраняет товар в корзине
- Entity Cart обновляется
- UI отображает новый товар

📖 Урок 3 - Storybook и документация

🔮 Шаг 01:

Миссия 1

Разработчик Костя подключает Storybook к проекту и задаётся вопросом: зачем вообще нужна эта библиотека?

Подсказка: Подумайте, как программисты тестируют и показывают UI без запуска всего приложения.

Ответ:

- Для изолированной разработки и документирования компонентов
- Для сборки production-бандла
- Для написания юнит-тестов
- Для управления версиями кода

🔮 Шаг 02:

Миссия 2

Лена заметила, что в Storybook можно переключать состояния кнопки (disabled, loading). Как это реализуется?

Подсказка: Здесь важны контрольные входные данные для компонентов.

Ответ:

- С помощью Storybook Controls (args)

- Через Webpack плагины
- Через настройки GitHub Actions
- Через конфигурацию ESLint

🔗 Шаг 03:

Миссия 3

Фронтендер Игорь хочет показать коллегам разные варианты карточки товара. Как ему правильно оформить stories?

Подсказка: Подумайте о структуре экспорта в Storybook.

Ответ:

- **Создать несколько экспортов с разными аргументами**
- Изменять состояние компонента прямо в JSX
- Использовать глобальные переменные
- Копировать код компонента для каждого состояния

🔗 Шаг 04:

Миссия 4

Юра хочет задокументировать пропсы компонента Avatar прямо в Storybook. Где это обычно отображается?

Подсказка: Вспомните вкладку, где разработчики видят описание и типы пропсов.

Ответ:

- **В Docs-панели Storybook**
- В настройках ESLint
- В Webpack DevServer
- В package.json

🔗 Шаг 05:

Миссия 5

Разработчица Маша хочет проверить совместимость компонентов с тёмной темой. Как это лучше всего сделать в Storybook?

Подсказка: Подумайте о глобальных настройках, которые можно переключать в реальном времени.

Ответ:

- **Использовать Storybook decorators для темы**
- Менять CSS напрямую в файлах
- Создать отдельное React-приложение для тёмной темы
- Запустить компонент в Node.js консоли

🔗 Шаг 06:

Миссия 6

Сопоставьте термин и его назначение в Storybook.

Подсказка: Обратите внимание на то, где задаются глобальные правила, а где описываются отдельные состояния.

Ответ:

- **Stories** — Описывают конкретные состояния компонента

- **Decorators** — Позволяют оборачивать компонент глобальными настройками или стилями
- **Args** — Задают входные параметры для компонентов
- **Docs** — Автоматически генерируемая документация

🔗 Шаг 07:

Миссия 7

Андрей подключает аддоны к Storybook. Сопоставьте аддон и его функционал.

Подсказка: У каждого аддона есть своя полезная ниша: от изменения цветов до тестов доступности.

Ответ:

- **@storybook/addon-actions** — Отслеживание событий и действий в UI
- **@storybook/addon-knobs** — Динамическое изменение пропсов компонентов
- **@storybook/addon-a11y** — Проверка доступности компонентов
- **@storybook/addon-viewport** — Тестирование компонентов под разные размеры экранов

🔗 Шаг 08:

Миссия 8

Свяжите тип документации с её назначением.

Подсказка: Документация может быть как автогенерируемой, так и ручной.

Ответ:

- **Autodocs** — Автоматически формирует описание компонентов на основе кода
- **MDX stories** — Позволяют писать документацию вручную с Markdown и JSX
- **ArgsTable** — Таблица с пропсами и их типами
- **Play function** — Добавляет тестируемые сценарии прямо в stories

🔗 Шаг 09:

Миссия 9

Компания внедряет Storybook в проект. Сопоставьте этап внедрения и его цель.

Подсказка: Подумайте о порядке: установка, настройка, создание примеров, подключение CI.

Ответ:

- **Установка Storybook** — Добавление зависимости и базовой конфигурации
- **Создание stories** — Документирование первого компонента
- **Добавление аддонов** — Расширение функциональности Storybook
- **Интеграция с CI/CD** — Автоматическая публикация Storybook

🔗 Шаг 10:

Миссия 10

Разработчики обсуждают: как поддерживать документацию в Storybook актуальной?

Сопоставьте подход и его эффект.

Подсказка: Важно автоматизировать, а не делать вручную.

Ответ:

- **Использование Autodocs** — Документация обновляется автоматически при изменении кода
- **Написание MDX** — Разработчик вручную описывает важные детали
- **Интеграция с тестами** — Проверка актуальности документации через CI

- **Обновление вручную** — Может привести к устареванию документации

🔧 Шаг 11:

Миссия 11

Федя хочет объяснить стажёрам разницу между Storybook и обычной документацией. Сопоставьте систему и её особенность.

Подсказка: Одна система — интерактивная, другая чаще статичная.

Ответ:

- **Storybook** — Даёт возможность интерактивно тестировать компоненты
- **Markdown-документация** — Текстовое описание без интерактивности
- **Swagger** — Документация API
- **Notion/Confluence** — Хранилище текстовых инструкций и гайдов

🔧 Шаг 12:

Миссия 12

Расставьте шаги настройки Storybook с нуля в правильном порядке:

Подсказка: Сначала инициализация, затем настройка, потом создание примеров.

Ответ:

- Установка Storybook через CLI
- Настройка конфигурационных файлов (.storybook)
- Создание первых stories
- Запуск Storybook локально

🔧 Шаг 13:

Миссия 13

Определи порядок действий при добавлении нового компонента в Storybook:

Подсказка: Сначала создаётся компонент, затем описывается его история, а потом тестируется.

Ответ:

- Создать React-компонент
- Написать stories для компонента
- Добавить аргументы (args) для вариаций
- Просмотреть в Storybook

🔧 Шаг 14:

Миссия 14

Расположите шаги по добавлению тёмной темы в Storybook:

Подсказка: Сначала создаётся тема, потом подключается декоратор, а потом проверяется результат.

Ответ:

- Создание объекта темы
- Подключение декоратора с темой
- Применение темы к нужным stories
- Визуальная проверка в Storybook

Шаг 15:

Миссия 15

Расставьте шаги подготовки документации через MDX:

Подсказка: Сначала пишется MDX, потом подключаются stories, а затем оформляются таблицы.

Ответ:

- Создание .stories.mdx файла
- Описание компонента с текстом и JSX
- Подключение ArgsTable
- Просмотр в Docs-панели

Шаг 16:

Миссия 16

Выстраивайте шаги интеграции Storybook в CI/CD:

Подсказка: Вспомните: сначала билд, потом деплой, потом просмотр.

Ответ:

- Сборка Storybook
- Публикация Storybook на сервере
- Просмотр Storybook через ссылку

Шаг 17:

Миссия 17

Расставьте шаги, как команда обновляет устаревшую документацию в Storybook:

Подсказка: Сначала анализ изменений, потом обновление stories, потом проверка в Docs.

Ответ:

- Проверка изменений в компонентах
- Обновление stories или MDX
- Запуск Storybook
- Визуальная проверка в Docs

Урок 4 - Сценарии end-to-end тестирования

Шаг 01:

Миссия 1

Фронтенд-разработчик Дима впервые настраивает E2E тесты для авторизации в React-приложении. Какой инструмент чаще всего используют для запуска браузерных сценариев?

Подсказка: Он умеет кликать по кнопкам и проверять текст на странице почти как человек.

Ответ:

- Cypress
- Jest
- ESLint
- Prettier

Шаг 02:

Миссия 2

QA-инженер Лена запускает E2E тест в CI. Какое преимущество даёт режим headless?

Подсказка: Представьте, что браузер работает без окна, но сценарии выполняются полностью.

Ответ:

- **Тесты выполняются быстрее и не требуют графического интерфейса**
- Тесты автоматически создают документацию Storybook
- Тесты запускаются только на мобильных устройствах
- Тесты не требуют написания кода

Шаг 03:

Миссия 3

Команда тестирует сценарий «добавление товара в корзину». Что важно сделать перед каждым тестом, чтобы результаты были стабильными?

Подсказка: Чистая база данных — залог повторяемости.

Ответ:

- **Сбрасывать или подготавливать тестовые данные**
- Отключать интернет
- Менять браузер на каждый тест
- Перезапускать операционную систему

Шаг 04:

Миссия 4

Разработчик Антон заметил, что тест падает только на продакшн-сервере. Что стоит проверить в первую очередь?

Подсказка: Иногда окружения слегка отличаются по данным или переменным.

Ответ:

- **Конфигурацию окружения и переменные среды**
- Версию редактора кода
- Наличие комментариев в тестах
- Размер экрана тестового компьютера

Шаг 05:

Миссия 5

При написании теста регистрации пользователь вводит пароль и нажимает Enter. Как правильно проверить успешность?

Подсказка: Смысл теста в том, чтобы убедиться, что страница действительно изменилась.

Ответ:

- **Проверить наличие элемента, который появляется только после успешной регистрации**
- Проверить количество строк в консоли разработчика
- Подсчитать количество нажатий клавиш
- Измерить скорость загрузки страницы

Шаг 06:

Миссия 6

Алексей впервые пишет E2E тест в Cypress для страницы логина. Какое действие должно быть первым?

Подсказка: Вспомните: прежде чем тестировать ввод, нужно попасть на саму страницу.

Ответ:

- **Открыть страницу логина**
- Ввести данные пользователя
- Проверить редирект
- Сделать скриншот страницы

Шаг 07:

Миссия 7

Марина пишет тест на корзину. Что она должна проверить в первую очередь после добавления товара?

Подсказка: Цель теста — убедиться, что действие действительно изменило интерфейс.

Ответ:

- **Что товар появился в списке корзины**
- Что открылась страница оплаты
- Что цена изменилась на сервере
- Что изменился заголовок вкладки браузера

Шаг 08:

Миссия 8

Иван тестирует фильтры каталога. Какой селектор в Cypress будет наиболее надёжным?

Подсказка: Лучше выбирать не стили или текст, а то, что редко меняется.

Ответ:

- **data-testid атрибут**
- CSS-класс
- Внутренний текст элемента
- Идентификатор стилей в SCSS

Шаг 09:

Миссия 9

Ольга пишет тест для регистрации нового пользователя. Что обязательно нужно сделать в конце сценария?

Подсказка: Представьте себя реальным пользователем — после регистрации вы ожидаете подтверждение.

Ответ:

- **Проверить сообщение об успешной регистрации**
- Сделать экспорт базы данных
- Отправить данные в API вручную
- Проверить наличие файла cookie

Шаг 10:

Миссия 10

Команда тестирует оплату. Какой сценарий E2E будет ключевым?

Подсказка: Выберите путь, который подтверждает, что пользователь действительно завершает покупку.

Ответ:

- **Оплата проходит и заказ отображается в истории**
- Кнопка оплаты меняет цвет
- Банковский API отвечает 200 OK
- Страница не падает с ошибкой

Шаг 11:

Миссия 11

Сопоставьте инструмент и его назначение в E2E тестировании.

Подсказка: Подумайте, какие библиотеки тестируют интерфейс, а какие — сервер.

Ответ:

- **Cypress** — Автоматизация браузерных действий
- **Playwright** — Альтернатива Cypress для UI-тестов
- **Supertest** — Проверка API
- **Jest** — Юнит-тестирование функций

Шаг 12:

Миссия 12

Сопоставьте тип теста с его задачей.

Подсказка: Представьте, на каком уровне проверяется логика: компонент, связка или весь путь.

Ответ:

- **Юнит-тест** — Проверка функции в изоляции
- **Интеграционный тест** — Проверка взаимодействия модулей
- **E2E тест** — Проверка сценария пользователя
- **Регрессионный тест** — Выявление багов после изменений

Шаг 13:

Миссия 13

Сопоставьте проблему с её вероятной причиной в E2E тестах.

Подсказка: Вспомните про асинхронность, селекторы и окружение.

Ответ:

- **Элемент не найден** — Неверный селектор
- **Тест падает только ночью** — Зависимость от времени
- **Нестабильный результат** — Отсутствие ожиданий
- **Тест не запускается** — Ошибка конфигурации окружения

Шаг 14:

Миссия 14

Сопоставьте библиотеку с её ролью в тестировании.

Подсказка: Некоторые инструменты нужны для UI, а некоторые — для сборки или проверки качества.

Ответ:

- **React Testing Library** — Тестирование компонентов
- **ESLint** — Статический анализ кода
- **Storybook** — Документация UI
- **Webpack** — Сборка проекта

 Шаг 15:

Миссия 15

Сопоставьте этап CI/CD с его задачей.

Подсказка: Вспомните: сначала проверка, потом сборка, а уже потом выкладка.

Ответ:

- **Lint** — Проверка стиля
- **Build** — Сборка артефактов
- **Test** — Запуск тестов
- **Deploy** — Выкатка приложения

 Шаг 16:

Миссия 16

Сопоставьте практику с её пользой.

Подсказка: Одни практики ускоряют тесты, другие — делают их стабильнее.

Ответ:

- **Моки и стабы** — Исключают внешние зависимости
- **Фикстуры** — Готовые тестовые данные
- **Снимки (snapshots)** — Сравнение верстки с эталоном
- **CI запуск** — Автоматическая проверка при каждом коммите

 Шаг 17:

Миссия 17

Расположите шаги теста авторизации.

Подсказка: Сначала ввод, потом действие, потом проверка.

Ответ:

- Открыть страницу логина
- Ввести логин и пароль
- Нажать кнопку «Войти»
- Проверить переход на главную страницу

 Шаг 18:

Миссия 18

Расположите шаги пайплайна CI.

Подсказка: Проверь код, собери его, протестируй и только потом выкатывай.

Ответ:

- Запуск линтера
- Сборка проекта
- Запуск тестов

- Деплой

Шаг 19:

Миссия 19

Расположите шаги теста формы регистрации.

Подсказка: Вспомните логику реального пользователя.

Ответ:

- Открыть страницу регистрации
- Заполнить поля формы
- Отправить форму
- Проверить сообщение об успехе

Модуль 7 - Оптимизация и производительность

Урок 1 - Оптимизация рендеринга компонентов

Шаг 01:

Миссия 1

Команда разрабатывает корпоративный портал. У Сергея компонент рендерится при каждом изменении родителя, хотя данные внутри не меняются. Что первым делом стоит попробовать?

Подсказка: Иногда достаточно простого метода, чтобы компонент перестал зря обновляться.

Ответ:

- **Оборачивать компонент в React.memo**
- Заменить все useState на Redux
- Удалить компонент и переписать на классы
- Применить setTimeout для задержки рендера

Шаг 02:

Миссия 2

Катя пишет модуль дашборда. Она заметила, что при каждом вводе текста пересчитывается сложная формула. Какой хук поможет ей избежать лишних вычислений?

Подсказка: Этот хук возвращает мемоизированное значение.

Ответ:

- **useMemo**
- useEffect
- useLayoutEffect
- useReducer

Шаг 03:

Миссия 3

Игорь оптимизирует фильтр товаров. Он передает callback в дочерний компонент, и этот callback пересоздается при каждом рендере. Какой хук использовать?

Подсказка: Этот хук сохраняет функции между рендерами.

Ответ:

- **useCallback**

- `useEffect`
- `useMemo`
- `useContext`

🔗 Шаг 04:

Миссия 4

Вася подключает библиотеку таблиц. При обновлении строки обновляется весь список. Какое решение может снизить количество перерисовок?

Подсказка: Подумайте, как изолировать обновление конкретного элемента.

Ответ:

- **Использовать ключи и мемоизацию строк**
- Удалить таблицу и заменить списком `ul/li`
- Переписать на классовые компоненты
- Применить `setInterval` для обновления

🔗 Шаг 05:

Миссия 5

Юля пишет форму регистрации. Компонент рендерится слишком часто из-за глобального состояния. Что может помочь сократить ненужные обновления?

Подсказка: Иногда полезно подключать только часть состояния, а не всё.

Ответ:

- **Использовать селекторы для выборки нужных данных**
- Подключить все данные сторы к каждому компоненту
- Хранить состояние в `window.globalState`
- Переписать форму на jQuery

🔗 Шаг 06:

Миссия 6

Степан заметил, что из-за тяжелого изображения страница подтормаживает. Какой приём из React может помочь отложить загрузку?

Подсказка: Подумайте о динамическом импорте и lazy loading.

Ответ:

- **React.lazy и Suspense**
- `setTimeout` перед загрузкой
- `useEffect` с бесконечным циклом
- Кэширование в `localStorage`

🔗 Шаг 07:

Миссия 7

Лена оптимизирует страницу с карточками товаров. Какой инструмент поможет ей быстро найти узкие места в рендеринге?

Подсказка: Это встроенный инструмент React для анализа производительности.

Ответ:

- **React Profiler**
- `console.log`

- `useDebugValue`
- CSS-анимации

🔗 Шаг 08:

Миссия 8

Саша учит новичков оптимизации React. Сопоставьте инструменты с их описанием.

Подсказка: Вспомните, какие хуки отвечают за мемоизацию значений, а какие — за мемоизацию функций.

Ответ:

- **`useMemo`** — Мемоизация значений
- **`useCallback`** — Мемоизация функций
- **`React.memo`** — Оптимизация функциональных компонентов
- **`Profiler`** — Инструмент анализа рендера

🔗 Шаг 09:

Миссия 9

На проекте обсуждают оптимизацию. Соотнесите проблему и решение.

Подсказка: Обратите внимание: разные проблемы решаются разными подходами.

Ответ:

- **Частые пересоздания функций** — `useCallback`
- **Тяжёлые вычисления** — `useMemo`
- **Много одинаковых компонентов** — `React.memo`
- **Непонятно, что замедляет** — `Profiler`

🔗 Шаг 10:

Миссия 10

Никита объясняет студентам паттерны. Сопоставьте термин и определение.

Подсказка: Думайте о том, что именно делает каждый паттерн.

Ответ:

- **`Code Splitting`** — Динамическая подгрузка модулей
- **`Virtualization`** — Отображение только видимой части списка
- **`Lazy Loading`** — Задержка загрузки компонента
- **`Selector`** — Выборка части состояния

🔗 Шаг 11:

Миссия 11

Аня столкнулась с множеством ненужных перерисовок. Сопоставьте приём и его пользу.

Подсказка: Каждый инструмент оптимизации решает конкретную задачу.

Ответ:

- **`React.memo`** — Сравнение пропсов и предотвращение лишнего рендера
- **`useMemo`** — Избежание лишних вычислений
- **`useCallback`** — Стабильность ссылок на функции
- **`Suspense`** — Асинхронная подгрузка компонентов

Шаг 12:

Миссия 12

Виктор делает код ревью. Он просит сопоставить неправильный подход и возможное решение.

Подсказка: Вспомните самые распространённые ошибки при работе со стейтом и пропсами.

Ответ:

- **Пересоздание массива в пропсах** — useМемо для мемоизации
- **Inline функции в JSX** — useCallback
- **Глобальное состояние в каждом компоненте** — Использовать селекторы
- **Огромный список рендерится сразу** — Виртуализация списка

Шаг 13:

Миссия 13

Расставьте шаги оптимизации компонента в правильном порядке.

Подсказка: Начинаем с анализа, затем применяем инструменты оптимизации и проверяем результат.

Ответ:

- Запустить Profiler и найти узкие места
- Применить React.мето или хуки оптимизации
- Проверить, что функциональность не нарушена
- Сравнить показатели до и после оптимизации

Шаг 14:

Миссия 14

Расположите шаги внедрения React.lazy для оптимизации загрузки.

Подсказка: Подумайте, что сначала нужно сделать, а что завершающее.

Ответ:

- Импортировать компонент через React.lazy
- Оборачивать в Suspense
- Добавить fallback-заглушку
- Протестировать корректность загрузки

Шаг 15:

Миссия 15

Разместите шаги оптимизации списка через виртуализацию в правильном порядке.

Подсказка: Сначала подключаем библиотеку, затем заменяем рендер списка и проверяем результат.

Ответ:

- Установить библиотеку виртуализации
- Заменить обычный список на виртуализированный
- Настроить размеры и высоту элементов
- Проверить производительность

Шаг 16:

Миссия 16

Расставьте порядок действий при использовании useMemo.

Подсказка: Важно сначала определить проблему, затем использовать хук и протестировать результат.

Ответ:

- Найти тяжёлое вычисление
- Оборачивать его в useMemo
- Определить зависимости
- Сравнить скорость до и после

💡 **Шаг 17:**

Миссия 17

Упорядочите шаги оптимизации через useCallback.

Подсказка: Сначала нужно найти проблему с пересозданием функций, затем применить оптимизацию.

Ответ:

- Замечаем частые пересоздания функций
- Оборачиваем функции в useCallback
- Передаем зависимости
- Проверяем, что дочерние компоненты рендерятся реже

📖 **Урок 2 - Использование React.memo и useMemo**

💡 **Шаг 01:**

Миссия 1

Артём сделал компонент профиля, и он каждый раз рендерится заново, даже если пропсы не изменились. Что лучше использовать?

Подсказка: Этот инструмент сравнивает пропсы и предотвращает ненужные рендеры.

Ответ:

- **React.memo**
- useState
- useEffect
- Context API

💡 **Шаг 02:**

Миссия 2

Марина написала компонент поиска. Каждый ввод символа вызывает сложные вычисления. Какое решение оптимально?

Подсказка: Этот хук позволяет кэшировать результат функции.

Ответ:

- **useMemo**
- useCallback
- useReducer
- useRef

🔗 Шаг 03:

Миссия 3

Иван использует `React.memo` для списка карточек. Но при изменении темы все карточки всё равно рендерятся заново. Почему?

Подсказка: Вспомните, как работает сравнение пропсов по ссылке.

Ответ:

- **Изменяется объект стилей, и ссылка всегда новая**
- `React.memo` работает только с числами
- Карточки нельзя мемоизировать
- Ошибка в `useEffect`

🔗 Шаг 04:

Миссия 4

Кирилл передаёт в дочерний компонент функцию фильтрации. Она создаётся заново при каждом рендере. Что стоит применить?

Подсказка: Этот хук запоминает функцию до тех пор, пока не изменятся зависимости.

Ответ:

- **`useCallback`**
- `useMemo`
- `useEffect`
- `useLayoutEffect`

🔗 Шаг 05:

Миссия 5

Света оптимизирует таблицу. Каждая строка обновляется при изменении любого элемента. Как минимизировать рендеры?

Подсказка: Подумайте, как выделить каждую строку в отдельный мемоизированный компонент.

Ответ:

- **Вынести строку в компонент и обернуть в `React.memo`**
- Переписать таблицу на `div`
- Использовать `setInterval`
- Перевести всё на `jQuery`

🔗 Шаг 06:

Миссия 6

Федор обернул компонент в `React.memo`, но оптимизации не произошло. В чём может быть причина?

Подсказка: Вспомните, как `React` сравнивает пропсы по умолчанию.

Ответ:

- **Передаются новые ссылки на объекты/массивы**
- `React.memo` работает только с классами
- Компонент слишком маленький
- Используется `useEffect`

🧭 Шаг 07:

Миссия 7

Настя хочет использовать `useMemo`. Для чего он не предназначен?

Подсказка: Этот хук оптимизирует вычисления, но не управляет состоянием.

Ответ:

- **Для хранения состояния приложения**
- Для мемоизации тяжёлых вычислений
- Для предотвращения пересоздания массивов
- Для оптимизации фильтрации

🧭 Шаг 08:

Миссия 8

Сопоставьте инструмент и его назначение.

Подсказка: Думайте о том, что именно мемоизируется — компонент, функция или вычисление.

Ответ:

- **React.memo** — Мемоизация функционального компонента
- **useMemo** — Мемоизация вычисленного значения
- **useCallback** — Мемоизация функции
- **Profiler** — Анализ производительности

🧭 Шаг 09:

Миссия 9

Соотнесите проблему и её оптимизационное решение.

Подсказка: Каждой проблеме соответствует определённый приём.

Ответ:

- **Тяжёлые вычисления** — `useMemo`
- **Функции пересоздаются** — `useCallback`
- **Компоненты рендерятся без изменений** — `React.memo`
- **Неясно, что замедляет** — `Profiler`

🧭 Шаг 10:

Миссия 10

Дима готовит лекцию по оптимизации. Сопоставьте термин и определение.

Подсказка: Вспомните популярные практики производительности в React.

Ответ:

- **Code Splitting** — Деление кода на части для подгрузки
- **Lazy Loading** — Отложенная загрузка компонентов
- **Virtualization** — Рендеринг только видимых элементов списка
- **Selector** — Выборка части состояния из стора

🧭 Шаг 11:

Миссия 11

Сопоставьте неверный подход и его исправление.

Подсказка: Вспомните, как избежать пересоздания массивов и функций.

Ответ:

- **Inline функции в JSX** — `useCallback`
- **Создание нового массива в пропсах** — `useMemo`
- **Рендер огромного списка сразу** — `Virtualization`
- **Передача ненужных пропсов** — Оптимизация через `React.memo`

🔮 Шаг 12:

Миссия 12

Саша объясняет студентам разницу. Сопоставьте утверждение и правильный инструмент.

Подсказка: Внимательно читайте, что именно нужно мемоизировать.

Ответ:

- **Избежать повторных вычислений** — `useMemo`
- **Сохранить функцию между рендерами** — `useCallback`
- **Предотвратить лишние рендеры компонента** — `React.memo`
- **Понять, какие части тормозят** — `Profiler`

🔮 Шаг 13:

Миссия 13

Валера проверяет проект. Соотнесите ошибку и её последствие.

Подсказка: Ошибки часто связаны с отсутствием мемоизации.

Ответ:

- **Не использовать `useMemo` при фильтрации** — Повторные вычисления при каждом рендере
- **Не использовать `React.memo` для карточек** — Лишние рендеры списка
- **Не использовать `useCallback` для коллбеков** — Дочерние компоненты рендерятся заново
- **Не анализировать через `Profiler`** — Сложно найти узкие места

🔮 Шаг 14:

Миссия 14

Упорядочьте шаги использования `React.memo`.

Подсказка: Сначала создаём компонент, потом оборачиваем и проверяем.

Ответ:

- Создать функциональный компонент
- Оборачивать его в `React.memo`
- Передать пропсы
- Проверить, что компонент не рендерится лишний раз

🔮 Шаг 15:

Миссия 15

Разместите шаги применения `useMemo` в нужном порядке.

Подсказка: Нужно определить тяжёлое вычисление и обернуть его.

Ответ:

- Определить вычисление, которое повторяется
- Оборачивать его в useMemo
- Передать зависимости
- Сравнить производительность до и после

 Шаг 16:

Миссия 16

Расставьте шаги оптимизации через useCallback.

Подсказка: Найдите место, где функции пересоздаются.

Ответ:

- Замечаем проблему с пересозданием функций
- Обернуть функцию в useCallback
- Передать зависимости
- Убедиться, что дочерние компоненты обновляются реже

 Шаг 17:

Миссия 17

Расположите шаги анализа производительности с помощью Profiler.

Подсказка: Важно сначала собрать данные, а потом оптимизировать.

Ответ:

- Открыть Profiler в DevTools
- Записать сессию работы
- Проанализировать, какие компоненты рендерятся лишней раз
- Применить React.memo или useMemo для оптимизации

 Урок 3 - Профилирование и устранение узких мест

 Шаг 01:

Миссия 1

Игорь заметил, что его React-приложение начинает тормозить при открытии страницы с большим списком пользователей. Что будет первым шагом для выявления узких мест?

Подсказка: Сначала нужно измерить производительность, прежде чем что-то оптимизировать.

Ответ:

- **Запустить профилирование с помощью DevTools**
- Сразу переписать весь компонент на class-компонент
- Удалить часть функциональности
- Установить все зависимости заново

 Шаг 02:

Миссия 2

Анна заметила, что при рендере сложной формы страница подвисает. Какой инструмент поможет ей найти проблемные компоненты?

Подсказка: Смотрите на визуальные показатели производительности и время рендеринга.

Ответ:

- **React Profiler**
- Redux DevTools
- Console.log во всех методах
- npm audit

🔮 **Шаг 03:**

Миссия 3

Дима заметил, что его список сообщений рендерится медленно из-за большого числа элементов. Как можно оптимизировать?

Подсказка: Не все элементы должны отображаться одновременно.

Ответ:

- **Использовать виртуализацию списка (react-window/react-virtualized)**
- **Разбить список на страницы или чанки**
- Рендерить все элементы сразу без оптимизации
- Сделать все элементы статическими изображениями

🔮 **Шаг 04:**

Миссия 4

Света профилирует приложение и видит, что компонент часто перерендеривается без необходимости. Что поможет уменьшить лишние рендеры?

Подсказка: Смотрите на мемоизацию и оптимизацию props.

Ответ:

- **Использовать React.memo для компонентов**
- **Применить useMemo или useCallback для вычислений и функций**
- Удалять ключи у элементов списка
- Ставить setTimeout на каждый рендер

🔮 **Шаг 05:**

Миссия 5

Миша замечает, что обновление состояния в родительском компоненте вызывает перерендер всех дочерних компонентов. Что поможет избежать лишних рендеров?

Подсказка: Обратите внимание на оптимизацию через мемоизацию и селекторы.

Ответ:

- **Использовать React.memo и правильные ключи**
- **Применить useCallback и useMemo**
- Удалять состояния у всех дочерних компонентов
- Переносить все состояния в Redux без анализа

🔮 **Шаг 06:**

Миссия 6

Катя замечает, что API-запросы блокируют UI при рендере. Что поможет избежать тормозов интерфейса?

Подсказка: Используйте асинхронность и разделение задач.

Ответ:

- **Выполнять API-запросы асинхронно с useEffect**
- **Показать skeleton UI, пока данные загружаются**
- Вызывать API синхронно в render
- Использовать alert для уведомления о загрузке

🔗 **Шаг 07:**

Миссия 7

Оля хочет уменьшить нагрузку на главный компонент, который рендерит много картинок. Что стоит попробовать?

Подсказка: Часто помогает ленивый рендер и мемоизация.

Ответ:

- **Применить lazy-loading изображений**
- **Использовать React.memo для компонентов картинок**
- Загрузить все картинки сразу в state
- Рендерить картинки через console.log

🔗 **Шаг 08:**

Миссия 8

Андрей использует React Profiler и видит, что компонент Button рендерится слишком часто. Как ему оптимизировать?

Подсказка: Рассмотрите мемоизацию функций и компонентов.

Ответ:

- **Обернуть Button в React.memo**
- **Использовать useCallback для передаваемых функций**
- Удалить все props у Button
- Переписать Button на div с onClick

🔗 **Шаг 09:**

Миссия 9

Сопоставьте показатели профилирования с их описанием.

Подсказка: Каждый показатель измеряет разный аспект производительности.

Ответ:

- **Actual Duration** — Время рендеринга компонента в миллисекундах
- **Base Duration** — Время рендеринга без учета оптимизаций
- **Render Count** — Количество повторных рендеров компонента
- **Commit Time** — Время внесения изменений в DOM

🔗 **Шаг 10:**

Миссия 10

Сопоставьте проблемную ситуацию с подходящей оптимизацией.

Подсказка: Подумайте, какие оптимизации уменьшают лишние рендеры или загрузку.

Ответ:

- **Большой список элементов** — Виртуализация списка

- **Частые ререндеры дочерних компонентов** — `React.memo` + `useCallback`
- **Тяжелые вычисления в render** — `useMemo` для вычислений
- **Асинхронные данные блокируют UI** — `Skeleton UI` + `useEffect`

🔮 Шаг 11:

Миссия 11

Сопоставьте инструмент профилирования с его назначением.

Подсказка: Каждый инструмент измеряет разные аспекты.

Ответ:

- **React Profiler** — Измеряет рендеринг компонентов и их производительность
- **Chrome DevTools Performance** — Показывает работу всего браузера и JS
- **Lighthouse** — Оценивает общую производительность страницы и рекомендации
- **Web Vitals** — Метрики скорости и отзывчивости страницы

🔮 Шаг 12:

Миссия 12

Сопоставьте подход оптимизации с его результатом.

Подсказка: Оптимизация должна уменьшать нагрузку на render или DOM.

Ответ:

- **useMemo** — Мемоизация вычислений для уменьшения повторных расчетов
- **useCallback** — Мемоизация функций для предотвращения лишних рендеров
- **React.memo** — Предотвращает повторный рендер компонента при неизменных props
- **Виртуализация списка** — Рендерит только видимые элементы списка

🔮 Шаг 13:

Миссия 13

Сопоставьте тип профилируемой задачи с инструментом.

Подсказка: Разные задачи требуют разных инструментов.

Ответ:

- **Проверка времени рендеринга компонента** — `React Profiler`
- **Общий JS и CSS performance** — `Chrome DevTools Performance`
- **Анализ отзывчивости страницы** — `Web Vitals`
- **Общая оценка скорости и SEO** — `Lighthouse`

🔮 Шаг 14:

Миссия 14

Расположите шаги профилирования React-компонента в правильном порядке:

Подсказка: Сначала измеряем, потом анализируем и исправляем.

Ответ:

- Запустить `React Profiler` и выполнить действия пользователя
- Собрать метрики рендеринга компонентов
- Выявить узкие места по длительности и числу рендеров
- Внедрить оптимизации и повторно проверить

Шаг 15:

Миссия 15

Расположите этапы оптимизации большого списка элементов в правильном порядке:

Подсказка: Начните с анализа, затем применяйте оптимизации.

Ответ:

- Проверить производительность через профайлер
- Применить виртуализацию или пагинацию
- Мемоизировать дочерние компоненты
- Повторно проверить производительность

Шаг 16:

Миссия 16

Расположите шаги оптимизации частых ререндеров по логике внедрения:

Подсказка: Сначала мемоизация функций, потом компонентов, потом проверка.

Ответ:

- Использовать `useCallback` для функций
- Применить `useMemo` для вычислений
- Оборачивать компоненты в `React.memo`
- Проверить результат профилированием

Шаг 17:

Миссия 17

Алексей заметил, что его приложение React стало тормозить при загрузке длинного списка пользователей. Что первым делом стоит использовать для анализа?

Подсказка: Начните с инструмента, который позволяет увидеть, какие компоненты рендерятся слишком часто.

Ответ:

- **React Profiler**
- `useMemo`
- `React Router`
- `setInterval`

Шаг 18:

Миссия 18

Ирина заметила, что каждый рендер её карточек пользователей занимает много времени. Какой инструмент поможет найти, какие именно компоненты тормозят?

Подсказка: Он показывает время рендера каждого компонента и количество повторных рендеров.

Ответ:

- **React Profiler**
- `useEffect`
- `Redux DevTools`
- `console.log`

Шаг 19:

Миссия 19

Сергей замечает, что список товаров рендерится медленно из-за повторных вычислений фильтров. Какой подход стоит применить?

Подсказка: Этот хук помогает запоминать результат вычислений между рендерами.

Ответ:

- **useMemo**
- useState
- useReducer
- useEffect

Урок 4 - Lazy-loading фич и компонентов

Шаг 01:

Миссия 1

Вася подключает новую фичу в своём React-приложении. Он хочет ускорить загрузку, чтобы главная страница открывалась быстрее. Что ему поможет?

Подсказка: Подумайте о разделении кода и подгрузке только нужного.

Ответ:

- **Использовать lazy-loading для фич**
- Подключить все компоненты в одном файле App.js
- Загрузить все библиотеки сразу при старте
- Использовать только class-компоненты

Шаг 02:

Миссия 2

Анна заметила, что при переходе на страницу профиля грузится много лишнего кода. Что из этого решает проблему?

Подсказка: Код подгружается только когда действительно нужен.

Ответ:

- **Импортировать компонент через React.lazy()**
- Загружать все роуты сразу при запуске
- Использовать только синхронные импорты
- Вынести компонент в глобальный стейт

Шаг 03:

Миссия 3

Игорь внедряет lazy-loading для модального окна. Но при первом открытии оно подгружается медленно. Что можно сделать?

Подсказка: Иногда лучше подгрузить чуть раньше, чем пользователь кликнет.

Ответ:

- **Использовать prefetch для ресурса**
- **Заранее подгрузить модальное окно при наведении на кнопку**
- Загрузить весь код модалки в App.js сразу
- Переписать модалку на class-компонент

🔗 Шаг 04:

Миссия 4

Света заметила, что при переходе по роутам у пользователей иногда появляется белый экран. Она использует `React.lazy()`. Что нужно добавить?

Подсказка: Вспомните, как React оборачивает динамические импорты.

Ответ:

- **Оборачивать компонент в**
- Переписать роуты на JSON
- Удалить lazy-loading
- Добавить `setTimeout` перед рендером

🔗 Шаг 05:

Миссия 5

Миша разрабатывает e-commerce платформу. Он хочет, чтобы корзина загружалась только при её открытии. Какой подход верный?

Подсказка: Вспомните про динамический импорт модулей.

Ответ:

- **Динамически импортировать компонент корзины**
- Заранее подключить корзину во все страницы
- Рендерить корзину скрытой на старте
- Хранить корзину в `localStorage`

🔗 Шаг 06:

Миссия 6

Катя подключает внешнюю библиотеку графиков. Она весит 2MB и замедляет загрузку главной страницы. Что лучше сделать?

Подсказка: Загружайте тяжёлые библиотеки только там, где они нужны.

Ответ:

- **Загрузить библиотеку лениво при открытии раздела с графиками**
- Импортировать библиотеку в `index.js`
- Сделать `npm install` во время работы приложения
- Дублировать код библиотеки в нескольких файлах

🔗 Шаг 07:

Миссия 7

Дима замечает, что пользователи редко заходят в раздел «Статистика». Что лучше сделать, чтобы приложение загружалось быстрее?

Подсказка: Оптимизируйте загрузку редко используемых страниц.

Ответ:

- **Отложить загрузку раздела «Статистика» через lazy-loading**
- Загрузить «Статистику» вместе с главной страницей
- Скопировать код статистики в несколько модулей
- Сделать раздел статическим HTML

🔗 Шаг 08:

Миссия 8

Оля экспериментирует с `Suspense`. Она хочет показать спиннер, пока грузится ленивый компонент. Как это сделать?

Подсказка: `Suspense` принимает один важный пропс.

Ответ:

- **Передать `fallback` со спиннером в `Suspense`**
- Написать `console.log` внутри компонента
- Добавить `alert` перед загрузкой
- Использовать `setInterval` для рендера

🔗 Шаг 09:

Миссия 9

Сопоставьте понятия `lazy-loading` с их описаниями.

Подсказка: Вспомните, как работают динамические импорты и `Suspense`.

Ответ:

- **`React.lazy`** — Функция для динамического импорта компонента
- **`Suspense`** — Компонент для отображения загрузки во время ленивого импорта
- **`Code splitting`** — Разделение приложения на более мелкие чанки
- **`Fallback`** — UI, который показывается во время загрузки

🔗 Шаг 10:

Миссия 10

Андрей внедряет `lazy-loading`. Сопоставьте ситуации с правильными решениями.

Подсказка: Подумайте, какие оптимизации подходят для разных страниц.

Ответ:

- **Редко используемая страница** — Загружать только при первом открытии
- **Крупная библиотека графиков** — Подключать лениво в модуле отчётов
- **Часто используемая кнопка** — Загрузить сразу при старте
- **Модальное окно** — Загружать динамически при вызове

🔗 Шаг 11:

Миссия 11

Сопоставьте инструменты оптимизации с их описанием.

Подсказка: Вспомните про `prefetch`, `preload` и `lazy`.

Ответ:

- **`Lazy-loading`** — Загрузка кода по запросу
- **`Prefetch`** — Фоновая загрузка ресурсов для будущего использования
- **`Preload`** — Приоритетная загрузка ресурса прямо сейчас
- **`Bundle splitting`** — Разделение кода на чанки для ускорения загрузки

🔗 Шаг 12:

Миссия 12

Марина анализирует загрузку приложения. Сопоставьте проблему с оптимизацией.

Подсказка: Подумайте, что помогает при большом bundle или медленных переходах.

Ответ:

- **Слишком большой initial bundle** — Code splitting
- **Долгая загрузка редко используемой страницы** — Lazy-loading
- **Медленный первый переход на модальку** — Prefetch модальки
- **Задержка при старте** — Удаление лишних зависимостей

🔮 Шаг 13:

Миссия 13

Сопоставьте компонент с правильным способом оптимизации загрузки.

Подсказка: Разные части приложения требуют разных стратегий загрузки.

Ответ:

- **Главный экран** — Загружается сразу
- **Админ-панель** — Lazy-loading при заходе
- **Редактор изображений** — Ленивая загрузка + prefetch
- **Форма логина** — Загружается синхронно

🔮 Шаг 14:

Миссия 14

Расставьте шаги внедрения lazy-loading в правильном порядке.

Подсказка: Вспомните: импорт → обёртка → Suspense → проверка.

Ответ:

- Импортировать компонент через React.lazy
- Оборачивать компонент в Suspense
- Указать fallback для загрузки
- Проверить работу через переход по роуту

🔮 Шаг 15:

Миссия 15

Пользователь открывает страницу профиля. Расставьте действия браузера и React при lazy-loading.

Подсказка: Сначала идёт запрос за кодом, потом показ fallback, затем рендер.

Ответ:

- Браузер отправляет запрос за чанком
- Показывается fallback UI
- Чанк загружается
- Компонент рендерится

🔮 Шаг 16:

Миссия 16

Расположите оптимизации по сложности внедрения (от простой к более сложной).

Подсказка: Начните с базовых стратегий.

Ответ:

- Использовать React.lazy

- Добавить Suspense с fallback
- Настроить prefetch ресурсов
- Реализовать динамический импорт с условиями

Модуль 8 - Практический проект

Урок 1 - Планирование структуры проекта по FSD

Шаг 01:

Миссия 1

Вы запускаете новый проект и хотите организовать структуру папок для большого приложения. Какой из подходов лучше всего использовать для работы с компонентами в FSD?

Подсказка: Организация компонентов по доменам позволит лучше разделить логику приложения.

Ответ:

- **Разделение по доменам (features)**
- Разделение по типам компонентов (UI, containers, utils)
- Хранение всех компонентов в одной папке 'components'
- Использование единой папки для всего приложения

Шаг 02:

Миссия 2

Ты на проекте, и нужно организовать взаимодействие между бизнес-логикой и компонентами интерфейса. Какой паттерн стоит использовать для управления состоянием?

Подсказка: Важно, чтобы состояние было легко передаваемо и управляемо на разных уровнях приложения.

Ответ:

- **Использование Redux или Context API для глобального состояния**
- Хранение состояния только в локальных компонентах
- Передача состояния через props на каждом уровне компонента
- Собственные решения без использования стандартных библиотек

Шаг 03:

Миссия 3

Проект растет, и теперь нужно разделить его на несколько модулей. Как ты будешь организовывать работу с API в каждом модуле?

Подсказка: Каждый модуль должен управлять своим состоянием и запросами к серверу самостоятельно.

Ответ:

- **Создание отдельных сервисов для каждого модуля с независимыми запросами к API**
- Создание одного глобального API-слоя для всего приложения
- Запросы должны обрабатываться только в компонентах UI
- Все запросы в одном большом сервисе, без разделения на модули

🔗 Шаг 04:

Миссия 4

📖 Задача: Сопоставьте архитектурные паттерны с их описаниями

Паттерны:

Ответ:

- Функциональный компонент
- Контейнерный компонент
- Сервис
- UI-компонент

Описание:

1. Реализует логику и рендерит UI, но не содержит бизнес-логики.
2. Представляет собой компоненты, которые занимаются взаимодействием с бизнес-логикой и передачей данных между компонентами.
3. Осуществляет общие операции, такие как взаимодействие с API.
4. Независимый компонент, который занимается исключительно рендерингом интерфейса.

🔗 Подсказка:

UI-компонент не должен содержать сложной логики, контейнерный компонент управляет данными, а сервисы отвечают за логику взаимодействия с внешними источниками данных.

- **Функциональный компонент** — Реализует логику и рендерит UI, но не содержит бизнес-логики.
- **Контейнерный компонент** — Представляет собой компоненты, которые занимаются взаимодействием с бизнес-логикой и передачей данных между компонентами.
- **Сервис** — Осуществляет общие операции, такие как взаимодействие с API.
- **UI-компонент** — Независимый компонент, который занимается исключительно рендерингом интерфейса.

🔗 Шаг 05:

Миссия 5

Ты работаешь в команде, и требуется соблюсти принцип SOLID при проектировании модулей. Какой принцип помогает создавать легко расширяемую и поддерживаемую систему?

Подсказка: Разделение ответственности между классами — ключ к поддерживаемости.

Ответ:

- **Принцип единой ответственности (SRP)**
- Принцип открытости/закрытости (OCP)
- Принцип подстановки Барбары Лисков (LSP)
- Принцип инверсии зависимостей (DIP)

🔗 Шаг 06:

Миссия 6

Вы хотите сделать ваше приложение масштабируемым и легко поддерживаемым. Какой подход к организации компонентов будет наиболее эффективным при использовании FSD?

Подсказка: Разделение на домены и независимая работа каждого модуля позволяет легко управлять изменениями и добавлять новые функции.

Ответ:

- **Разделение приложения на независимые модули (features)**
- Использование единой папки для всех компонентов и сервисов
- Разделение компонентов по типам (UI, containers, utils)
- Хранение всех файлов в одной большой папке

Шаг 07:

Миссия 7

 Задача: Сопоставьте архитектурные паттерны с их описаниями

Паттерны:

Ответ:

- Доменный слой
- Интерфейс
- Сервисный слой
- Презентер

Описание:

1. Отвечает за взаимодействие между слоями и обработку бизнес-логики.
2. Обрабатывает визуальное представление данных и взаимодействует с пользователем.
3. Предоставляет доступ к данным через API и бизнес-логику.
4. Определяет правила и логику взаимодействия с объектами домена.

Подсказка:

Каждый слой должен выполнять свою роль. Сервисный слой отвечает за логику работы с данными, а презентер — за отображение данных.

- **Доменный слой** — Определяет правила и логику взаимодействия с объектами домена.
- **Интерфейс** — Обрабатывает визуальное представление данных и взаимодействует с пользователем.
- **Сервисный слой** — Предоставляет доступ к данным через API и бизнес-логику.
- **Презентер** — Отвечает за взаимодействие между слоями и обработку бизнес-логики.

Шаг 08:

Миссия 8

Вы хотите сделать ваше приложение масштабируемым и легко поддерживаемым. Какой подход к организации компонентов будет наиболее эффективным при использовании FSD?

Подсказка: Разделение на домены и независимая работа каждого модуля позволяет легко управлять изменениями и добавлять новые функции.

Ответ:

- **Разделение приложения на независимые модули (features)**
- Использование единой папки для всех компонентов и сервисов
- Разделение компонентов по типам (UI, containers, utils)
- Хранение всех файлов в одной большой папке

Шаг 09:

Миссия 9

 Задача: Сопоставьте архитектурные паттерны с их описаниями

Паттерны:

Ответ:

- Доменный слой
- Интерфейс
- Сервисный слой
- Презентер

Описание:

1. Отвечает за взаимодействие между слоями и обработку бизнес-логики.
2. Обрабатывает визуальное представление данных и взаимодействует с пользователем.
3. Предоставляет доступ к данным через API и бизнес-логику.
4. Определяет правила и логику взаимодействия с объектами домена.

 **Подсказка:**

Каждый слой должен выполнять свою роль. Сервисный слой отвечает за логику работы с данными, а презентер — за отображение данных.

- **Доменный слой** — Определяет правила и логику взаимодействия с объектами домена.
- **Интерфейс** — Обрабатывает визуальное представление данных и взаимодействует с пользователем.
- **Сервисный слой** — Предоставляет доступ к данным через API и бизнес-логику.
- **Презентер** — Отвечает за взаимодействие между слоями и обработку бизнес-логики.

 **Шаг 10:**

Миссия 10

 **Задача:** Сопоставьте принципы архитектуры с их объяснениями

Принципы:

Ответ:

- Слабая связанность
- Повторное использование кода
- Сильная модульность
- Чистота интерфейсов

Объяснения:

1. Каждый компонент должен выполнять свою задачу и не зависеть от реализации других компонентов.
2. Код должен быть структурирован так, чтобы можно было повторно использовать отдельные части в других модулях.
3. Код должен быть разделен на независимые блоки, которые можно тестировать и поддерживать отдельно.
4. Интерфейсы должны быть максимально простыми и понятными для использования другими компонентами.

 **Подсказка:**

Принципы FSD направлены на то, чтобы код был чистым, поддерживаемым и легко масштабируемым.

- **Слабая связанность** — Каждый компонент должен выполнять свою задачу и не зависеть от реализации других компонентов.
- **Повторное использование кода** — Код должен быть структурирован так, чтобы можно было повторно использовать отдельные части в других модулях.

- **Сильная модульность** — Код должен быть разделен на независимые блоки, которые можно тестировать и поддерживать отдельно.
- **Чистота интерфейсов** — Интерфейсы должны быть максимально простыми и понятными для использования другими компонентами.

🔮 Шаг 11:

Миссия 11

📖 Задача: Сопоставьте принципы FSD с их значениями

Принципы:

Ответ:

- Модульность
- Слабая связанность
- Чистота интерфейсов
- Инкапсуляция

Значения:

1. Каждый компонент или модуль должен быть независимым и выполнять свою четко определенную задачу.
2. Интерфейсы компонентов должны быть максимально простыми и понятными для использования другими модулями.
3. Компоненты или модули должны быть связаны минимально, чтобы изменение одного компонента не затрагивало остальные.
4. Каждый компонент должен скрывать свою внутреннюю логику, позволяя взаимодействовать только снаружи через четко определенные интерфейсы.

🔮 Подсказка:

Основные принципы FSD помогают поддерживать приложение в хорошем состоянии и легко его расширять.

- **Модульность** — Каждый компонент или модуль должен быть независимым и выполнять свою четко определенную задачу.
- **Слабая связанность** — Компоненты или модули должны быть связаны минимально, чтобы изменение одного компонента не затрагивало остальные.
- **Чистота интерфейсов** — Интерфейсы компонентов должны быть максимально простыми и понятными для использования другими модулями.
- **Инкапсуляция** — Каждый компонент должен скрывать свою внутреннюю логику, позволяя взаимодействовать только снаружи через четко определенные интерфейсы.

🔮 Шаг 12:

Миссия 12

📖 Задача: Сопоставьте паттерны архитектуры с их основными характеристиками

Паттерны:

Ответ:

- FSD
- MVC
- MVVM
- Clean Architecture

Характеристики:

1. Компоненты разделяются по функциональности, каждый домен имеет свою собственную логику.
2. Основное внимание уделяется разделению представления, модели и контроллера.
3. Акцент на взаимодействие между моделью, представлением и моделью представления.
4. Каждый слой системы независим, и изменения в одном слое не затрагивают другие слои.

 Подсказка:

Каждый паттерн решает свою задачу в зависимости от особенностей проекта. FSD ориентирован на доменную архитектуру.

- **FSD** — Компоненты разделяются по функциональности, каждый домен имеет свою собственную логику.
- **MVC** — Основное внимание уделяется разделению представления, модели и контроллера.
- **MVVM** — Акцент на взаимодействие между моделью, представлением и моделью представления.
- **Clean Architecture** — Каждый слой системы независим, и изменения в одном слое не затрагивают другие слои.

 Шаг 13:

Миссия 13

Расположите этапы проектирования архитектуры в правильном порядке:

1. Планирование разделения функционала на домены
2. Определение зависимостей между модулями
3. Разработка компонентов с возможностью повторного использования
4. Поддержание слабой связанности между модулями

 Подсказка:

Подумайте о том, как сначала нужно спланировать структуру, затем обеспечить повторное использование компонентов и минимизировать зависимости.

Ответ:

- Планирование разделения функционала на домены
- Определение зависимостей между модулями
- Разработка компонентов с возможностью повторного использования
- Поддержание слабой связанности между модулями

 Урок 2 - Разработка фич и слоев

 Шаг 01:

Миссия 1

Вас попросили добавить новую кнопку в интерфейс приложения. Какой подход в React лучше всего использовать для этого?

Подсказка: Нужно подумать о том, как компонент должен быть обновлен при взаимодействии с пользователем.

Ответ:

- **Создать компонент с обработчиком события и рендерить кнопку**
- Использовать global state для этой кнопки
- Просто добавить HTML-код кнопки в главный файл
- Использовать события для глобального DOM

Шаг 02:

Миссия 2

Какой из методов жизненного цикла компонента используется для загрузки данных из API при монтировании компонента?

Подсказка: Этот метод вызывается только один раз при первом рендере компонента.

Ответ:

- **componentDidMount**
- render
- componentDidUpdate
- getSnapshotBeforeUpdate

Шаг 03:

Миссия 3

 Задача: Сопоставьте метод жизненного цикла с его правильным описанием

Методы:

Ответ:

- componentDidMount
- shouldComponentUpdate
- componentWillUnmount

Описание:

1. Этот метод вызывается перед удалением компонента из DOM.
2. Используется для предотвращения повторного рендера компонента при неизменении данных.
3. Вызывается сразу после монтирования компонента в DOM.

 **Подсказка:**

Обратите внимание на момент, когда каждый метод жизненного цикла должен быть вызван, и какие задачи он решает.

- **componentDidMount** — Вызывается сразу после монтирования компонента в DOM.
- **shouldComponentUpdate** — Используется для предотвращения повторного рендера компонента при неизменении данных.
- **componentWillUnmount** — Этот метод вызывается перед удалением компонента из DOM.

Шаг 04:

Миссия 4

Какой компонент является правильным решением для организации маршрутизации в вашем приложении?

Подсказка: Этот компонент предоставляет возможность перехода между страницами в вашем приложении.

Ответ:

- **BrowserRouter**
- Route
- Link
- NavLink

🔗 Шаг 05:

Миссия 5

📖 Задача: Сопоставьте компонент с его функцией

Компоненты:

Ответ:

- BrowserRouter
- Route
- Link

Функции:

1. Предоставляет возможность переходить по ссылкам между страницами.
2. Определяет, какой компонент должен быть отрендерен для заданного пути.
3. Организует маршрутизацию в приложении.

🔗 Подсказка:

Обратите внимание на функциональность каждого компонента в контексте маршрутизации.

- **BrowserRouter** — Организует маршрутизацию в приложении.
- **Route** — Определяет, какой компонент должен быть отрендерен для заданного пути.
- **Link** — Предоставляет возможность переходить по ссылкам между страницами.

🔗 Шаг 06:

Миссия 6

Расположите шаги в процессе создания приложения на React в правильном порядке:

1. Настройка среды разработки
2. Создание компонентов
3. Подключение библиотек
4. Настройка роутинга
5. Рендеринг приложения

🔗 Подсказка:

Сначала нужно подготовить окружение, а потом постепенно добавить функциональные части, такие как компоненты и роутинг.

Ответ:

- Настройка среды разработки
- Подключение библиотек
- Создание компонентов
- Настройка роутинга
- Рендеринг приложения

🔗 Шаг 07:

Миссия 7

Какой метод используется для обновления состояния в функциональном компоненте с использованием хука useState?

Подсказка: Этот метод изменяет значение состояния внутри компонента.

Ответ:

- **setState**

- useEffect
- render
- componentDidMount

🔗 Шаг 08:

Миссия 8

В каком случае лучше использовать хук useEffect?

Подсказка: Хук используется для выполнения побочных эффектов в компоненте, таких как запросы к API или подписки на события.

Ответ:

- **Для выполнения побочных эффектов, таких как запросы к API**
- Для изменения состояния компонента
- Для рендеринга JSX в компоненте
- Для определения стилей компонента

🔗 Шаг 09:

Миссия 9

📄 Задача: Сопоставьте хук с его назначением

Хуки:

Ответ:

- useState
- useEffect
- useContext

Назначение:

1. Позволяет работать с состоянием внутри функционального компонента.
2. Позволяет выполнять побочные эффекты, такие как запросы или подписки.
3. Позволяет передавать данные через компоненты, не передавая их явно через props.

🔗 Подсказка:

Подумайте, какой хук используется для работы с состоянием, какой — для побочных эффектов и какой — для передачи данных в дерево компонентов.

- **useState** — Позволяет работать с состоянием внутри функционального компонента.
- **useEffect** — Позволяет выполнять побочные эффекты, такие как запросы или подписки.
- **useContext** — Позволяет передавать данные через компоненты, не передавая их явно через props.

🔗 Шаг 10:

Миссия 10

Расположите ходы создания функционального компонента с хуками в правильном порядке:

1. Создание компонента с хуком useState
2. Подключение хука useEffect для выполнения побочных эффектов
3. Использование хука useContext для получения данных из контекста
4. Рендеринг компонента

🔗 Подсказка:

Необходимо сначала создать компонент с состоянием, затем подключить эффекты и контекст. И только потом рендерить компонент с данными.

Ответ:

- Создание компонента с хуком `useState`
- Подключение хука `useEffect` для выполнения побочных эффектов
- Использование хука `useContext` для получения данных из контекста
- Рендеринг компонента

 **Шаг 11:**

Миссия 11

Какой хук позволяет работать с контекстом и получать доступ к данным, переданным через `Context.Provider` ?

Подсказка: Этот хук позволяет получить данные без необходимости прокидывать их через все промежуточные компоненты.

Ответ:

- **`useContext`**
- `useState`
- `useReducer`
- `useEffect`

 **Шаг 12:**

Миссия 12

Какой хук используется для обновления состояния компонента с учетом предыдущего состояния?

Подсказка: Он позволяет работать с состоянием, обновляя его на основе текущего значения.

Ответ:

- **`useReducer`**
- `useState`
- `useEffect`
- `useRef`

 **Шаг 13:**

Миссия 13

Какой хук в React используется для создания рефа и доступа к DOM-элементам?

Подсказка: Этот хук позволяет сохранять ссылку на DOM-элемент и манипулировать им.

Ответ:

- **`useRef`**
- `useState`
- `useEffect`
- `useReducer`

 **Шаг 14:**

Миссия 14

Расположите шаги реализации контекста в React в правильном порядке:

1. Создание Context с помощью `React.createContext`

2. Оборачивание компонентов в Context.Provider
3. Использование useContext для получения данных из контекста

 Подсказка:

Контекст нужно создать и передать через провайдер, а данные можно извлечь с помощью хука.

Ответ:

- Создание Context с помощью React.createContext
- Оборачивание компонентов в Context.Provider
- Использование useContext для получения данных из контекста

 Шаг 15:

Миссия 15

 **Задача:** Сопоставьте хук с его возможностями

Хуки:

Ответ:

- useRef
- useState
- useEffect

Возможности:

1. Позволяет создавать ссылки на DOM-элементы.
2. Позволяет изменять состояние компонента.
3. Позволяет выполнять побочные эффекты, такие как запросы или подписки.

 Подсказка:

Рефы, состояние и побочные эффекты — каждый хук решает свою задачу в компоненте.

- **useRef** — Позволяет создавать ссылки на DOM-элементы.
- **useState** — Позволяет изменять состояние компонента.
- **useEffect** — Позволяет выполнять побочные эффекты, такие как запросы или подписки.

 Шаг 16:

Миссия 16

Что из следующего является основным преимуществом использования React?

Подсказка: Это позволяет эффективно обновлять и рендерить только те части интерфейса, которые были изменены.

Ответ:

- **Виртуальный DOM**
- Серверный рендеринг
- Сторонние библиотеки
- Обширная документация

 Шаг 17:

Миссия 17

Расположите шаги реализации нового функционала в React-приложении в правильном порядке:

1. Анализ требований и планирование архитектуры
2. Реализация компонентов с использованием хуков

3. Тестирование компонентов
4. Интеграция с серверной частью (если необходимо)

Подсказка:

Сначала нужно спланировать, потом создать компоненты и протестировать их. Интеграция обычно идет после тестов.

Ответ:

- Анализ требований и планирование архитектуры
- Реализация компонентов с использованием хуков
- Тестирование компонентов
- Интеграция с серверной частью (если необходимо)

Шаг 18:

Миссия 18

Что делает компонент `React.StrictMode` в приложении React?

Подсказка: Этот компонент помогает выявить потенциальные проблемы в приложении во время разработки.

Ответ:

- **Выявляет проблемы в коде в режиме разработки**
- Ускоряет рендеринг приложения
- Оптимизирует работу с состоянием
- Автоматически обновляет компоненты

Урок 3 - Подключение state и routing

Шаг 01:

Миссия 1

Алекс создает приложение для заметок и хочет подключить глобальное состояние через Redux. Что он должен сделать в первую очередь?

Подсказка: Подумайте, с чего начинается интеграция state в приложение.

Ответ:

- **Создать store и подключить Provider к корневому компоненту**
- Начать писать UI-компоненты без store
- Добавить роутинг перед state
- Сразу подключить компоненты к API

Шаг 02:

Миссия 2

Маша работает над маршрутизацией в приложении. Какой компонент React Router отвечает за отображение страницы по пути URL?

Подсказка: Вспомните, какой компонент связывает путь и отображаемый компонент.

Ответ:

- **Route**
- Link
- BrowserRouter
- Switch

Шаг 03:

Миссия 3

Петя заметил, что его компонент Cart нужно обновлять при изменении глобального состояния. Как ему подключить компонент к Redux?

Подсказка: Подумайте, какой метод связывает компонент с store.

Ответ:

- **Использовать connect() или useSelector для доступа к state**
- Добавить state как prop вручную
- Создать новый store внутри компонента
- Вызывать dispatch в каждом рендере вручную

Шаг 04:

Миссия 4

Лена хочет, чтобы при переходе по ссылке страница обновлялась без полной перезагрузки. Какой компонент React Router ей нужен?

Подсказка: Подумайте, какой элемент отвечает за навигацию без перезагрузки.

Ответ:

- **Link**
- Route
- Switch
- BrowserRouter

Шаг 05:

Миссия 5

Дима хочет обновлять состояние корзины при добавлении товаров. Какой подход верный?

Подсказка: Подумайте, где должны храниться данные, которые нужны в разных компонентах.

Ответ:

- **Хранить состояние в Redux и обновлять через dispatch**
- Хранить состояние локально в Cart и передавать через props
- Создать глобальную переменную window.cart
- Не использовать state и перезагружать страницу вручную

Шаг 06:

Миссия 6

 Задача: Сопоставьте термин с его назначением при работе с state и routing.

Подсказка: Обратите внимание, какой компонент хранит состояние, а какой отвечает за маршрут.

Ответ:

- **Store** — Хранение глобального состояния приложения
- **Provider** — Подключение store к React-приложению
- **Route** — Отображение компонента по определенному пути URL
- **Link** — Навигация без перезагрузки страницы

🔦 Шаг 07:

Миссия 7

📖 Задача: Сопоставьте хук или функцию с её применением в React Redux.

Подсказка: Один хук выбирает данные, другой отправляет действия.

Ответ:

- **useSelector** — Получение состояния из Redux
- **useDispatch** — Отправка действий в Redux
- **connect** — Связывает компонент с store через HOC
- **Provider** — Оборачивает приложение и предоставляет store

🔦 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте термин с примером использования в проекте с routing.

Подсказка: Подумайте, где прописываются пути и куда они ведут.

Ответ:

- **BrowserRouter** — Оборачивает приложение и управляет историей URL
- **Route** — Определяет компонент для конкретного пути
- **Switch** — Выбирает первый Route, который совпадает с URL
- **Link** — Ссылка для навигации по маршрутам

🔦 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте часть Redux с её функцией в приложении.

Подсказка: State — это данные, actions — это события.

Ответ:

- **State** — Хранение данных приложения
- **Action** — Объект, описывающий изменение состояния
- **Reducer** — Функция, обновляющая state на основе action
- **Dispatch** — Отправка action в store

🔦 Шаг 10:

Миссия 10

📖 Задача: Сопоставьте термин с примером использования в React Router.

Подсказка: Подумайте, какой компонент нужен для навигации, а какой — для отображения содержимого.

Ответ:

- **Route** — Отображение компонента по заданному пути URL
- **Link** — Навигация между страницами без перезагрузки
- **Redirect** — Перенаправление с одного пути на другой
- **Switch** — Выбор первого подходящего маршрута для отображения

🔦 Шаг 11:

Миссия 11

Расположите шаги подключения Redux в правильном порядке:

1. Создать store с редьюсерами
2. Оборачивать корневой компонент в Provider
3. Подключить компоненты к store через connect() или хуки
4. Использовать dispatch для изменения состояния

Подсказка: Начинаем с создания store, потом подключаем приложение, а далее используем state и actions.

Ответ:

- Создать store с редьюсерами
- Оборачивать корневой компонент в Provider
- Подключить компоненты к store через connect() или хуки
- Использовать dispatch для изменения состояния

Шаг 12:

Миссия 12

Расположите шаги маршрутизации страницы в правильном порядке:

1. Оборачиваем приложение в BrowserRouter
2. Создаем Route для каждого компонента
3. Добавляем ссылки через Link
4. Навигация и отображение компонентов при изменении URL

Подсказка: Сначала настройка маршрутизатора, затем создаем маршруты и ссылки, и только потом происходит переход.

Ответ:

- Оборачиваем приложение в BrowserRouter
- Создаем Route для каждого компонента
- Добавляем ссылки через Link
- Навигация и отображение компонентов при изменении URL

Шаг 13:

Миссия 13

Расположите шаги добавления функциональности корзины с Redux:

1. Определяем начальное состояние корзины
2. Создаем actions для добавления и удаления товаров
3. Создаем reducer для обработки actions
4. Подключаем компонент Cart к store через useSelector и useDispatch

Подсказка: Начало всегда с данных, затем создаем действия и редьюсер, и только потом используем в компоненте.

Ответ:

- Определяем начальное состояние корзины
- Создаем actions для добавления и удаления товаров
- Создаем reducer для обработки actions
- Подключаем компонент Cart к store через useSelector и useDispatch

Шаг 14:

Миссия 14

Расположите шаги подключения маршрутов для нескольких страниц:

1. Оборачиваем приложение в BrowserRouter
2. Создаем массив объектов с путями и компонентами
3. Проходим по массиву и создаем Route для каждого элемента
4. Добавляем навигацию через Link или NavLink

Подсказка: Подумайте о последовательности: маршрутизатор, маршруты, ссылки, переход.

Ответ:

- Оборачиваем приложение в BrowserRouter
- Создаем массив объектов с путями и компонентами
- Проходим по массиву и создаем Route для каждого элемента
- Добавляем навигацию через Link или NavLink

Шаг 15:

Миссия 15

Расположите шаги интеграции Redux и React Router в одно приложение:

1. Создаем store и редьюсеры
2. Оборачиваем приложение в Provider
3. Оборачиваем приложение в BrowserRouter
4. Создаем Route и подключаем компоненты к store

Подсказка: Подумайте, что должно быть обернуто сначала: store или маршрутизатор?

Ответ:

- Создаем store и редьюсеры
- Оборачиваем приложение в Provider
- Оборачиваем приложение в BrowserRouter
- Создаем Route и подключаем компоненты к store

Шаг 16:

Миссия 16

Расположите шаги создания страницы профиля пользователя с routing и state:

1. Создаем компонент Profile
2. Добавляем Route для пути '/profile'
3. Подключаем state пользователя через useSelector
4. Добавляем возможность обновления данных через dispatch

Подсказка: Сначала компонент, потом путь, затем получение и изменение данных.

Ответ:

- Создаем компонент Profile
- Добавляем Route для пути '/profile'
- Подключаем state пользователя через useSelector
- Добавляем возможность обновления данных через dispatch

Урок 4 - Тестирование и оптимизация

Шаг 01:

Миссия 1

Какой инструмент позволяет тестировать компоненты React изолированно?

Подсказка: Ищите инструмент, который имитирует DOM и позволяет запускать тесты без браузера.

Ответ:

- **Jest + React Testing Library**
- Chrome DevTools
- Redux DevTools
- npm install

Шаг 02:

Миссия 2

Какой метод помогает оптимизировать повторный рендер компонента?

Подсказка: Обратите внимание на мемоизацию и кеширование вычислений.

Ответ:

- **React.memo**
- useEffect без зависимостей
- useState для каждого элемента
- console.log

Шаг 03:

Миссия 3

Что из ниже перечисленного является примером unit-теста?

Подсказка: Проверьте, тестирует ли код отдельную функцию или компонент.

Ответ:

- **Проверка функции суммирования чисел**
- Тестирование полной страницы со всеми компонентами
- Загрузка данных с API в браузере
- Проверка стилей CSS

Шаг 04:

Миссия 4

 Сопоставьте понятия с их определением.

Подсказка: Подумайте о назначении каждого инструмента и термина.

Ответ:

- **Unit-тест** — Тестирует отдельную функцию или компонент
- **Integration-тест** — Проверяет взаимодействие нескольких компонентов
- **End-to-end тест** — Проверяет приложение от начала до конца
- **Snapshot-тест** — Сравнивает текущий рендер с сохраненным эталоном

Шаг 05:

Миссия 5

 Сопоставьте методы оптимизации с их действием.

Подсказка: Подумайте, какие методы помогают уменьшить лишние рендеры и вычисления.

Ответ:

- **React.memo** — Мемоизирует компонент, предотвращая повторный рендер
- **useMemo** — Мемоизирует вычисленное значение между рендерами
- **useCallback** — Мемоизирует функцию между рендерами
- **Lazy loading** — Загружает компонент только при необходимости

Шаг 06:

Миссия 6

 Сопоставьте типы тестов с примером.

Подсказка: Проверьте, какой тест проверяет поведение целой страницы, а какой — отдельной функции.

Ответ:

- **Unit-тест** — Проверка функции форматирования даты
- **Integration-тест** — Проверка работы формы с API
- **E2E тест** — Проверка покупки товара от добавления до оплаты
- **Snapshot-тест** — Проверка визуального отображения компонента

Шаг 07:

Миссия 7

Расположите шаги написания unit-теста в правильном порядке:

1. Импортируем функцию или компонент
2. Определяем входные данные и ожидаемый результат
3. Вызываем функцию или рендерим компонент
4. Проверяем результат с помощью assertions

Подсказка: Сначала импорт, затем подготовка данных, выполнение и проверка.

Ответ:

- Импортируем функцию или компонент
- Определяем входные данные и ожидаемый результат
- Вызываем функцию или рендерим компонент
- Проверяем результат с помощью assertions

Шаг 08:

Миссия 8

Расположите шаги оптимизации рендеринга компонента:

1. Выявляем, какой компонент часто перерендеривается
2. Применяем React.memo или useMemo
3. Проверяем работу с помощью React DevTools
4. Вносим исправления и проверяем производительность

Подсказка: Сначала идентифицируем проблему, затем применяем оптимизацию и проверяем результат.

Ответ:

- Выявляем, какой компонент часто перерендеривается
- Применяем React.memo или useMemo
- Проверяем работу с помощью React DevTools
- Вносим исправления и проверяем производительность

 Шаг 09:

Миссия 9

Расположите шаги создания snapshot-теста:

1. Рендерим компонент с тестовыми данными
2. Сохраняем snapshot в файл
3. Сравниваем новый рендер с сохраненным snapshot
4. Обновляем snapshot при необходимости

Подсказка: Сначала рендерим, затем сохраняем snapshot, потом проверяем и обновляем.

Ответ:

- Рендерим компонент с тестовыми данными
- Сохраняем snapshot в файл
- Сравниваем новый рендер с сохраненным snapshot
- Обновляем snapshot при необходимости

 Шаг 10:

Миссия 10

Расположите шаги интеграции unit-тестов в CI/CD:

1. Пишем тесты для функций и компонентов
2. Добавляем команду запуска тестов в pipeline
3. Запускаем pipeline и проверяем результаты
4. Исправляем ошибки, если тесты не прошли

Подсказка: Сначала пишем тесты, затем подключаем к CI/CD и проверяем.

Ответ:

- Пишем тесты для функций и компонентов
- Добавляем команду запуска тестов в pipeline
- Запускаем pipeline и проверяем результаты
- Исправляем ошибки, если тесты не прошли

 Шаг 11:

Миссия 11

Расположите шаги интеграции unit-тестов в CI/CD:

1. Пишем тесты для функций и компонентов
2. Добавляем команду запуска тестов в pipeline
3. Запускаем pipeline и проверяем результаты
4. Исправляем ошибки, если тесты не прошли

Подсказка: Сначала пишем тесты, затем подключаем к CI/CD и проверяем.

Ответ:

- Пишем тесты для функций и компонентов
- Добавляем команду запуска тестов в pipeline
- Запускаем pipeline и проверяем результаты
- Исправляем ошибки, если тесты не прошли

Шаг 12:

Миссия 12

Расположите шаги профилирования производительности React-приложения:

1. Открываем React DevTools Profiler
2. Запускаем действия пользователя, вызывающие рендеры
3. Анализируем длительность рендеров
4. Оптимизируем компоненты с долгими рендерами

Подсказка: Сначала собираем данные, затем анализируем и оптимизируем.

Ответ:

- Открываем React DevTools Profiler
- Запускаем действия пользователя, вызывающие рендеры
- Анализируем длительность рендеров
- Оптимизируем компоненты с долгими рендерами

Шаг 13:

Миссия 13

Расположите шаги оптимизации загрузки больших данных:

1. Выявляем компоненты, которые рендерят много элементов
2. Применяем виртуализацию списков (react-window/react-virtualized)
3. Проверяем скорость рендера и отзывчивость интерфейса
4. Дорабатываем оптимизацию при необходимости

Подсказка: Сначала анализируем узкое место, затем применяем подход, проверяем результат.

Ответ:

- Выявляем компоненты, которые рендерят много элементов
- Применяем виртуализацию списков (react-window/react-virtualized)
- Проверяем скорость рендера и отзывчивость интерфейса
- Дорабатываем оптимизацию при необходимости

Шаг 14:

Миссия 14

Расположите шаги lazy-loading компонентов:

1. Идентифицируем тяжелые компоненты
2. Используем React.lazy и Suspense
3. Проверяем, что компоненты подгружаются при необходимости
4. Оптимизируем fallback UI при загрузке

Подсказка: Сначала выделяем кандидатов на ленивую загрузку, затем подключаем lazy и проверяем UX.

Ответ:

- Идентифицируем тяжелые компоненты
- Используем React.lazy и Suspense
- Проверяем, что компоненты подгружаются при необходимости
- Оптимизируем fallback UI при загрузке

 Шаг 15:

Миссия 15

Расположите шаги проверки производительности useEffect:

1. Определяем эффекты с долгой обработкой
2. Добавляем console.time и console.timeEnd для измерений
3. Сравниваем время выполнения с разными зависимостями
4. Оптимизируем зависимости и код внутри useEffect

Подсказка: Сначала измеряем время работы, потом анализируем и оптимизируем.

Ответ:

- Определяем эффекты с долгой обработкой
- Добавляем console.time и console.timeEnd для измерений
- Сравниваем время выполнения с разными зависимостями
- Оптимизируем зависимости и код внутри useEffect

 Шаг 16:

Миссия 16

Расположите шаги оптимизации рендеринга таблиц с большим количеством данных:

1. Разбиваем таблицу на страницы (pagination)
2. Используем виртуальный список для рендеринга видимых строк
3. Проверяем время рендеринга каждой страницы
4. Оптимизируем стили и минимизируем лишние перерендеры

Подсказка: Сначала делаем разбиение, потом рендерим только видимое и проверяем производительность.

Ответ:

- Разбиваем таблицу на страницы (pagination)
- Используем виртуальный список для рендеринга видимых строк
- Проверяем время рендеринга каждой страницы
- Оптимизируем стили и минимизируем лишние перерендеры

 Шаг 17:

Миссия 17

Расположите шаги улучшения скорости загрузки API данных:

1. Определяем часто используемые данные
2. Применяем кэширование на уровне компонента или запроса
3. Проверяем уменьшение количества сетевых запросов
4. Обновляем стратегию кэширования при необходимости

Подсказка: Сначала выявляем горячие точки, затем кэшируем и проверяем эффективность.

Ответ:

- Определяем часто используемые данные
- Применяем кэширование на уровне компонента или запроса
- Проверяем уменьшение количества сетевых запросов
- Обновляем стратегию кэширования при необходимости